MASTER DSA - 2

Version 1.0

Contents:

I. **LINKED LIST:**

1. **Reverse a linked list**
2. **Middle element of linked list**
3. **Merge two sorted linked list**
4. **Remove nth node from back of linked list**
5. **Add two no.s as linked list**
6. **Find intersection point of Y linked list**
7. **Detect a cycle in linked list**
8. **Find the starting point of loop of linked list**
9. **Reverse in group of k**
10. **Check linked list is palindrome or not**
11. **Flattening of linked list**
12. **Rotated a linked list**
13. **Clone a linked list using random and next pointer**

II. **TREES:**

1. **Inorder Traversal of Binary Tree**
2. **Preorder Traversal of Binary Tree**
3. **Postorder Traversal of Binary Tree**
4. **LeftView of Binary Tree**
5. **BottomView of BT**
6. **TopView of BT**
7. **Preorder Inorder Postorder Traversals in One Traversal.**
8. **Vertical order traversal**
9. **Root to node path in BT**
10. **Max width of BT**
11. **Level order traversal of BT**
12. **Min depth of BT**
13. **Diameter of BT**
14. **Check if balanced BT or not**
15. **LCA of BT**
16. **Check if 2 BT are identical**
17. **Zigzag traversal of BT**
18. **Boundary traversal of BT**
19. **Maximum path sum of BT**
20. **Construct the BT from in order to preorder**
21. **Construct Binary Tree from Inorder and Postorder**
22. **Symmetric Binary Tree**
23. **Flatten Binary Tree to LinkedList**

24. **Check if Binary Tree is the mirror of itself or not**
25. **Check for Children Sum Property**
26. **Construct Binary Tree from Preorder and Postorder Traversal**

III. **Binary Search Tree:**

1. **Populate Next Right pointers of Tree**
2. **Search given Key in BST**
3. **Construct BST from given keys**
4. **Construct BST from preorder traversal**
5. **Check is a BT is BST or not**
6. **Find LCA of two nodes in BST**
7. **Find the inorder predecessor/successor of a given Key in BST.**
8. **Floor in a BST**
9. **Ceil in a BST**
10. **Find K-th smallest element in BST**
11. **Find K-th largest element in BST**
12. **Find a pair with a given sum in BST**
13. **BST iterator**
14. **Size of the largest BST in a Binary Tree**
15. **Serialize and deserialize Binary Tree**
16. **Delete Node in a BST**

IV. **Trees Miscellaneous**

1. **Binary Tree to Double Linked List**
2. **Find median in a stream of running integers.**
3. **K-th largest element in a stream.**
4. **Distinct numbers in Window.**
5. **K-th largest element in an unsorted array.**
6. **Flood-fill Algorithm**

V. **BIT MANIPULATION:**

1. **Gray Code**

VI. **DESIGN Q'S:**

1. **LRU**
   a. **USING CIRCULAR LINKED LIST**
   b. **USING LINKEDHASHMAP**
   c. **USING DOUBLY LINKED LIST**
   d. **USING HASHMAP**
2. **LFU**
3. **Design a Browser History**
4.

I. Linked Lists

1. Reverse a linked list

Problem Statement: Given the *head* of a singly linked list, write a program to reverse the linked list, and return *the head pointer to the reversed list*.

Examples: **Input Format**: head = [3,6,8,10] This means the given linked list is 3->6->8->10 with head pointer at node 3.

Result: Output = [10,6,8,3] This means, after reversal, the list should be 10->6->8->3 with the head pointer at node 10.

Method - 1 iterative

```
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode cur = head;
        while (cur != null){
            ListNode nxt = cur.next;
            cur.next = prev;
            prev= cur;
            cur = nxt;
        }
        return prev; } }
```

Method - 2 recursive

```
class Solution {
    public ListNode reverseList(ListNode head) {
        if (head == null || head.next == null) return head;
        ListNode cur = reverseList(head.next);
        head.next.next = head;
        head.next = null;
        return cur; }}
```

Method - 3 using stack

```
class Solution {
    public ListNode reverseList(ListNode head) {
        Stack<ListNode> stack = new Stack();
        ListNode node = head;
        while (node != null){
            ListNode cur = node;
            ListNode temp = cur.next;
            cur.next = null;
            stack.push(cur);
            node = temp;
        }
        ListNode ans1 = new ListNode(0);
        ListNode ans = ans1;
        while (! stack.isEmpty()){
```

```
            ans.next = stack.pop();
            ans = ans.next;
        }
        return ans1.next;
    }}
```

2. Middle element of linked list

Problem Statement: Given the **head** of a singly linked list, return *the middle node of the linked list*. If there are two middle nodes, return the second middle node.

Input Format:

(Pointer / Access to the **head** of a Linked list)

head = [1,2,3,4,5]

Result: [3,4,5]

(As we will return the middle of Linked list the further linked list will be still available)

Explanation: The middle node of the list is node 3 as in the below image.

Method 1: Naive Approach

Intuition: We can traverse through the Linked List while maintaining a count of nodes let's say in variable **n** , and then traversing for 2nd time for **n/2** nodes to get to the middle of the list.

```
ListNode middleNode(ListNode* head) {
       int n = 0;
       ListNode temp = head;
       while(temp) {
              n++;
            temp = temp.next;
       }
       temp = head;

       for(int i = 0; i < n / 2; i++) {
              temp = temp->next;
       }
       return temp; }
```

Solution 2: [Efficient] Tortoise-Hare-Approach

Unlike the above approach, we don't have to maintain nodes count here and we will be able to find the middle node in a single traversal so this approach is more efficient.

Intuition: In the Tortoise-Hare approach, we increment slow ptr by 1 and fast ptr by 2, so if take a close look fast ptr will travel double than that of the slow pointer. So when the fast ptr will be at the end of Linked List, slow ptr would have covered half of Linked List till then. So slow ptr will be pointing towards the middle of the Linked List.

Approach:

1. Create two pointers slow and fast and initialise them to a head pointer.
2. Move slow ptr by one step and simultaneously fast ptr by two steps until fast ptr is NULL or next of fast ptr is NULL.
3. When the above condition is met, we can see that the slow ptr is pointing towards the middle of Linked List and hence we can return the slow pointer.

```
    ListNode middleNode(ListNode* head) {
        ListNode slow = head, fast = head;
        while (fast && fast->next)
            slow = slow.next,
            fast = fast.next.next;
        return slow;
    }
```

3. Merge two sorted linked list

Problem Statement: Given two singly linked lists that are sorted in increasing order of node values, merge two sorted linked lists and return them as a sorted list. The list should be made by splicing together the nodes of the first two lists.

Input Format :

l1 = {3,7,10}, l2 = {1,2,5,8,10}

Output : {1,2,3,5,7,8,10,10}

Method - 1 copying to the other linked list and putting the elements from each respectively.

Method - 2 in place doing the stuff

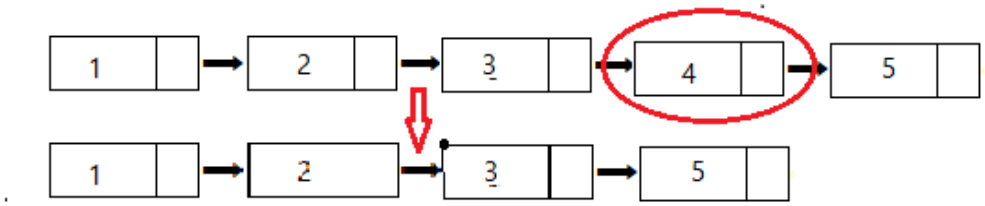
```
class Solution {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode h = new ListNode(0),  s = h;
        while (l1 != null && l2 != null){
            if (l1.val <= l2.val){
                s.next = l1;
```

```
                l1 = l1.next;
            }else{
                s.next = l2;
                l2 = l2.next;
            }
            s = s.next;
        }
        if (l1 != null){
            s.next = l1;
        }else if (l2 != null){
            s.next = l2;
        }
        return h.next;}}
```

4. Remove nth node from back of linked list

Problem Statement: *Given a linked list, and a number N. Find the Nth node from the end of this linkedlist, and delete it. Return the head of the new modified linked list.*

```
Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]
```



Method - 1

Intuition: We can traverse through the Linked List while maintaining a count of nodes, let's say in variable ***count*** , and then traversing for the 2nd time for (n – ***count***) nodes to get to the nth node of the list.

```
Method - 2 Two pointer approach
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode start = new ListNode();
        start.next = head;
        ListNode fast = start;
        ListNode slow = start;

        for(int i = 0; i < n; ++i)
            fast = fast.next;

        while(fast.next != null) {
            fast = fast.next;
            slow = slow.next;
        }

        slow.next = slow.next.next;

        return start.next; }}
```

5. Add two no.s as linked list

Given the heads of two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

Input Format:

num1 = 342, num2 = 564

l1 = [2,4,3] & l2 = [5,6,4]

Result: sum = 807; L = [7,0,8]

Explanation: Since the digits are stored in reverse order, reverse the numbers first to get the or
original number and then add them as → 342 + 465 = 807.

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(), temp = dummy;
        int carry = 0;
        while( l1 != null || l2 != null || carry == 1) {
            int sum = 0;
            if(l1 != null) {
                sum += l1.val;
                l1 = l1.next;
```

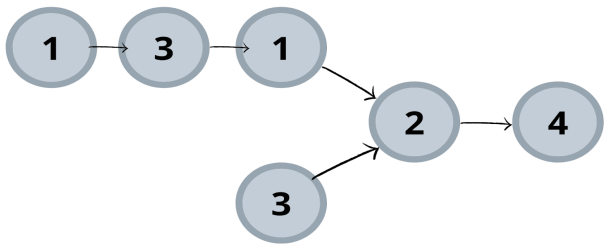
```
            }

            if(l2 != null) {
                sum += l2.val;
                l2 = l2.next;
            }

            sum += carry;
            carry = sum / 10;
            ListNode node = new ListNode(sum % 10);
            temp.next = node;
            temp = temp.next;
        }
        return dummy.next; }}
```

Time Complexity: O(max(m,n)). Assume that m and n represent the length of l1 and l2 respectively, the algorithm above iterates at most max(m,n) times. **Space Complexity**: O(max(m,n)). The length of the new list is at most max(m,n)+1.

6. Find intersection point of Y linked list

Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

```
Input: List 1 = [1,3,1,2,4], List 2 = [3,2,4]  Output: 2

Explanation: Here, both lists intersecting nodes start from node 2.
```



7. Detect a cycle in linked list

Given *head*, the head of a linked list, determine if the linked list has a cycle in it. There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Return *true* if there is a cycle in the linked list. Otherwise, return *false*.

Example 1:

Input: Head = [1,2,3,4]

Output: true

Explanation: Here, we can see that we can reach node at position 1 again by following the next pointer. Thus, we return true for this case.

Method - 1: Slow and Faster Pointer

```
static boolean cycleDetect(Node head) {
    if(head == null) return false;
    Node fast = head;
    Node slow = head;

    while(fast.next != null && fast.next.next != null) {
        fast = fast.next.next;
        slow = slow.next;
        if(fast == slow) return true;
    }
    return false; }
```

Method - 2: Hashing

Approach: We need to keep track of all the nodes we have visited till now so that once we visit the same node again we can say that a cycle is detected. The process is as follows:

- Use a hash table for storing nodes.
- Start iterating through the lists.
- If the current node is present in the hash table already, this indicates the cycle is present in the linked list and returns true.
- Else move insert the node in the hash table and move ahead.
- If we reach the end of the list, which is NULL, then we can say that the given list does not have a cycle in it and hence we return false.

```
static boolean cycleDetect(Node head) {
    HashSet <Node> hashTable=new HashSet<>();
```

```
    while (head != null) {
        if (hashTable.contains(head)) return true;
        hashTable.add(head);
        head = head.next;
    }
    return false;}
```

8. Find the starting point of loop of linked list

Given the head of a linked list, return *the node where the cycle begins. If there is no cycle, return* null.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that the tail's next pointer is connected to (0-indexed). It is -1 if there is no cycle. Note that pos is not passed as a parameter.

Do not modify the linked list.

Example 1:

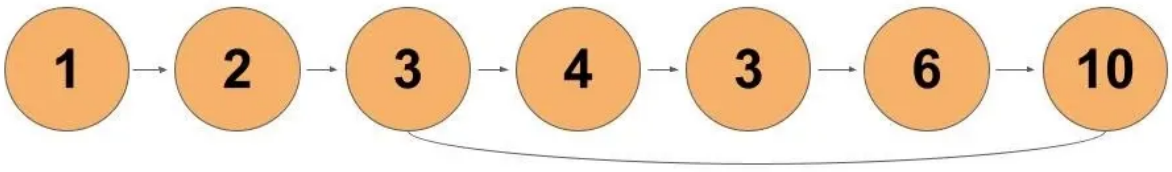
Input:

```
head = [1,2,3,4,3,6,10]
```

Output:

```
tail connects to node index 2
```

Explanation:



Method - 1 fast and slow pointer

```
static Node detectCycle(Node head) {
    if(head == null||head.next == null) return null;

    Node fast = head;
    Node slow = head;
    Node entry = head;

    while(fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;

        if(slow == fast) {
            while(slow != entry) {
                slow = slow.next;
                entry = entry.next;
            }
            return slow;
        }
    }
    return null; }
```

Method - 2 using Hash

```
static Node detectCycle(Node head) {
    HashSet<Node> st=new HashSet<>();
    while(head != null) {
        if(st.contains(head)) return head;
        st.add(head);
        head = head.next;
    }
    return null; }
```

9. Reverse in group of k

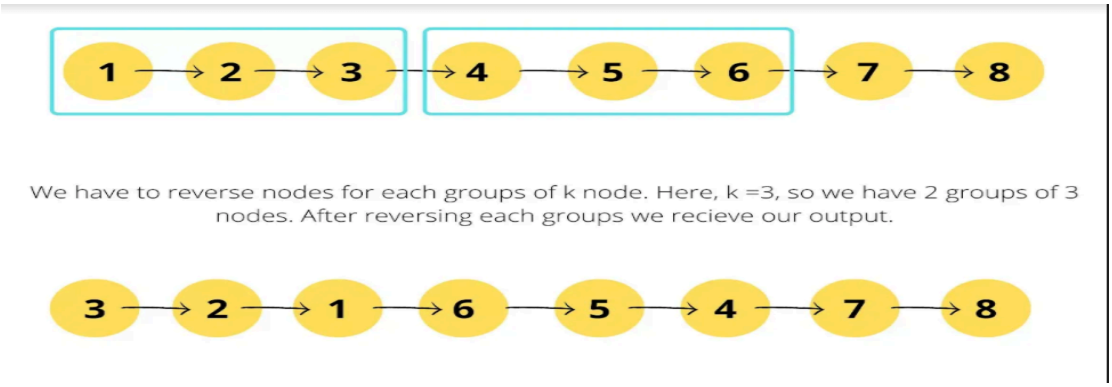
Given the head of a linked list, reverse the nodes of the list k at a time, and return *the modified list*. k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

Example 1:

Input: head = [1,2,3,4,5,6,7,8] k=3

Output:head = [3,2,1,6,5,4,7,8]

Explanation:



The following steps are needed to arrive at the desired output.

- Create a dummy node. Point next to this node to head of the linked list provided.
- Iterate through the given linked list to get the length of the list.
- Create three pointer pre,cur and nex to reverse each group. Why? If we observe output, we can see that we have to reverse each group, apart from modifying links of group.
- Iterate through the linked list until the length of list to be processed is greater than and equal to given k.
- For each iteration, point cur to pre->next and nex to nex->next.
- Start nested iteration for length of k.
- For each iteration, modify links as following steps:-
- cur->next = nex->next
- nex->next = pre->next
- pre->next = nex
- nex = cur->next
- Move pre to cur and reduce length by k.

```
public Node reverseKNodes(Node head,int k) {
    if (head == null||head.next == null) return head;

    int length = lengthOfLinkedList(head);

    Node dummyHead = new Node(0);
    dummyHead.next = head;

    Node pre = dummyHead, cur, nex;

    while (length >= k) {
```

```
        cur = pre.next;
        nex = cur.next;
        for (int i = 1; i < k; i++) {
            cur.next = nex.next;
            nex.next = pre.next;
            pre.next = nex;
            nex = cur.next;
        }
        pre = cur;
        length -= k;
    }
    return dummyHead.next; }
```

Method - 2 my soln Recursive

```
class Solution {
    public ListNode reverseKGroup(ListNode node, int k)  {
        ListNode cur = node;
        int cntr = k;
        ListNode prev = null, nxt = cur;
        while (cntr > 0 && cur != null){
            nxt = cur.next;
            cur.next = prev;
            prev = cur;
            cur = nxt;
            cntr -= 1;
        }
        boolean cond = false;
        cntr = k;
        ListNode c = cur;
        while (cntr > 0 && c != null){
            cntr--;
            c = c.next;
        }
        if (cntr == 0) cond = true;
        if (!cond){
            node.next = cur;
        }
        if (nxt != null && cond){
            node.next = reverseKGroup(nxt, k);
        }
        return prev; }}
```

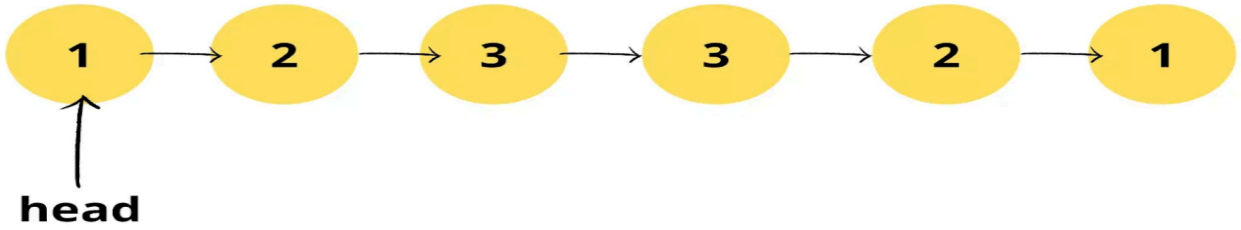
10. Check linked list is palindrome or not

Given the head of a singly linked list, return true if it is a palindrome.

Input: `head = [1,2,3,3,2,1]`

Output: `true`

Explanation: If we read elements from left to right, we get [1,2,3,3,2,1]. When we read elements from right to left, we get [1,2,3,3,2,1]. Both ways list remains same and hence, the given linked list is palindrome.



Method -1 using extra ds (dont' use)

```
static boolean isPalindrome(Node head) {
    ArrayList<Integer> arr=new ArrayList<>();
    while(head != null) {
        arr.add(head.num);
        head = head.next;
    }
    for(int i=0;i<arr.size()/2;i++)
        if(arr.get(i) != arr.get(arr.size()-i-1)) return false;
    return true;  }
```

Method - 2 reversing at middle and comparing

```
static Node reverse(Node ptr) {
    Node pre=null;
    Node nex=null;
    while(ptr!=null) {
        nex = ptr.next;
        ptr.next = pre;
        pre=ptr;
        ptr=nex;
    }
    return pre;
}

static boolean isPalindrome(Node head) {
    if (head==null||head.next==null) return true;

    Node slow = head, fast = head;

    while(fast.next!=null&&fast.next.next!=null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    slow.next = reverse(slow.next);
    slow = slow.next;
    Node dummy = head;
```

```
    while(slow!=null) {
        if(dummy.num != slow.num) return false;
        dummy = dummy.next;
        slow = slow.next;
    }
    return true; }
```

11. Flattening of linked list

Given a Linked List of size N, where every node represents a sub-linked-list and contains two pointers:

(i) a next pointer to the next node,

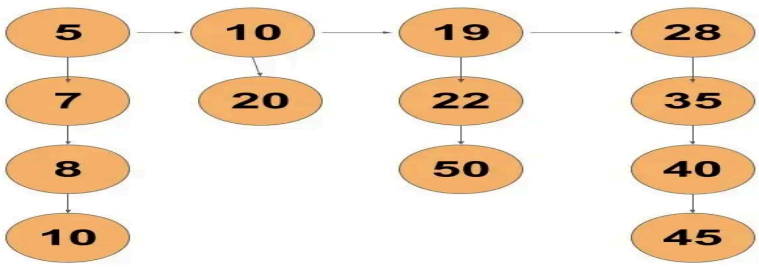
(ii) a bottom pointer to a linked list where this node is head.

Each of the sub-linked-list is in sorted order. Flatten the Link List such that all the nodes appear in a single level while maintaining the sorted order. Note: The flattened list will be printed using the bottom pointer instead of the next pointer.

Example 1:

Input: Number of head nodes = 4

Array holding length of each list with head and bottom = [4,2,3,4]
Elements of entire linked list =
[5,7,8,30,10,20,19,22,50,28,35,40,45]



Output: Flattened list = [5,7,8,10,19,20,22,28,30,35,40,45,50]

Explanation: Flattened list is the linked list consisting entire elements of the given list in sorted order.

Approach:

Since each list, followed by the bottom pointer, are in sorted order. Our main aim is to make a single list in sorted order of all nodes. So, we can think of a merge algorithm of merge sort. The process to flatten the given linked list is as follows:-

- We will recurse until the head pointer moves null. The main motive is to merge each list from the last.
- Merge each list chosen using the merge algorithm. The steps are
- Create a dummy node. Point two pointers, i.e, temp and res on dummy node. res is to keep track of dummy nodes and temp pointer is to move ahead as we build the flatten list.
- We iterate through the two chosen. Move head from any of the chosen list ahead whose current pointed node is smaller.
- Return the new flattened list found.

```
class Solution {
    Node mergeTwoLists(Node a, Node b) {
        Node temp = new Node(0), res = temp;
        while(a != null && b != null) {
            if(a.data < b.data) {
                temp.bottom = a;
                temp = temp.bottom;
                a = a.bottom;
            }
            else {
                temp.bottom = b;
                temp = temp.bottom;
                b = b.bottom;
            }
        }
        if(a != null) temp.bottom = a;
        else temp.bottom = b;
        return res.bottom;
    }
    Node flatten(Node root) {
            if (root == null || root.next == null)
                return root;
            // recur for list on right
            root.next = flatten(root.next);
            // now merge
            root = mergeTwoLists(root, root.next);
            // return the root
            // it will be in turn merged with its left
            return root;  }}
```

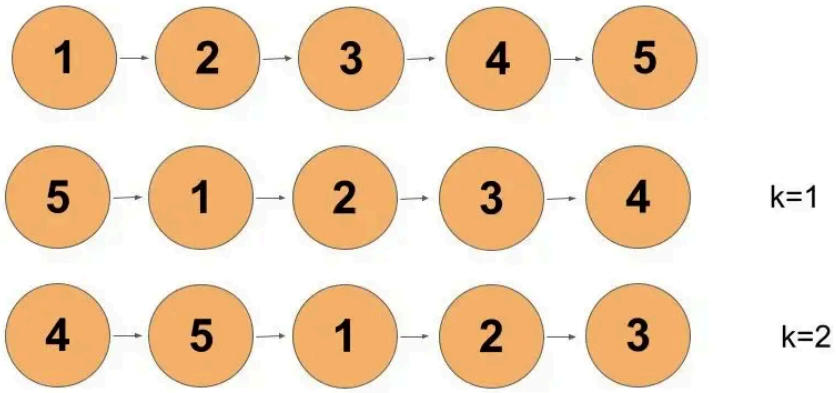
12. Rotated a linked list

Given the head of a linked list, rotate the list to the right by k places.

Example 1: Input: head = [1,2,3,4,5] k = 2

Output: head = [4,5,1,2,3]

Explanation: We have to rotate the list to the right twice.



Method - 1: Brute Force

```
static Node rotateRight(Node head, int k) {
    if (head == null || head.next == null) return head;
    for (int i = 0; i < k; i++) {
      Node temp = head;
      while (temp.next.next != null) temp = temp.next;
      Node end = temp.next;
      temp.next = null;
      end.next = head;
      head = end;
    }
    return head; }
```

Method - 2 optimal soln.

Let's take an example.

head = [1,2,3,4,5] k = 2000000000

If we see brute force approach, it will take O(5*2000000000) which is not a good time complexity when we can optimize it. We can see that for every k which is multiple of the length of the list, we get back the original list. Try to operate brute force on any linked list for k as multiple of the length of the list. This gives us a hint that for k greater than the length of the list, we have to rotate the list for k%length of the list. This reduces our time complexity.

Steps to the algorithm:-

- Calculate length of the list
- Connect the last node to the first node, converting it to a circular linked list.
- Iterate to cut the link of the last node and start a node of k%length of list rotated list.

```
static Node rotateRight(Node head,int k) {
    if(head == null||head.next == null||k == 0) return head;
    //calculating length
    Node temp = head;
    int length = 1;
    while(temp.next != null) {
        ++length;
        temp = temp.next;
    }
    //link last node to first node
    temp.next = head;
    k = k%length; //when k is more than length of list
    int end = length-k; //to get end of the list
    while(end--!=0) temp = temp.next;
    //breaking last node link and pointing to NULL
    head = temp.next;
    temp.next = null;

    return head;
}
```

13. Clone a linked list using random and next pointer

A linked list of length `n` is given such that each node contains an additional random pointer, which could point to any node in the list, or `null`. Construct a deep copy of the list. The deep copy should consist of exactly `n` brand new nodes, where each new node has its value set to the value of its corresponding original node. Both the `next` and `random` pointer of the new nodes should point to new nodes in the copied list such that the

pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

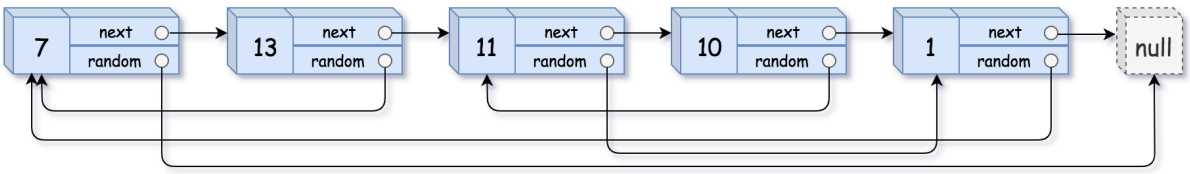
For example, if there are two nodes X and Y in the original list, where X.random --> Y, then for the corresponding two nodes x and y in the copied list, x.random --> y. Return *the head of the copied linked list*.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of [val, random_index] where:

- val: an integer representing Node.val
- random_index: the index of the node (range from 0 to n-1) that the random pointer points to, or null if it does not point to any node.

Your code will only be given the head of the original linked list.

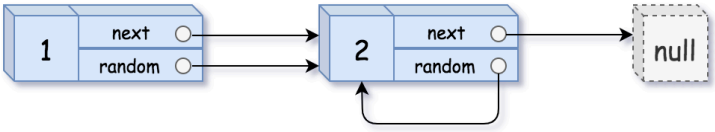
Example 1:



Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]

Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]

Example 2:



Input: head = [[1,1],[2,1]]

Output: [[1,1],[2,1]]

```
Method - 1 using 3 rounds
class Solution {
    public Node copyRandomList(Node head) {
        Node iter = head, next;
      // First round: make copy of each node,
      // and link them together side-by-side in a single list.
      while (iter != null) {
```

```
        next = iter.next;
        Node copy = new Node(iter.val);
        iter.next = copy;
        copy.next = next;
        iter = next;
      }
      // Second round: assign random pointers for the copy nodes.
      iter = head;
      while (iter != null) {
        if (iter.random != null) iter.next.random = iter.random.next;
        iter = iter.next.next;
      }
      // Third round: restore the original list, and extract the copy list.
      iter = head;
      Node newH = new Node(0);
      Node copy, copyIter = newH;
      while (iter != null) {
        next = iter.next.next;
        // extract the copy
        copy = iter.next;
        copyIter.next = copy;
        copyIter = copy;
        // restore the original list
        iter.next = next;
        iter = next;
      }
      return newH.next; }}
```

Method - 2 using hashMap

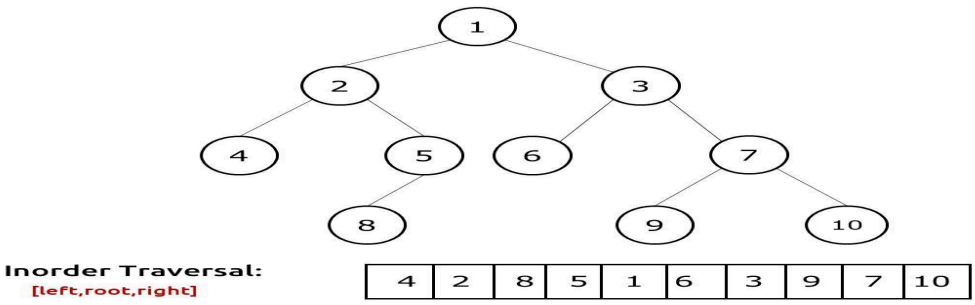
```
class Solution {
    public Node copyRandomList(Node head) {
        HashMap<Node, Node> map = new HashMap<Node, Node>();
        //copy list nodes into hash table
        Node p = head;
        while(p != null){
            map.put(p, new Node(p.val));
            p = p.next;
        }
        //deep copy using hash table
        Node q = head;
        while(q != null){
            map.get(q).next = map.get(q.next);
            map.get(q).random = map.get(q.random);
            q = q.next;
        }
        return map.get(head);} }
```

I. TREES:

1. Inorder Traversal of Binary Tree

Given a Binary Tree. Find and print the inorder traversal of Binary Tree.

Input:



Output: The inOrder Traversal is : 4 2 8 5 1 6 3 9 7 10

Method - 1 iterative

Intuition: In inorder traversal, the tree is traversed in this way: **root, left, right.** We first visit the left child, after returning from it we print the current node value, then we visit the right child. The fundamental problem we face in this scenario is that there is no way that we can move from a child to a parent. To solve this problem, we use an explicit stack data structure. While traversing we can insert node values to the stack in such a way that we always get the next node value at the top of the stack.

Approach: The algorithm approach can be stated as:

- We first take an explicit stack data structure and set an infinite loop.
- In every iteration we check whether our current node is pointing to NULL or not.
- If it is not pointing to null, we simply push the current value to the stack and move the current node to its left child.
- If it is pointing to NULL, we first check whether the stack is empty or not. If the stack is empty, it means that we have traversed the tree and we break out of the loop.
- If the stack is not empty, we pop the top value of the stack, print it and move the current node to its right child.
- → Stack is a Last-In-First-Out (LIFO) data structure, therefore when we encounter a node, we simply push it to the stack and try to find nodes on its left. When the current node points to NULL, it means that there is nothing left to traverse and we should move to the parent. This parent is always placed at the top of the stack. If the stack is empty, then we had already traversed the whole tree and should stop the execution.

```
static ArrayList < Integer > inOrderTrav(Node curr) {
        ArrayList < Integer > inOrder = new ArrayList < > ();
        Stack < Node > s = new Stack < > ();
        while (true) {
            if (curr != null) {
                s.push(curr);
                curr = curr.left;
            } else {
                if (s.isEmpty()) break;
                curr = s.peek();
                inOrder.add(curr.data);
                s.pop();
                curr = curr.right;
            }
        }
        return inOrder; }
```

Method - 2 Recursive

```
static void inOrderTrav(Node curr, ArrayList < Integer > inOrder) {
        if (curr == null)
            return;

        inOrderTrav(curr.left, inOrder);
        inOrder.add(curr.data);
        inOrderTrav(curr.right, inOrder);
    }
```

2. Preorder Traversal of Binary Tree

Method - 1 iterative

```
static ArrayList < Integer > preOrderTrav(Node curr) {
        ArrayList < Integer > preOrder = new ArrayList < Integer > ();
        if (curr == null)
            return preOrder;
        Stack < Node > s = new Stack < > ();
        s.push(curr);
        while (!s.isEmpty()) {
            Node topNode = s.peek();
            preOrder.add(topNode.data);
            s.pop();
            if (topNode.right != null)
                s.push(topNode.right);
            if (topNode.left != null)
                s.push(topNode.left);
        }
        return preOrder; }
```

Method - 2 recursive

```
 static void preOrderTrav(Node curr, ArrayList < Integer > preOrder) {
        if (curr == null)
            return;

        preOrder.add(curr.data);
        preOrderTrav(curr.left, preOrder);
        preOrderTrav(curr.right, preOrder);
    }
```

3. Postorder Traversal of Binary Tree

Method - 1 recursive

```
 static void postOrderTrav(Node curr, ArrayList < Integer > postOrder) {
        if (curr == null)
            return;

        postOrderTrav(curr.left, postOrder);
        postOrderTrav(curr.right, postOrder);
        postOrder.add(curr.data);
    }
```

Method - 2 iterative

```
static ArrayList < Integer > postOrderTrav(Node curr) {

        ArrayList < Integer > postOrder = new ArrayList < > ();
        if (curr == null) return postOrder;

        Stack < Node > s1 = new Stack < > ();
        Stack < Node > s2 = new Stack < > ();
        s1.push(curr);
        while (!s1.isEmpty()) {
            curr = s1.pop();
            s2.push(curr);
            if (curr.left != null)
                s1.push(curr.left);
            if (curr.right != null)
                s1.push(curr.right);
        }
        while (!s2.isEmpty()) {
            postOrder.add(s2.pop().data);
        }
        return postOrder;
    }
```

Method - 3 single stack

```
static ArrayList < Integer > postOrderTrav(Node curr) {
```

```
        ArrayList < Integer > postOrder = new ArrayList < > ();
        if (cur == null) return postOrder;

        Stack < Node > st = new Stack < > ();
        while (cur != null || !st.isEmpty()) {

            if (cur != null) {
                st.push(cur);
                cur = cur.left;
            } else {
                Node temp = st.peek().right;
                if (temp == null) {
                    temp = st.peek();
                    st.pop();
                    postOrder.add(temp.data);
                    while (!st.isEmpty() && temp == st.peek().right) {
                        temp = st.peek();
                        st.pop();
                        postOrder.add(temp.data);
                    }
                } else cur = temp;
            }
        }
        return postOrder;
    }
```

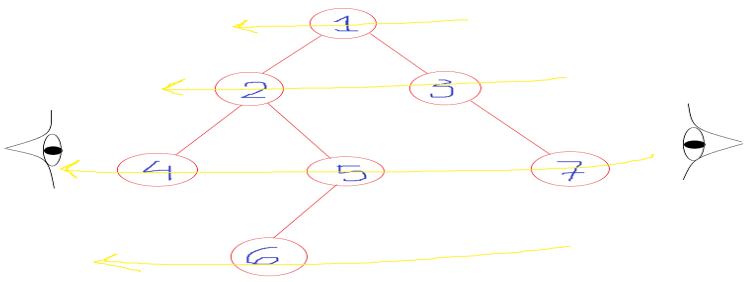
4. LeftView of Binary Tree

Given a Binary Tree, find the **Right/Left view** of it. The right view of a Binary Tree is a set of nodes visible when the tree is viewed from the **right** side. The left view of a Binary Tree is a set of nodes visible when the tree is viewed from the **left** side.

Intuition: We have to do a Recursive Level Order Traversal.

Root Right Left —-> for Right view Root Left Right ——> for Left view

The First Node of every level will be the Right View. The last Node of Every level will be the Left View.



Approach:

- Create an vector data structure inside both the left and the right side view function
- Call for the recursive _left and recursive_right function respectively with the (root,level,vector). Here level will be initially passed as 0.
- Return the vector.
- Now in the recursive_left function
    - If vector size is equal to the level then push_back its node value to the vector data structure.
    - Otherwise call recursive_left for (node->left,level+1,vector)
    - Call recursive_left for (node->right,level+1,vector)
- Now in the recursive_right function
    - If vector size is equal to the level then push_back its node value to the vector data structure.
    - Otherwise call recursive_right for (node->right,level+1,vector)
    - Call recursive_right for (node->left,level+1,vector)

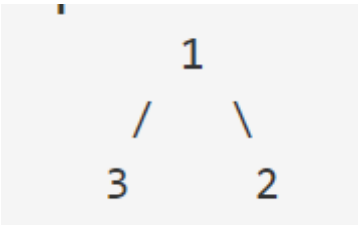
Tip: The Code for the Left and the Right View is almost identical.
In the Right view code first, you have to call the recursive function for the right then the left node
AND In the Right view code first, you have to call the recursive function for the Left than the right node

```
public class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> result = new ArrayList<Integer>();
        rightView(root, result, 0);
        return result; }
    public void rightView(TreeNode curr, List<Integer> result, int currDepth){
        if(curr == null){  return; }
        if(currDepth == result.size()){
            result.add(curr.val);  }
        rightView(curr.right, result, currDepth + 1);
        rightView(curr.left, result, currDepth + 1);
        }}
```

```
public class Solution {
    public List<Integer> lightSideView(TreeNode root) {
        List<Integer> result = new ArrayList<Integer>();
        leftView(root, result, 0);
        return result; }
    public void leftView(TreeNode curr, List<Integer> result, int currDepth){
        if(curr == null){  return; }
        if(currDepth == result.size()){
            result.add(curr.val); }
        leftView(curr.left, result, currDepth + 1);
        leftView(curr.right, result, currDepth + 1);  }}
```

5. BottomView of BT

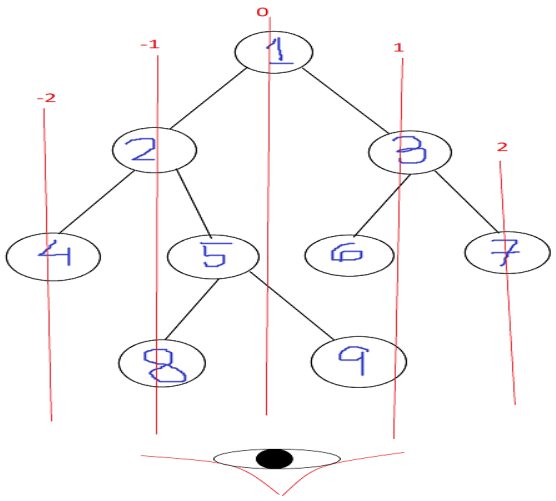
Problem Statement: Given a binary tree, print the bottom view from left to right. A node is included in the bottom view if it can be seen when we look at the tree from the bottom.



Example 1: `Input:`

`Output:` 3 1 2

Intuition: We can mark straight lines like in the image below and mark them with +ve and -ve indexes. The Last node of every line will be my Bottom view.



Approach:

- First we have to make a queue of pair which have nodes and their respective +ve and -ve indexes.
- Then we need a map data structure to store the lines and the nodes. This map will store the data in the form of sorted orders of keys(Lines).
- Here we will follow the level order traversal.
- Traverse through the nodes starting with root,0 and store them to the queue.
- Until the queue is not empty, store the node and line no. in 2 separate variables .
- Then store the line and the node->val to the map, if there will be any node value present that corresponds to a line in the map , it will be replaced by the new node value and by this we will get the last node of each line.
- Store the node->left and node->right along with their line nos. to the queue.
- Then print the node->val from the map

Tip: Here there is only 1 small difference from the Top View of the Tree. Here we don't need to check whether the node is previously present on the map or node before entering it. We have to replace the node of each line if that was previously present on the map.

```
class Solution {
    //Function to return a list containing the bottom view of the given tree.
    public ArrayList <Integer> bottomView(Node root)  {
        ArrayList<Integer> ans = new ArrayList<>();
        if (root == null) return ans;
        Map<Integer, Integer> map = new TreeMap<>();
        Queue<Node> q = new LinkedList<Node>();
        root.hd = 0;
        q.add(root);
        while(!q.isEmpty()) {
```

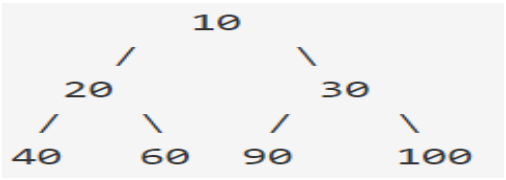
```
            Node temp = q.remove();
            int hd = temp.hd;
            map.put(hd, temp.data);
            if(temp.left != null) {
                temp.left.hd = hd - 1;
                q.add(temp.left);
            }
            if(temp.right != null) {
                temp.right.hd = hd + 1;
                q.add(temp.right);
            }  }
```

```
        for (Map.Entry<Integer,Integer> entry : map.entrySet()) {
            ans.add(entry.getValue());    }
        return ans; }}
```

6. TopView of BT

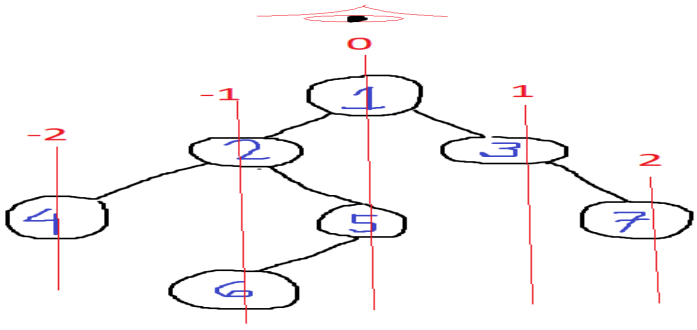
Problem Statement: Given below is a binary tree. The task is to print the top view of the binary tree. The top view of a binary tree is the set of nodes visible when the tree is viewed from the top.

Input:



Output: 40 20 10 30 100

Intuition: We can mark straight lines like in the image below and mark them with +ve and -ve indexes. The first node of every line will be my top view.



Approach:

- First we have to make a queue of pair which have nodes and their respective +ve and -ve indexes.
- Then we need a map data structure to store the lines and the nodes. This map will store the data in the form of sorted orders of keys(Lines).

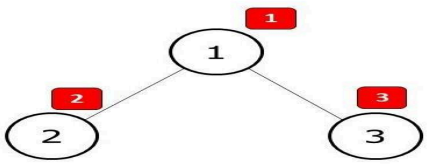
- Here we will follow the level order traversal.
- Traverse through the nodes starting with root,0 and store them to the queue.
- Until the queue is not empty, store the node and line no. in 2 separate variable .
- Then check if that line is present in the map or not
- If not present then store the line and the node->val to the map
- Otherwise store the node->left and node->right along with there line nos. to the queue.
- Then print the node->val from the map

```
class Solution {
    //Function to return a list of nodes visible from the top view
    //from left to right in Binary Tree.
    static ArrayList<Integer> topView(Node root) {
        ArrayList<Integer> ans = new ArrayList<>();
        if(root == null) return ans;
        Map<Integer, Integer> map = new TreeMap<>();
        Queue<Pair> q = new LinkedList<Pair>();
        q.add(new Pair(root, 0));
        while(!q.isEmpty()) {
            Pair it = q.remove();
            int hd = it.hd;
            Node temp = it.node;
            if(map.get(hd) == null) map.put(hd, temp.data);
            if(temp.left != null) {

                q.add(new Pair(temp.left, hd - 1));
            }
            if(temp.right != null) {

                q.add(new Pair(temp.right, hd + 1));
            }}

        for (Map.Entry<Integer,Integer> entry : map.entrySet()) {
            ans.add(entry.getValue());  }
        return ans;  }}
```

7. Preorder Inorder Postorder Traversals in One Traversal.

Preorder Inorder Postorder Traversals in One Traversal. Write a program to print Preorder, Inorder, and Postorder traversal of the tree in a single traversal.

Intuition: If we study the preorder, inorder and postorder traversals, we will observe a pattern. To understand this pattern, we take a simple example:



Preorder : 1

Inorder : 2

Postorder : 3

The number inside the red boxes is the visit when we print the node. In preorder traversal, we print a node at the first visit itself. Whereas, in inorder traversal at the first visit to a node, we simply traverse to the left child. It is only when we return from the left child and visit that node the second time, that we print it. Similarly, in postorder traversal, we print a node in its third visit after visiting both its children.

Approach:

The algorithm steps can be described as follows:

We take a stack data structure and push a pair<val, num> to it. Here Val is the value of the root node and num the visit to the node. Initially, the num is 1 as we are visiting the root node for the first time. We also create three separate lists for preorder, inorder and postorder traversals. Then we set an iterative loop to run till the time our stack is non-empty.

In every iteration, we pop the top of the stack (say, T). Then we check the second value(num) of T. Three cases can arise:

- **Case 1 : When num==1**
  - This means that we are visiting the node for the very first time, therefore we push the node value to our preorder list. Then we push the same node with num=2(for Case 2). After this, we want to visit the left child. Therefore we make a new pair Y(<val, num>) and push it to the stack (if there exists a left child). The val of Y is equal to the left child's node value and num is equal to 1.
- **Case 2 : When num==2**
  - This means that we are visiting the node for the second time, therefore on our second visit to the node, we push the node value to our inorder list. Then we push the same node with num=3(for Case 3). After this, we want to visit the right child. Therefore as in the first case, we check if there exists a right child or not. If there is, we push the right child and num value=1 as a pair to our stack.
- **Case 3 When num==3**
  - This means that we are visiting the node for the third time. Therefore we will push that node's value to our postorder list. Next, we simply want to return to the parent so we will not push anything else to the stack.

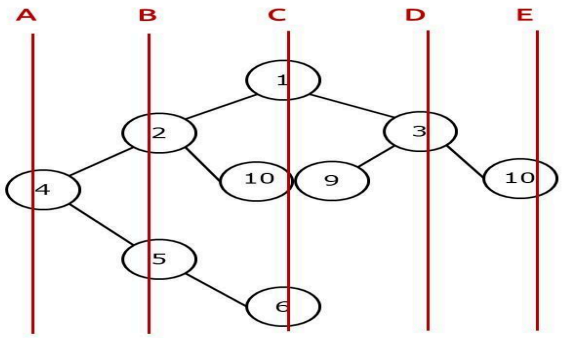
```
class Pair {
    TreeNode node;
    int num;
    Pair(TreeNode _node, int _num) {
```

```
        num = _num;
        node = _node;
    }
}
    public static void allTraversal(TreeNode root, List<Integer> pre,
                                    List<Integer> in , List <Integer> post) {
        Stack < Pair > st = new Stack < Pair > ();
        st.push(new Pair(root, 1));

        if (root == null) return;

        while (!st.isEmpty()) {
            Pair it = st.pop();

            // this is part of pre
            // increment 1 to 2
            // push the left side of the tree
            if (it.num == 1) {
                pre.add(it.node.val);
                it.num++;
                st.push(it);

                if (it.node.left != null) {
                    st.push(new Pair(it.node.left, 1));
                }
            }

            // this is a part of in
            // increment 2 to 3
            // push right
            else if (it.num == 2) { in .add(it.node.val);
                it.num++;
                st.push(it);

                if (it.node.right != null) {
                    st.push(new Pair(it.node.right, 1));
                }
            }
            // don't push it back again
            else {
                post.add(it.node.val);
            }
        }}
```

8. Vertical order traversal

Problem Description:

Vertical Order Traversal, as the name suggests is a traversal in which we divide the binary tree in verticals from left to right, and in every vertical, we print the nodes from top to bottom.



Vertical Order Traversal

A	4
B	2, 5
C	1, 9, 10, 6
D	3
E	10

Note: There can be duplicate values among the nodes (here 10 is repeated twice). If two or more nodes are overlapping at the same position(here 10 and 9), then they will be printed in ascending order.

Solution :

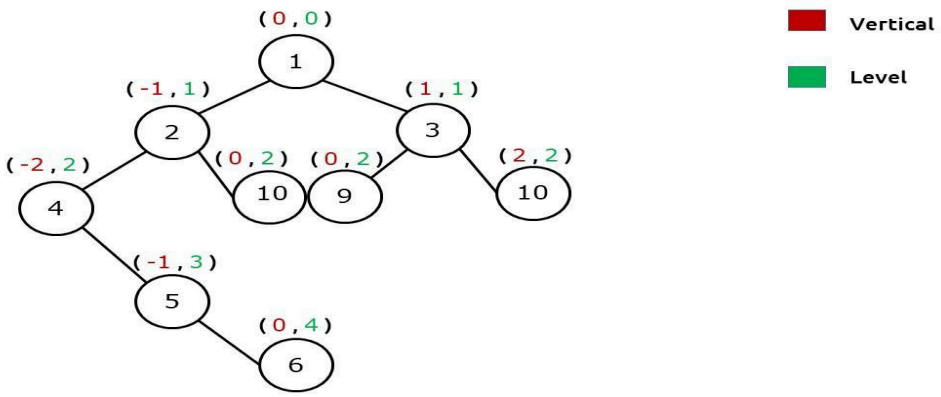
Intuition: First of all, we need to assign a vertical and a level to every node. Once we assign them, we need to figure out a suitable data structure to store them. This data structure should give us the nodes with left-side vertical first and in every vertical, top-level node should appear first.

Approach: We will perform a tree traversal and assign a vertical and level to every node. Based on this vertical and node, we store the node in our special data structure. For easy understanding, we break it into these steps:

Step-1: Assigning vertical and level to every node

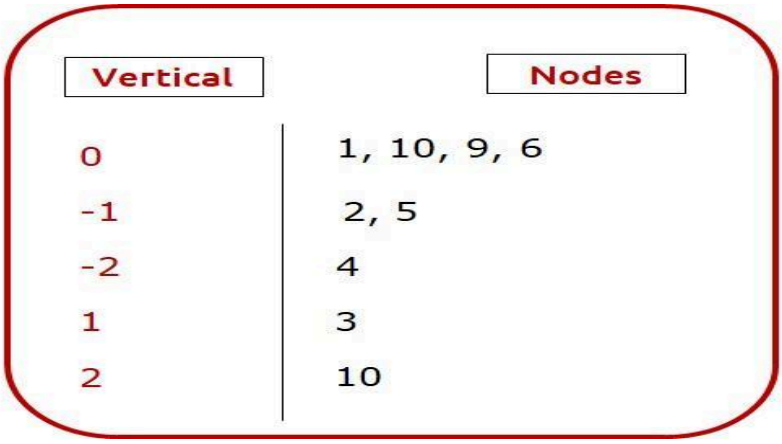
We can perform any tree traversal for this step. Here we are taking the example of level order traversal. In the initial step when we push the node to our queue, we will push two more variables to it, one for the vertical and one for the level (both initialized as zero). Now whenever we push the left child of the node, we decrease vertical by 1 and increase level by 1, whereas whenever we push the right child of the node, we increase both vertical and level by one.

Verticals and levels will be marked as follows:



Step-2: Storing Verticals and levels to our data structure

When we are assigning the verticals and levels, it is very important to store them in a suitable data structure. After assigning we have:



In the vertical order traversal, we need to print the nodes of the left vertical first, therefore in our example, nodes of -2 vertical will be the first to be printed. Therefore, we need a data structure that can store nodes according to their vertical value and give us the nodes of least values first. Hence we will use a **map** as it satisfies both criterias.

Now, we will explore X. In a single vertical we want to get the nodes by their levels. In our example, vertical 0 has nodes of three different levels, we will print level 0 first (node 1), then level 2 (node 9 and 10) and at last level 4 (node 6). Therefore as in the case of our verticals, we will again use **map** to store nodes level-wise inside the vertical. So, our data structure will become:

Now, we will explore Y. There can be a case that two or more nodes overlap at the same vertical and level, as with case of node 9 and node 10 at vertical 0 and level 2. In such a case we will print nodes with lesser value first. Therefore for every level, we need a data-structure which can store node values in a sorted order. Moreover, as duplicate values are allowed in our tree, our data structure needs to handle this well. This is acheived by using **multiset** **in C++**. Multiset is similar to a set which keeps elements in sorted order but also allows duplicates. **In Java**, we can use **priority queues** as it gives us the minimum value at the top.

Therefore, our final data structure will be:

Note: This is not the only choice for the data structure. You are free to choose any other data structure as well but the requirements remain the same. Instead of a multiset/priority queue, simple lists can also be used but then we will need to sort them in our last step.

Step 3: Printing vertical order traversal from our data structure

In the last step, we iterate over our verticals by using the data structure of step 2. In every iteration, we take a list(to store all nodes of that vertical) and again iterate over the levels. We then push the nodes of the level (stored in the multiset/priority queue) and push it to our vertical list. Once we iterate over all verticals we get our vertical order traversal.

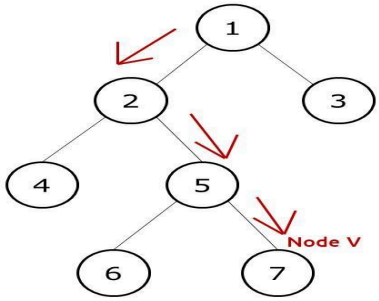
```
class Tuple {
    TreeNode node;
    int row;
    int col;
    public Tuple(TreeNode _node, int _level, int _ver) {
        node = _node;
        level = _level;
        ver = _ver;
    }
}
public class TUF { // row → level, col → ver
    public static List < List < Integer >> findVertical(TreeNode root) {
        TreeMap < Integer, TreeMap < Integer, PriorityQueue < Integer >>> map = new
TreeMap < > ();
        Queue < Tuple > q = new LinkedList < Tuple > ();
        q.offer(new Tuple(root, 0, 0));
        while (!q.isEmpty()) {
            Tuple tuple = q.poll();
            TreeNode node = tuple.node;
            int x = tuple.level, y = tuple.ver;
            // if (!map.containsKey(x)) {
            //    map.put(x, new TreeMap < > ());
            // }
             map.put(x, map.getOrDefault(x, new TreeMap<>());
            // if (!map.get(x).containsKey(y)) {
            //   map.get(x).put(y, new PriorityQueue < > ());
            // }
             map.get(x).put(y, map.get(x).getOrDefault(y, new PriorityQueue<>()));
              map.get(x).get(y).offer(node.data);

            if (node.left != null) {
                q.offer(new Tuple(node.left, x - 1, y + 1));
            }
            if (node.right != null) {
                q.offer(new Tuple(node.right, x + 1, y + 1));
            }
        }
        List < List < Integer >> list = new ArrayList < > ();
        for (TreeMap < Integer, PriorityQueue < Integer >> ys: map.values()) {
            list.add(new ArrayList < > ());
            for (PriorityQueue < Integer > nodes: ys.values()) {
                while (!nodes.isEmpty()) {
                    list.get(list.size() - 1).add(nodes.poll());
                }
            } }
        return list;}}
```

9. Root to node path in BT

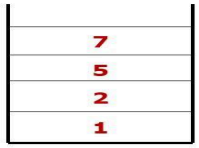
We are given a binary tree T and a node V. We need to print a path from the root of the tree to the node.

Note: No two nodes in the tree have the same data value. It is assured that the node V is present and a path always exists.

Solution & Intuition: First of all, we need to find the node V in our tree for which we need to find the path. We can use any depth-first traversal technique (preorder, inorder, postorder) in order to find the required node. If we look at the diagram below, we see that whenever we find the required node, its path is well present in our recursion call stack. We just need to figure out how we can use the recursive calls to print the required path.



Binary Tree T



Recursion call stack

Approach:

We will use an external list to store our path. This list will be passed by reference to our recursive function. Moreover, we can set the return value of our function as boolean, this will help us to know whether node V was found in a subtree or not.

The algorithm steps can be stated as follows:

- We pass the function with our root node, the path list and node V.
- For the base case, if root is pointing to NULL, we return false as clearly node V can't be found.
- Now we first push the node to our path list.
- Then we check whether the current node is the target node or not, if it is then no further execution is needed and we return to the parent function.

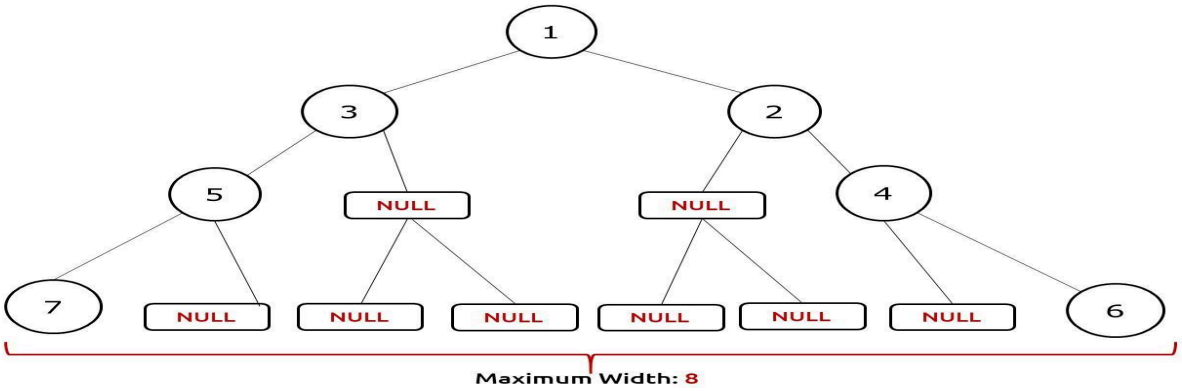
- If not, then we recursively call its left and right child to find the target node V. If any one of them returns true, it means we have found node V at lower levels and return true from the current function.
- If the value is not found at the current node and neither in any of the recursive calls, it means that the value is not present in the current sub-tree, therefore we pop out the current node from the path list and return false.

```
public class TUF {
    static boolean getPath(Node root, ArrayList < Integer > arr, int x) {
        // if root is NULL there is no path
        if (root == null)
            return false;

        // push the node's value in 'arr'
        arr.add(root.data);

        // if it is the required node return true
        if (root.data == x)
            return true;

        // else check whether the required node lies
        // in the left subtree or right subtree of the current node
        if (getPath(root.left, arr, x) ||
            getPath(root.right, arr, x))
            return true;

        // required node does not lie either in the
        // left or right subtree of the current node
        // Thus, remove current node's value from 'arr'and then return false
        arr.remove(arr.size() - 1);
        return false;
    }
```

10. Max width of BT

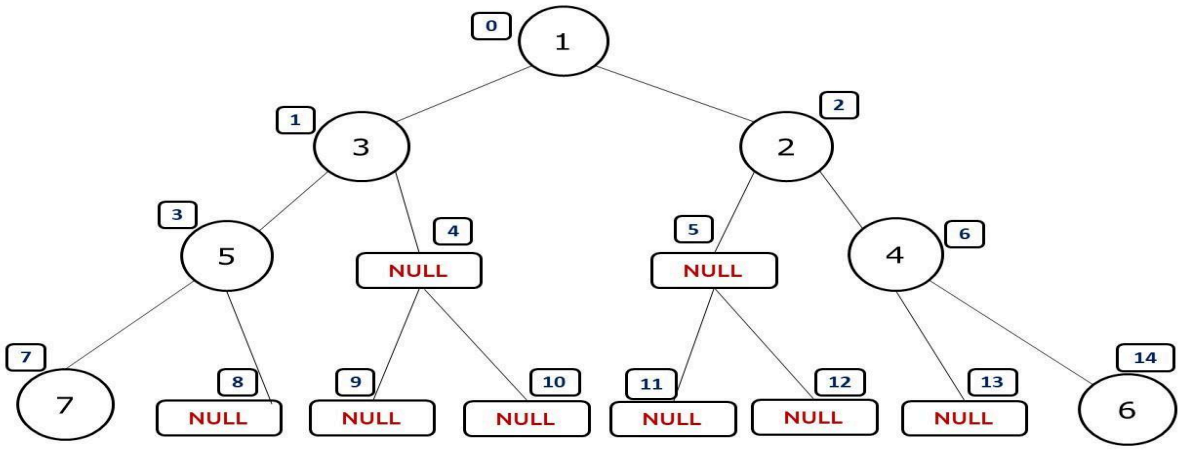
Problem Description: The maximum width of a binary tree is the maximum of all the level widths. Width for a level is defined as the maximum number of nodes between the leftmost and rightmost node of the level(including the end nodes and the null nodes between the end nodes).

Solution & Intuition: First of all, we need to understand the meaning of width at a level clearly. In the following image, we can see how the null nodes play an important role in the width calculation

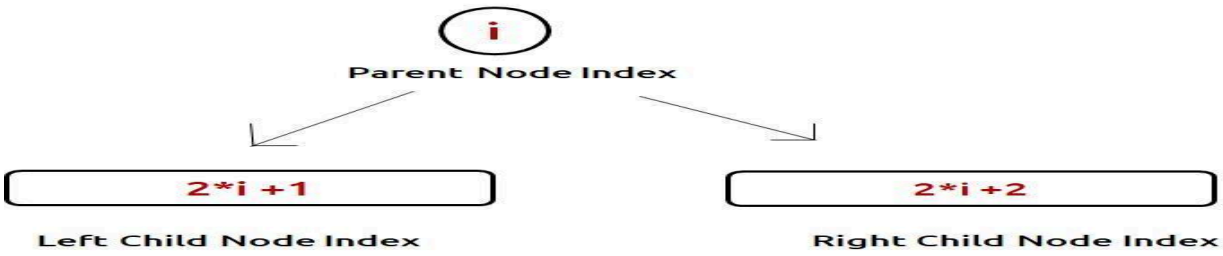


Now we see that the width is defined by the nodes of one particular level. Therefore we can use a level order traversal to traverse the tree and in every level, we try to find the leftmost and rightmost node of that level. To achieve this we would need a proper indexing strategy to uniquely index nodes of a level. Once we know the leftMost and rightMost nodes, width can be defined as (rightMost- leftMost +1).

Approach: We will perform a special level order traversal with two loops where inner loops traverse the nodes of a single level. This is to ensure that we can do our calculations once a single level is traversed. In the traversal, we will assign an index to a node. The indexing strategy is described as below:



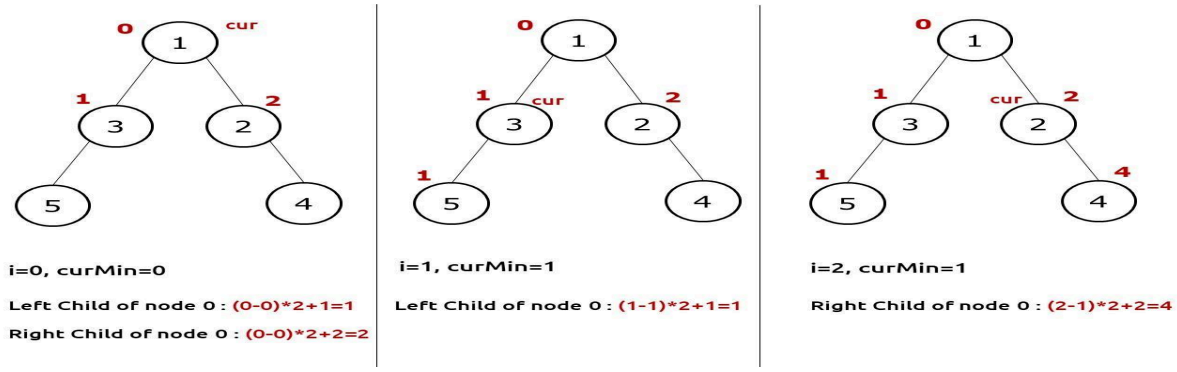
If we index the tree as shown above we can easily calculate the width of the tree as rightMostNode – leftMostNode +1. Then we can return the maximum width as our answer. To store the index, we can use a pair of values in our queue(that we use for level order traversal). If we are at index i, then its left and right child are(in 0-based indexing): 2*i+1 and 2*i+2 respectively. Please note that NULL nodes are not hampering the indexing in any way.



Prevention of Integer Overflow

This approach has a problem, as we are multiplying 2 to the current index, it can happen in a tree that we overshoot the bound of an integer. Therefore, we need to find a strategy to prevent it.

Before starting a level, we can store the left-most index in a variable(say curMin). Now whenever we assign the index for its children, we take the parent node index as (i-curMin) rather than i. The below illustration will clear the concept.



As our final answer is a range of nodes in a level, i,e rightMost- leftMost+1; this strategy will not affect the answer and at the same time prevent the integer overflow case.

The algorithm approach can be stated as:

- We take a queue and push the root node along with index 0.
- We traverse the tree using a level order traversal.
- In the level order traversal we set another loop to run for the size of the queue, so that we visit the same level nodes inside it.
- Before a level starts, we use a variable(say curMin) to store the index of the first node.
- We assign an index to every node, and to its children as described above.
- When the inner loop is at the first node of a level, we store its index in another variable(sayleftMost)
- When the inner loop is at the last node of a level, we store its index in another variable(say rightMost)

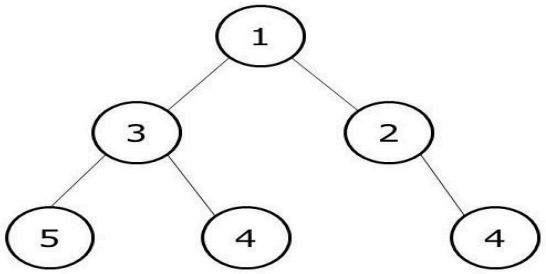
- After a level in the outer loop, we calculate the width of the level as (rightMost – leftMost +1).
- We return the maximum width as the answer.

```
class Solution {
    public static int widthOfBinaryTree(TreeNode root) {
        if(root == null) return 0;
        int ans = 0;
        Queue<Pair> q = new LinkedList<>();
        q.offer(new Pair(root, 0));
        while(!q.isEmpty()){
            int size = q.size();
            int mmin = q.peek().num;    //to make the id starting from zero
            int first = 0,last = 0;
            for(int i=0; i<size; i++){
                int cur_id = q.peek().num-mmin;
                TreeNode node = q.peek().node;
                q.poll();
                if(i==0) first = cur_id;
                if(i==size-1) last = cur_id;
                if(node.left != null)
                    q.offer(new Pair(node.left, cur_id*2+1));
                if(node.right != null)
                    q.offer(new Pair(node.right, cur_id*2+2));
            }
            ans = Math.max(ans, last-first+1);
        }
        return ans; }
```

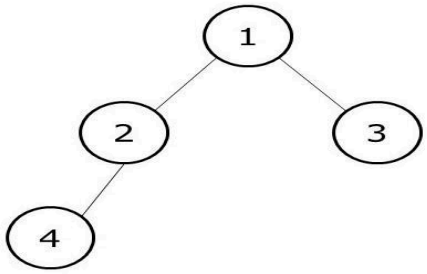
11. Max width of BT

Write a program to find the Maximum Width of A Binary Tree.

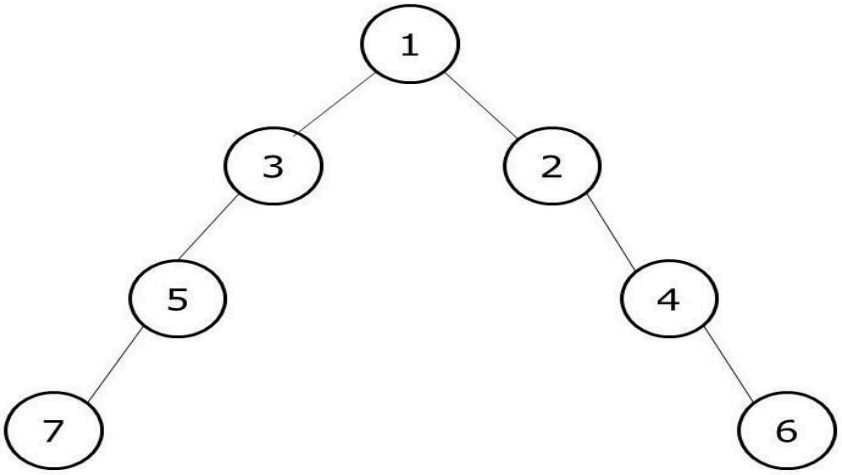
Examples:



Maximum Width: 4



Maximum Width: 2



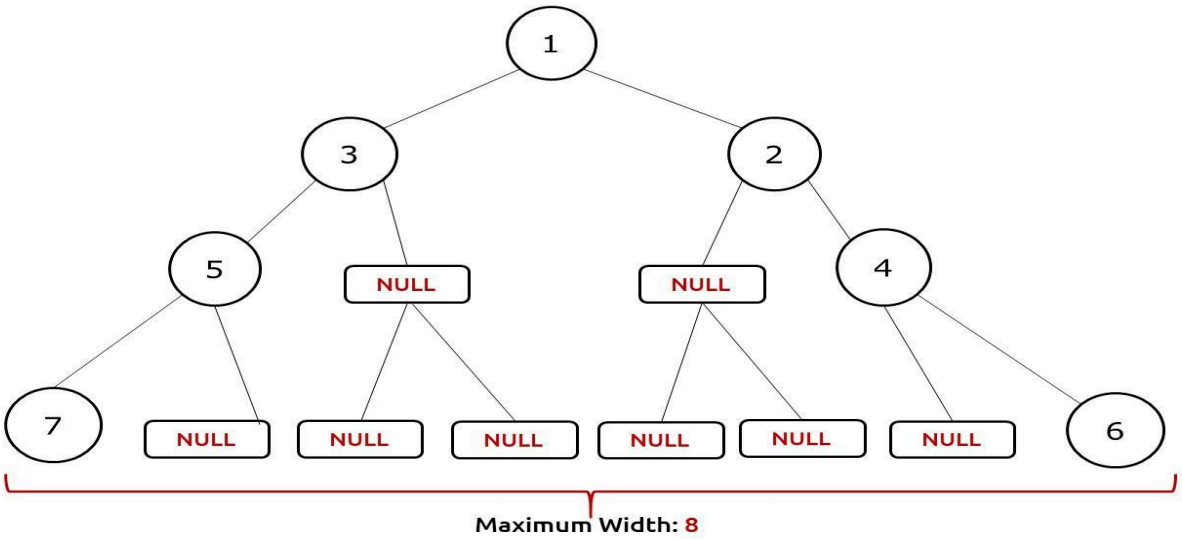
Maximum Width: 8

Problem Description: The maximum width of a binary tree is the maximum of all the level widths. Width for a level is defined as the maximum number of nodes between the leftmost and rightmost node of the level(including the end nodes and the null nodes between the end nodes).

Pre-req: Level Order Traversal

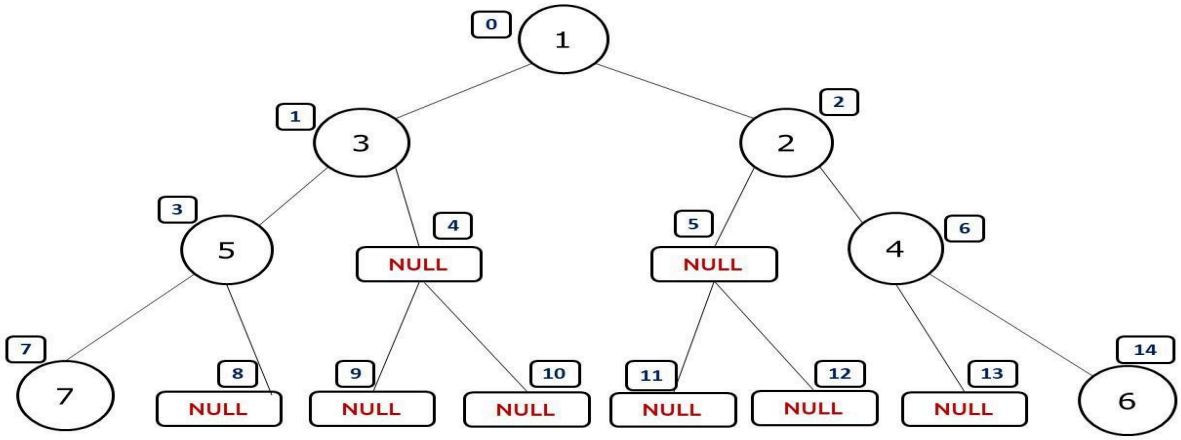
Solution :

Intuition: First of all, we need to understand the meaning of width at a level clearly. In the following image, we can see how the null nodes play an important role in the width calculation

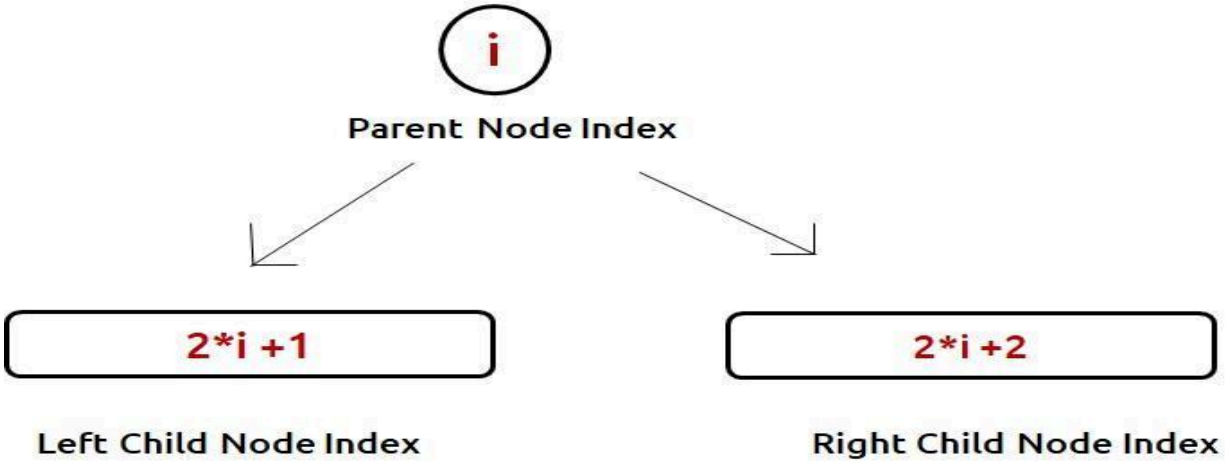


Now we see that the width is defined by the nodes of one particular level. Therefore we can use a level order traversal to traverse the tree and in every level, we try to find the leftmost and rightmost node of that level. To achieve this we would need a proper indexing strategy to uniquely index nodes of a level. Once we know the leftMost and rightMost nodes, width can be defined as (rightMost- leftMost +1).

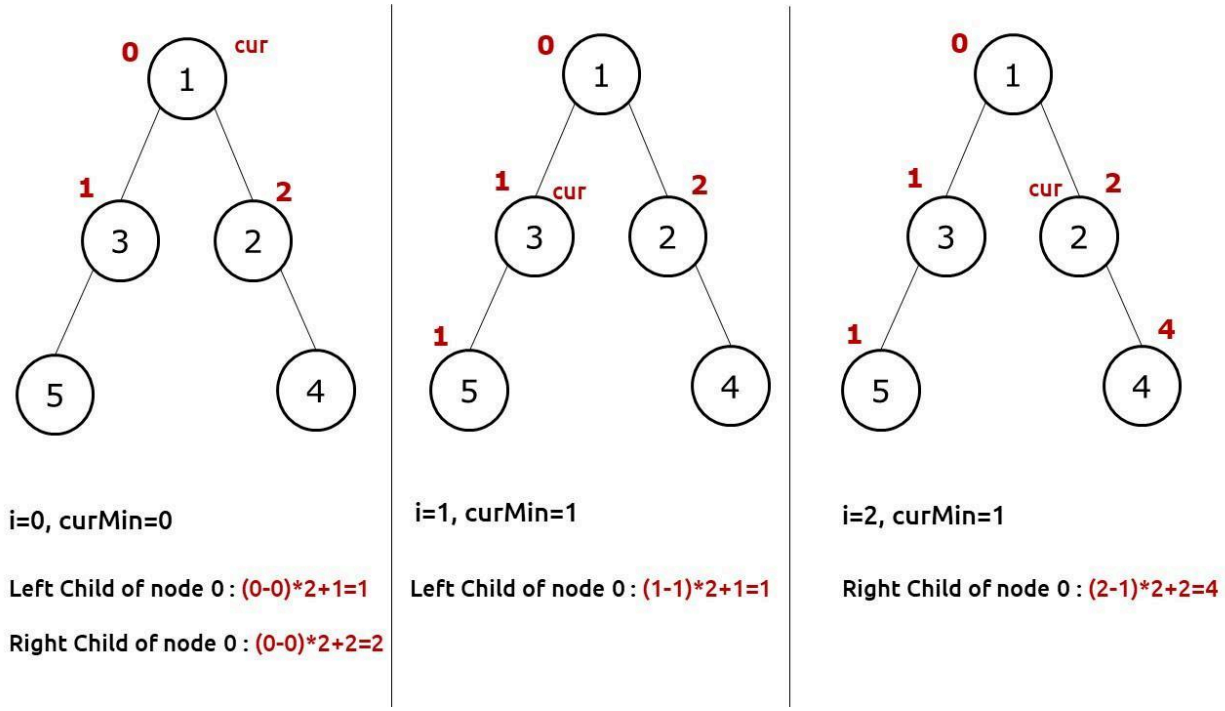
Approach: We will perform a special level order traversal with two loops where inner loops traverse the nodes of a single level. This is to ensure that we can do our calculations once a single level is traversed. In the traversal, we will assign an index to a node. The indexing strategy is described as below:



If we index the tree as shown above we can easily calculate the width of the tree as rightMostNode – leftMostNode +1. Then we can return the maximum width as our answer. To store the index, we can use a pair of values in our queue(that we use for level order traversal). If we are at index i, then its left and right child are(in 0-based indexing): 2*i+1 and 2*i+2 respectively. Please note that NULL nodes are not hampering the indexing in any way.



Prevention of Integer Overflow This approach has a problem, as we are multiplying 2 to the current index, it can happen in a tree that we overshoot the bound of an integer. Therefore, we need to find a strategy to prevent it.
Before starting a level, we can store the left-most index in a variable(say curMin). Now whenever we assign the index for its children, we take the parent node index as (i-curMin) rather than i. The below illustration will clear the concept.



As our final answer is a range of nodes in a level, i,e rightMost- leftMost+1; this strategy will not affect the answer and at the same time prevent the integer overflow case.
The algorithm approach can be stated as:

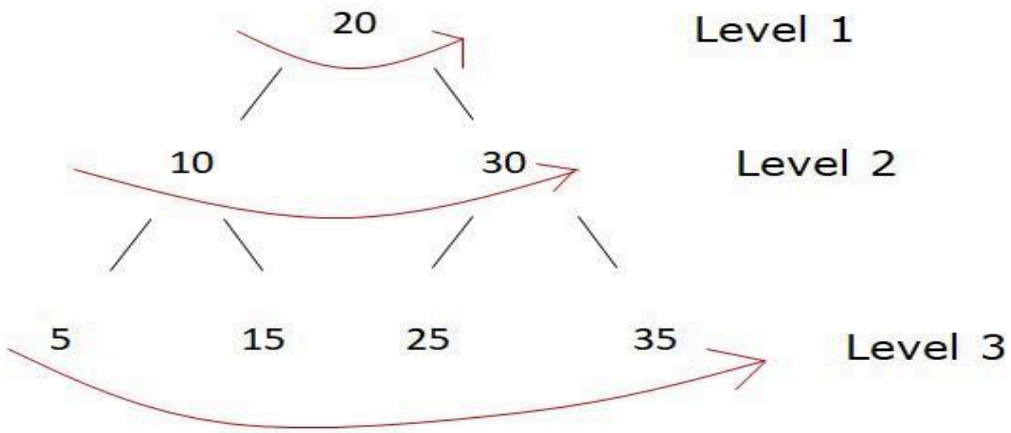
- We take a queue and push the root node along with index 0.
- We traverse the tree using a level order traversal.
- In the level order traversal we set another loop to run for the size of the queue, so that we visit the same level nodes inside it.
- Before a level starts, we use a variable(say curMin) to store the index of the first node.
- We assign an index to every node, and to its children as described above.
- When the inner loop is at the first node of a level, we store its index in another variable(sayleftMost)
- When the inner loop is at the last node of a level, we store its index in another variable(say rightMost)
- After a level in the outer loop, we calculate the width of the level as (rightMost – leftMost +1).
- We return the maximum width as the answer.

Dry Run: In case you want to watch the dry run for this approach, please watch the video attached below.

```
class TreeNode {
  int data;
  TreeNode  left,  right;
  TreeNode(int data) {
      this.data=data;
      left=null;
      right=null;
  } }
class Pair {
    TreeNode node;
    int num;
    Pair(TreeNode _node, int _num) {
        num = _num;
        node = _node;
    } }
class Solution {
    public static int widthOfBinaryTree(TreeNode root) {
        if(root == null) return 0;
        int ans = 0;
        Queue<Pair> q = new LinkedList<>();
        q.offer(new Pair(root, 0));
        while(!q.isEmpty()){
            int size = q.size();
            int mmin = q.peek().num;    //to make the id starting from zero
            int first = 0,last = 0;
            for(int i=0; i<size; i++){
                int cur_id = q.peek().num-mmin;
                TreeNode node = q.peek().node;
                q.poll();
                if(i==0) first = cur_id;
                if(i==size-1) last = cur_id;
                if(node.left != null)
                    q.offer(new Pair(node.left, cur_id*2+1));
                if(node.right != null)
                    q.offer(new Pair(node.right, cur_id*2+2));
            }
            ans = Math.max(ans, last-first+1);
        }
        return ans; }}
```

12. Level order traversal of BT

Problem Statement: Level order traversal of a binary tree. Given the root node of the tree and you have to print the value of the level of the node by level.
Example 1:



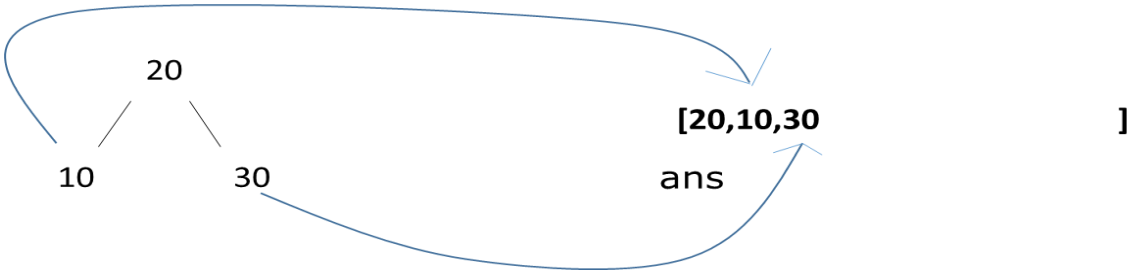
Output: 20 10 30 5 15 25 35

We will print the nodes of the first level (20), then we will print nodes of second level(10,30) and at last we will print nodes of the last level(5,15,25,35)

Solution: Let us consider the example 1 given above.

We need to print a level by level traversal. Let us say, we store all the levels in a data structure named 'ans'. We can use a vector<int> or List<int> to store our answer.
Now we need to store the first level to 'ans'. In any given binary tree, the first level will always be the root node, so we can easily store it.

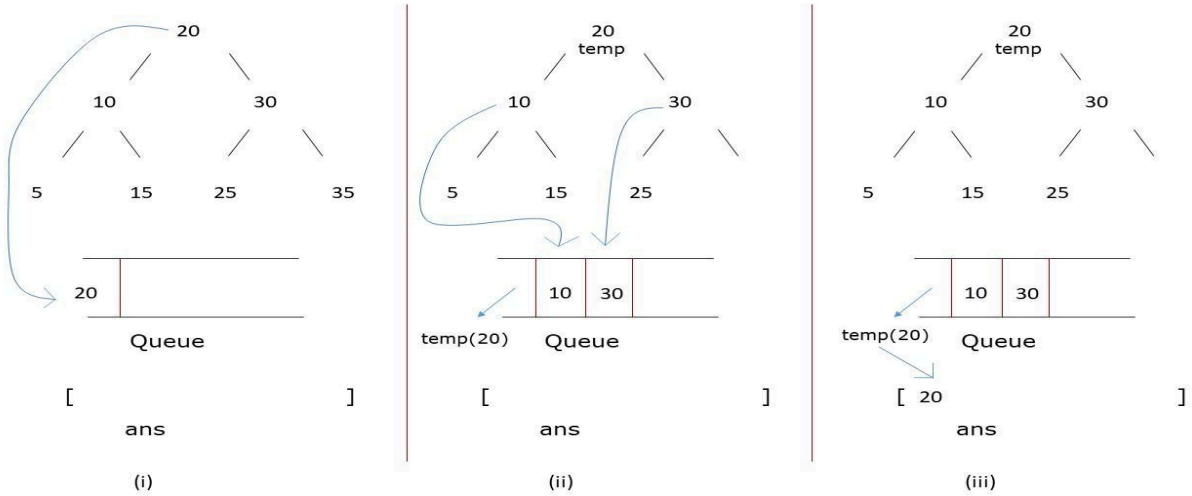
Next, we want to store the second level, this can be done by pushing the left child of root and then the right child of the root to our 'ans' data structure.



Now what about the next level? We just can't keep writing root→left→left / root→left→right and so on to reach further levels. We need an additional data structure to store all the nodes of a level. When we are at level 1, we want its left child to be stored first, followed by the right child as the next level and store it in our data structure. Similarly at level 2, we want to access first the left child (which was added first to the data structure), to store the nodes of the next level.
As there is a first-in first-out (FIFO) operation, we will use the queue data structure.
Approach:
The algorithm steps are stated as:

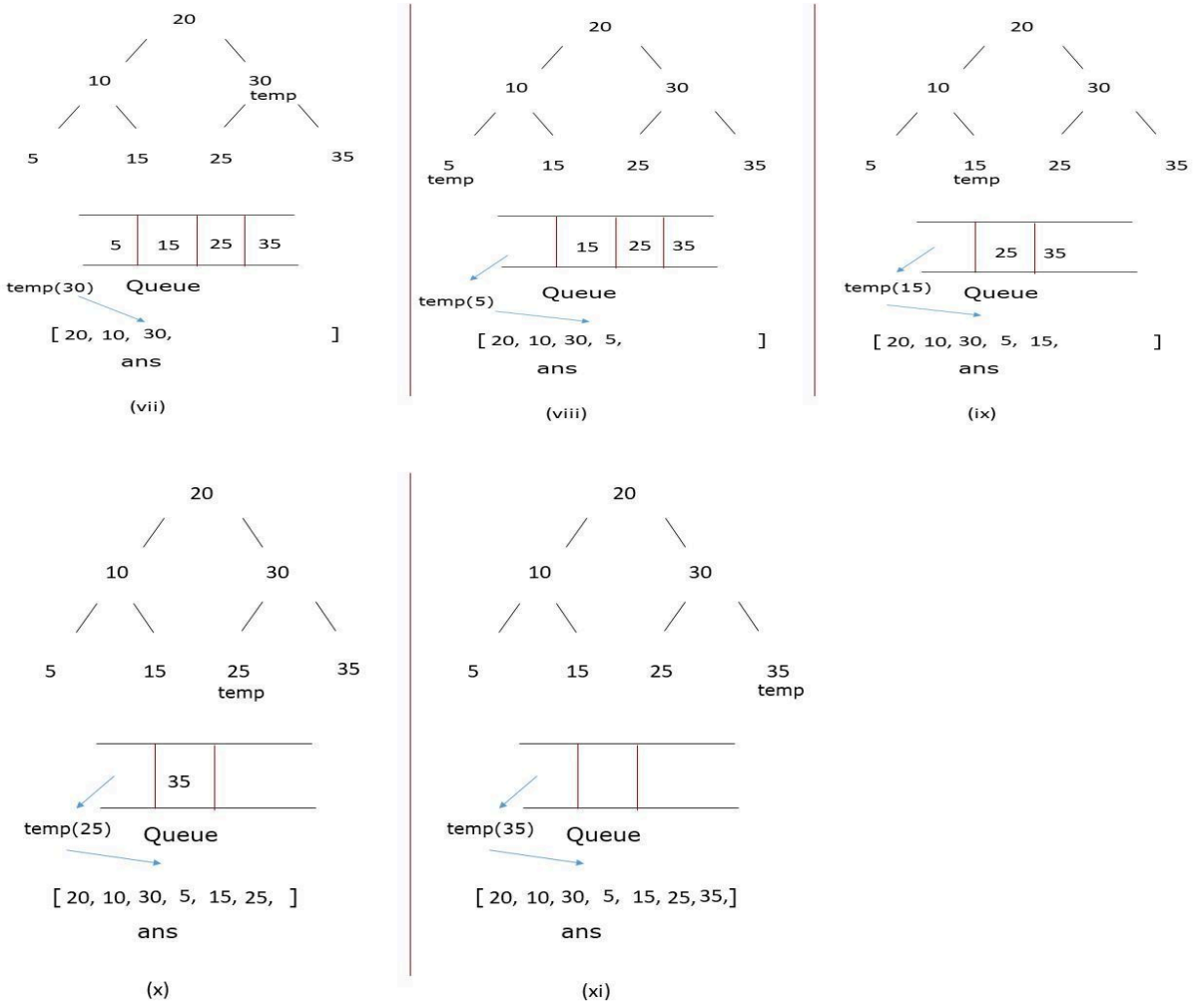
- Take a queue data structure and push the root node to the queue.
- Set a while loop which will run till our queue is non-empty.
- In every iteration, pop out from the front of the queue and assign it to a variable (say temp).
- If temp has a left child, push it to the queue.
- If temp has a right child, push it to the queue.
- At last push the value of the temp node to our "ans" data structure.

Dry Run:
We will discuss example 1.



1. First we insert the root node to the queue.
2. We set our while loop and pop its front element as temp Then we inserted the left child followed by the right child of temp to the queue.
3. Then we push the temp value to our ans list.

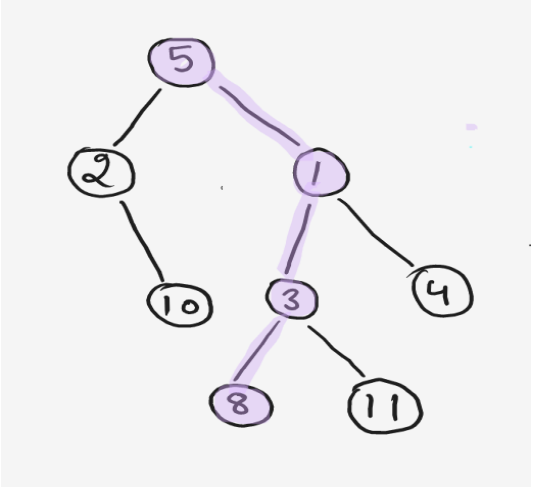
Then we follow these steps for all nodes till the time our queue is non-empty, as shown in the figures below.




```
public List<Integer> levelOrder(TreeNode root) {
      Queue<TreeNode> queue = new LinkedList<TreeNode>();
      List<Integer> wrapList = new List<Integer>();
      if(root == null) return wrapList;

      queue.offer(root);
      while(!queue.isEmpty()){

          if(queue.peek().left != null)
              queue.offer(queue.peek().left);

          if(queue.peek().right != null)
              queue.offer(queue.peek().right);

          wrapList.add(queue.poll().val);
      }
      return wrapList; }
```

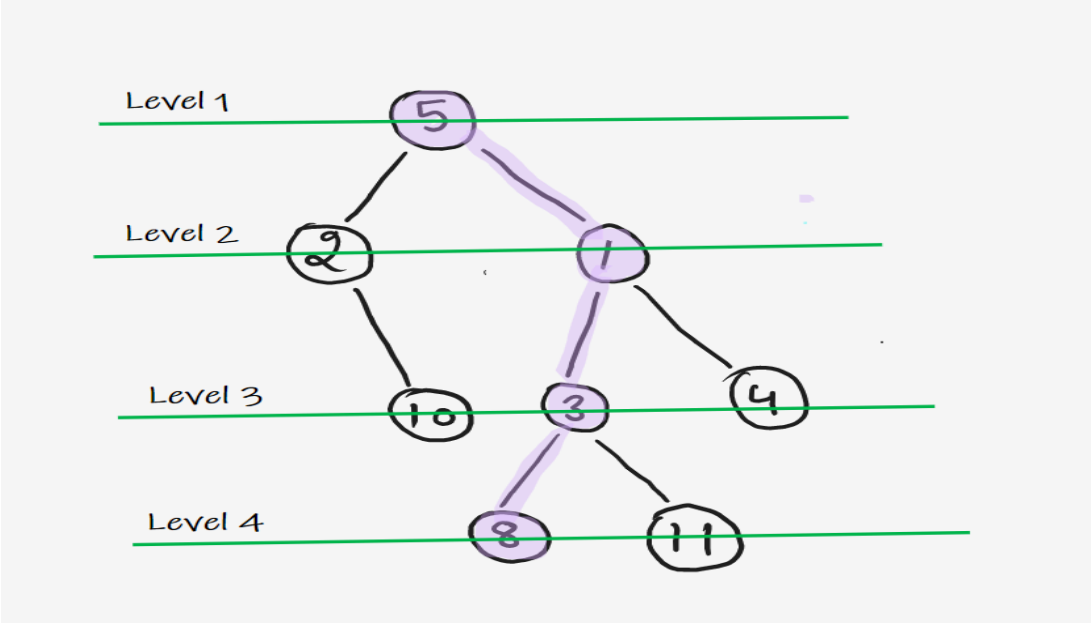
13. Min depth of BT

Problem Statement: Find the Maximum Depth of Binary Tree. Maximum Depth is the count of nodes of the longest path from the root node to the leaf node.Input Format: Given the root of Binary Tree



Result: 4

Solution 1:
Intuition + Approach: Using LEVEL ORDER TRAVERSAL
If we observe carefully, the depth of the Binary Tree is the number of levels in the binary tree. So, if we simply do a level order traversal on the binary tree and keep a count of the number of levels, it will be our answer.



In this example, if we start traversing the tree level-wise, then we can reach at max Level 4, so our answer is 4. Because the maximum depth we can achieve is indicated by the last level at which we can travel.

```
public class tUf {
    private static int levelOrder( Node root ){
        if( root == null ){
            return 0;
        }

        LinkedList<Node> queue = new LinkedList<>();
        queue.addLast(root);

        int level = 0;

        while( queue.size() > 0 ){
            int size = queue.size();

            while( size-- > 0 ){
                Node remNode = queue.removeFirst();
                if( remNode.left != null ){
                    queue.addLast( remNode.left );
                }
                if( remNode.right != null ){
                        queue.addLast( remNode.right );
                }
            }
            level++;
        }

        return level; }  }
```

Solution 2:
Intuition: Recursively (Post Order Traversal)
If we have to do it recursively, then what we can think of is, If I have Maximum Depth of Left subtree and Maximum Depth of Right subtree then what will be the height or depth of the tree?

Exactly,

1 + max(depth of left subtree, depth of right subtree)

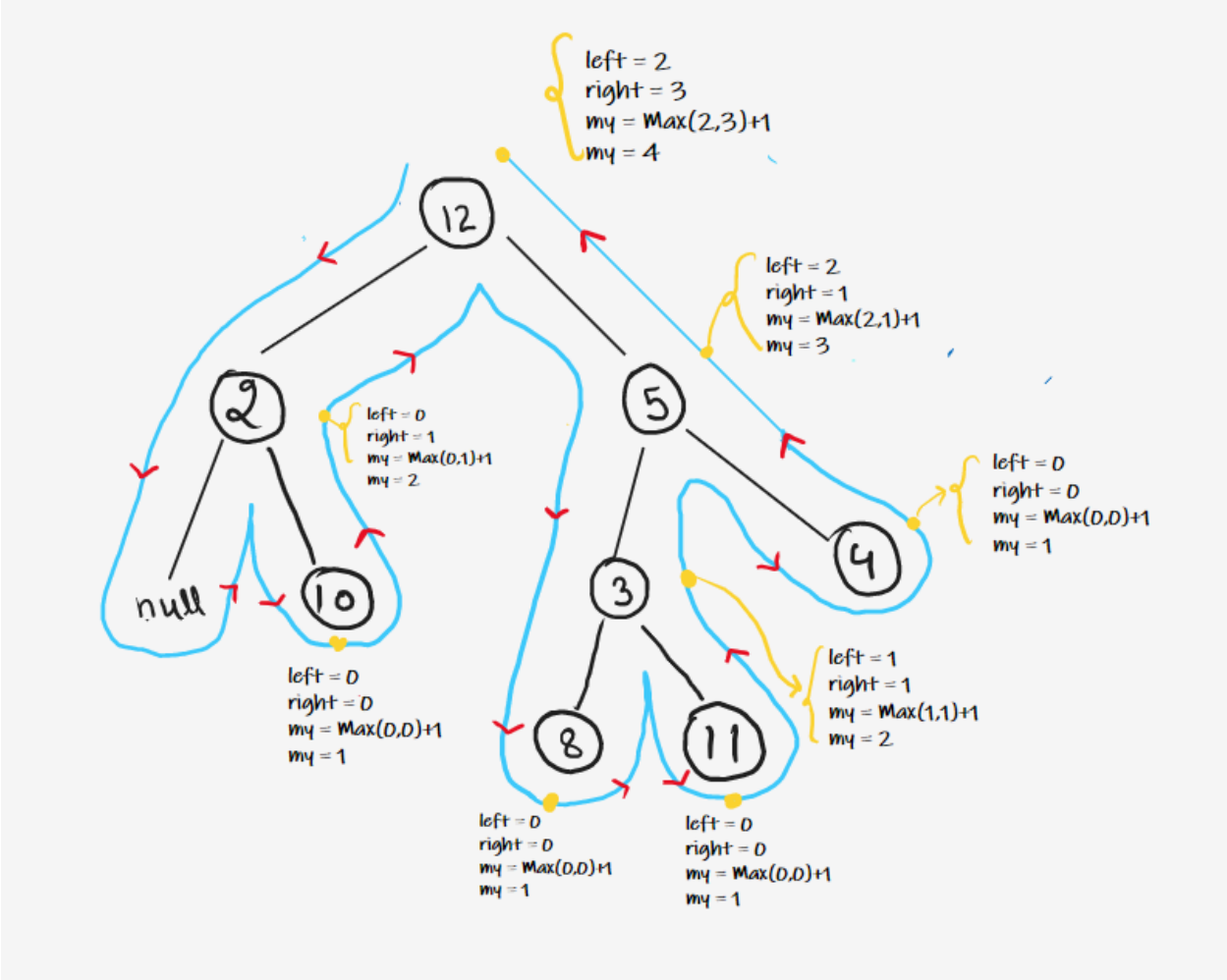
So, to calculate the Maximum Depth, we can simply take the maximum of the depths of the left and right subtree and add 1 to it.

Why take Maximum?? Because we need maximum depth so if we know left & right children's maximum depth then we'll definitely get to the maximum depth of the entire tree.

Approach :

- We start to travel recursively and do our work in Post Order.
- Reason behind using Post Order comes from our intuition , that if we know the result of left and right child then we can calculate the result using that.
- This is exactly an indication of PostOrder, because in PostOrder we already calculated results for left and right children than we do it for current node.
- So for every node post order, we do Max(left result , right result) + 1 and return it to the previous call.
- Base Case is when root == null so we need to return 0;

Dry Run : In Post Order, we start to travel on the example given in the below diagram



- Reach on **Node 10** , Left child = null so 0 , Right child = null so 0 & add 1 for node 10 so max depth till node 10 is max(0,0) + 1 = 1.
- Reach on **Node 2** , Left child = null so 0 , Right child = will give 1 & add 1 for node 2 so max depth till node 2 is max(0,1) + 1 = 2.
- Reach on **Node 8** , Left child = null so 0 , Right child = null so 0 & add 1 for node 8 so max depth till node 8 is max(0,0) + 1 = 1.
- Reach on **Node 11** , Left child = null so 0 , Right child = null so 0 & add 1 for node 11 so max depth till node 11 is max(0,0) + 1 = 1.

- Reach on **Node 3** , Left child will give 1 , Right child = will give 1 & add 1 for node 3 so max depth till node 3 is max(1,1) + 1 = 2.
- Reach on **Node 4** , Left child = null so 0 , Right child = null so 0 & add 1 for node 4 so max depth till node 4 is max(0,0) + 1 = 1.
- Reach on **Node 5** , Left child will give 2 , Right child = will give 1 & add 1 for node 5 so max depth till node 5 is max(2,1) + 1 = 3.
- Reach on **Node 12** , Left child will give 2 , Right child = will give 3 & add 1 for node 12 so max depth till node 12 is max(2,3) + 1 = 4.
- Hence 4 is our final ans.

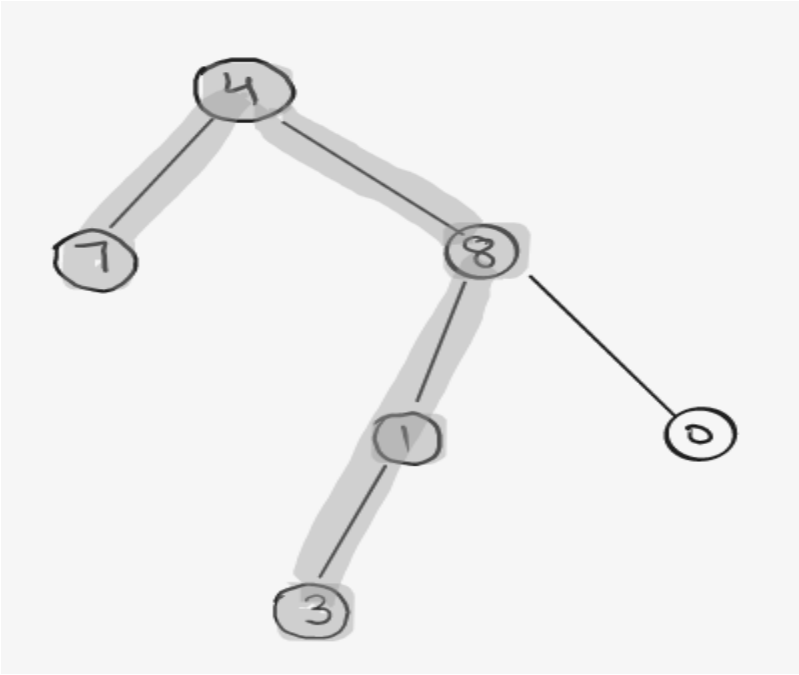
Code:

```
class Solution {
    public int maxDepth(TreeNode root) {
        if(root == null) return 0;

        int lh = maxDepth(root.left);
        int rh = maxDepth(root.right);

        return 1 + Math.max(lh, rh); }}
```

14. Diameter of BT

Problem Statement: Find the Diameter of a Binary Tree. Diameter is the length of the longest path between any 2 nodes in the tree and this path may or may not pass from the root.

Example 1: Input Format: Given the root of Binary Tree

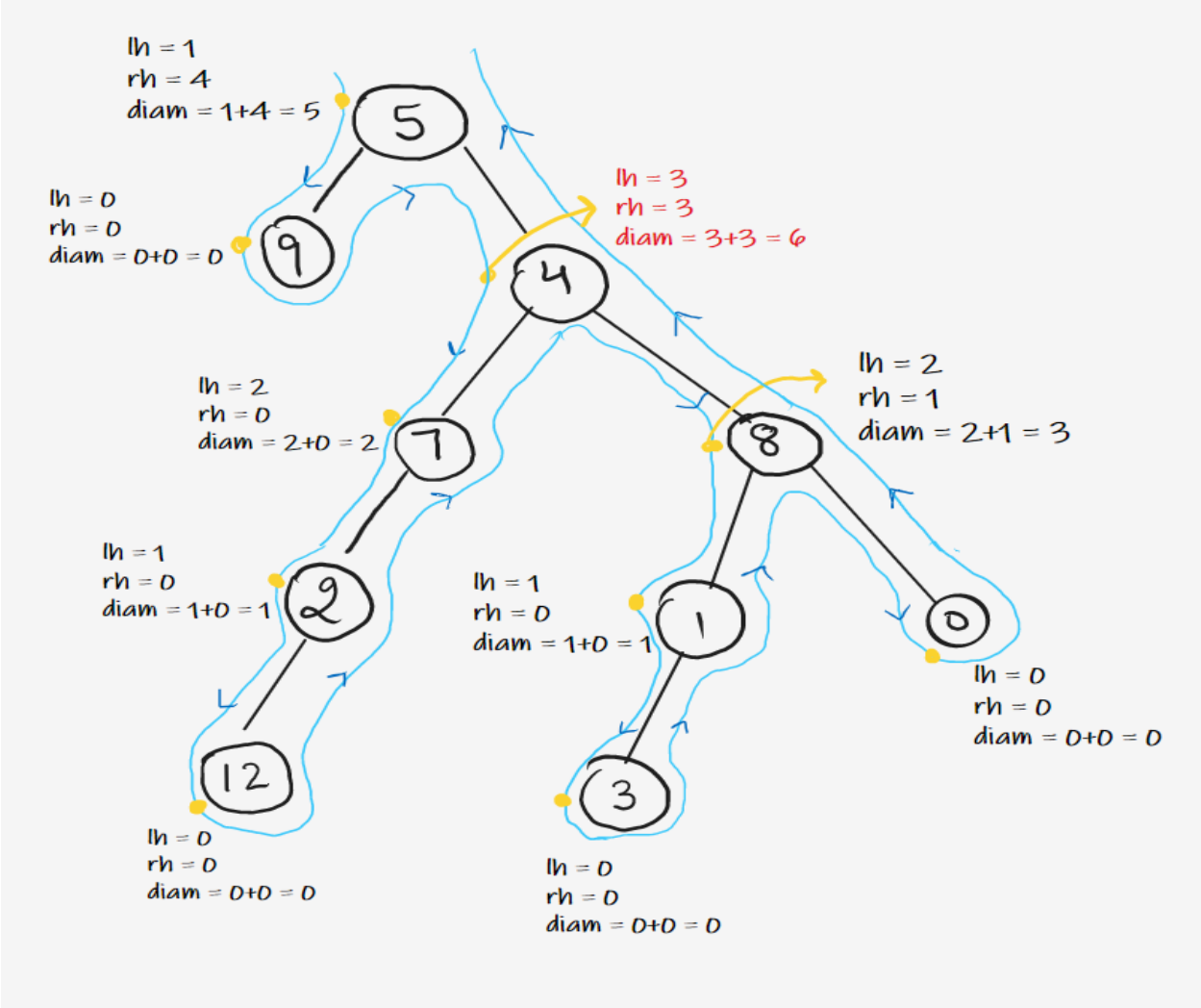


Diameter of a Binary Tree

Result: 4
Explanation: Longest Path available is 7 – 4 – 8 – 1 – 3 of length 4
Solution 1: Naive approach
Intuition : The idea is to consider every node as a *Curving Point* in diameter. For our understanding, we can define the curving point as the node on the diameter path which has the maximum height. In the above examples, the Curving Point is node **4** in Example 1 and node **8** in Example 2.
Now, if we observe carefully, we can see that the diameter of the tree can be defined as left subtree height + right subtree height from the Curving Point.
So, the idea to find the Curving Point is, consider every node in the tree as a curving point and calculate the diameter for every curving point and return the maximum of all diameters.
Diameter at given Curving Point = Left Height + Right Height
Approach :

- Traverse the tree recursively.
- At every node, calculate height of left and right subtrees.
- Calculate the diameter for every node using the above formula.
- Calculate the maximum of all diameters. This can be done simply using a variable passed by reference in the recursive calls or a global static variable.

dry-run :



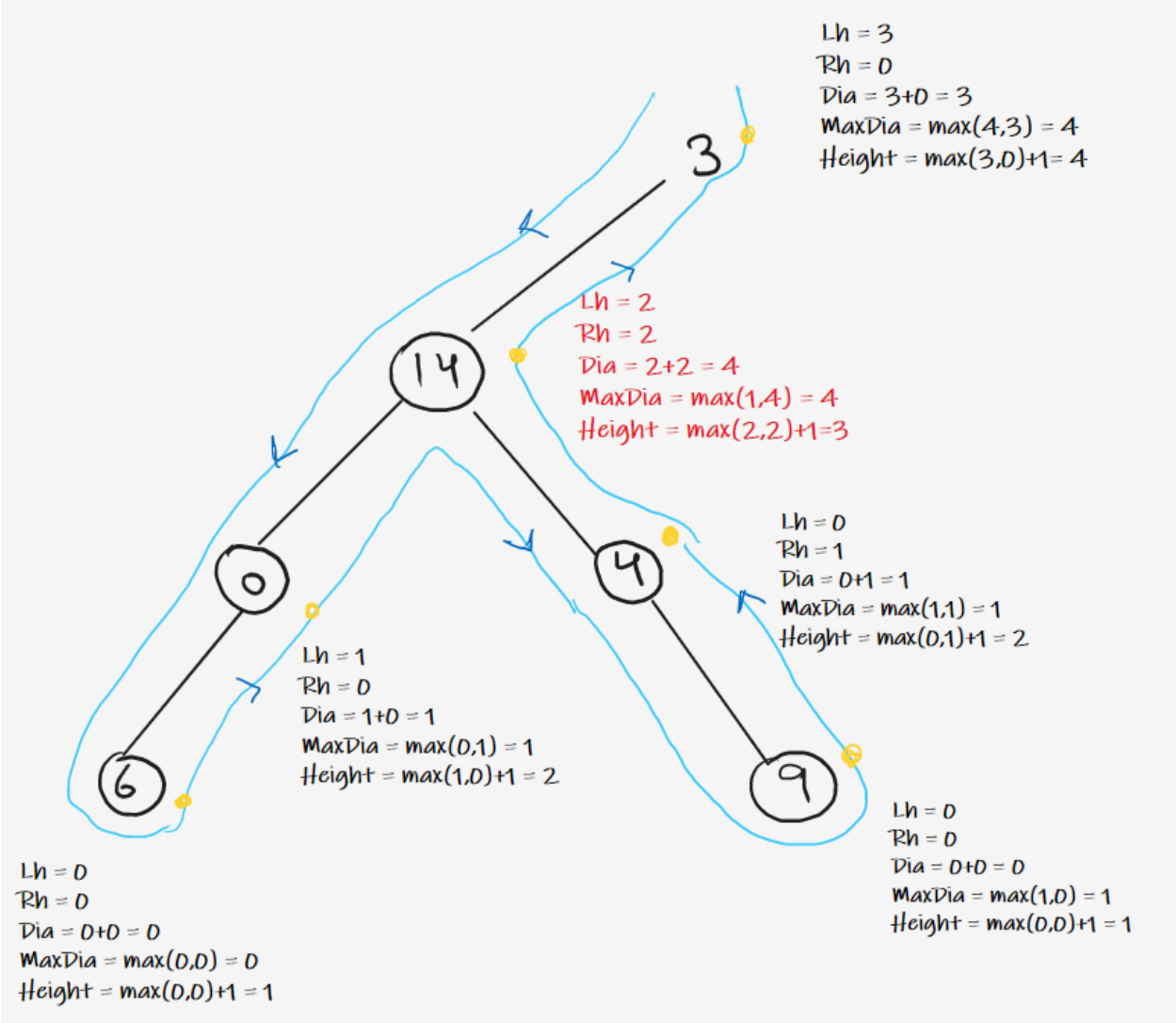
Start traversing the tree.

- Initialize Maximum Diameter variable with Integer.MIN_VALUE.
- Reach on **Node 5** , call Height Function , Left height = 1 , Right height = 4 so Diameter is (1 + 4) = 5. Hence , Maximum Diameter = Max(Integer.MIN_VALUE , 5) = 5.
- Reach on **Node 9** , call Height Function , Left height = 0 , Right height = 0 so Diameter is (0 + 0) = 0. Hence , Maximum Diameter = Max(5 , 0) = 5.
- Reach on **Node 4** , call Height Function , Left height = 3 , Right height = 3 so Diameter is (3 + 3) = 6. Hence , Maximum Diameter = Max(5 , 6) = 6.
- Reach on **Node 7** , call Height Function , Left height = 2 , Right height = 0 so Diameter is (2 + 0) = 2. Hence , Maximum Diameter = Max(6,2) = 6.
- Reach on **Node 2** , call Height Function , Left height = 1 , Right height = 0 so Diameter is (1 + 0) = 1. Hence , Maximum Diameter = Max(6 , 1) = 6.
- Reach on **Node 12** , call Height Function , Left height = 0 , Right height = 0 so Diameter is (0 + 0) = 0. Hence , Maximum Diameter = Max(6 , 0) = 6.
- Reach on **Node 8** , call Height Function , Left height = 2 , Right height = 1 so Diameter is (2 + 1) = 3. Hence , Maximum Diameter = Max(6 , 3) = 6.
- Reach on **Node 1** , call Height Function , Left height = 1 , Right height = 0 so Diameter is (1 + 0) = 1. Hence , Maximum Diameter = Max(6 , 1) = 6.
- Reach on **Node 3** , call Height Function , Left height = 0 , Right height = 0 so Diameter is (0 + 0) = 0. Hence , Maximum Diameter = Max(6 , 0) = 6.
- Reach on **Node 0** , call Height Function , Left height = 0 , Right height = 0 so Diameter is (0 + 0) = 0. Hence , Maximum Diameter = Max(6 , 0) = 6.

Time Complexity: O(N*N) (For every node, Height Function is called which takes O(N) time hence for every node it becomes N*N)
Space Complexity: O(1)(Extra Space) + **O(H)** (Recursive Stack Space where **"H"** is the height of tree)
Solution 2: Post Order Traversal
Intuition :
Is it possible to optimize the above solution further? Which operation do you think is very repetitive in nature in the above solution?
Height of the subtrees.
Can we use postorder traversal to calculate everything in a single traversal of the tree?
Yes, as in post-order traversal, we have to completely traverse the left and right subtree before visiting the root node.
So, the idea is to use post-order traversal and keep calculating the height of the left and right subtrees.
Once we have the heights at the current node, we can easily calculate both the diameter and height of the current node.
Approach :

- Start traversing the tree recursively and do work in Post Order.
- In the Post Order of every node , calculate diameter and height of the current node.
- If current diameter is maximum then update the variable used to store the maximum diameter.
- Return height of current node to the previous recursive call.

Dry Run :
In Post Order, Start traversing the tree:



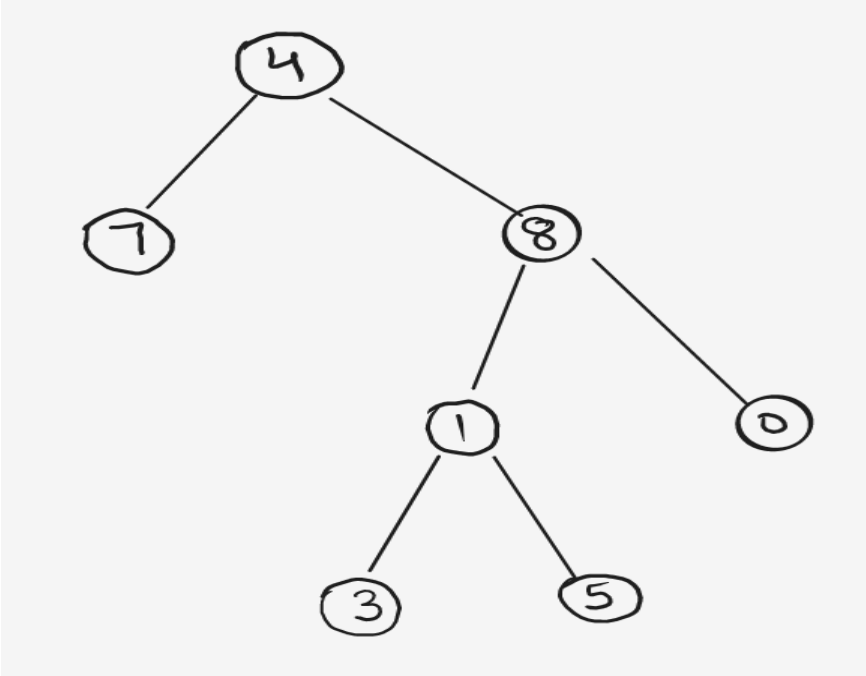
- Reach on **Node 6 ,** Left height = 0 as left == null , Right height = 0 as right == null so Diameter is (0 + 0) = 0. Hence , Maximum Diameter = Max(0 , 0) = 0 and return height = max(0,0)+1 = 1.
- Reach on **Node 0**, Left height = 1 , Right height = 0 as right == null so Diameter is (1 + 0) = 1. Hence , Maximum Diameter = Max(0 , 1) = 1 and return height = max(1,0)+1 = 2.
- Reach on **Node 9 ,** Left height = 0 as left == null , Right height = 0 as right == null so Diameter is (0 + 0) = 0. Hence , Maximum Diameter = Max(1 , 0) = 1 and return height = max(0,0)+1 = 1.
- Reach on **Node 4 ,** Left height = 0 as left == null , Right height = 1 , so Diameter is (0 + 1) = 1. Hence , Maximum Diameter = Max(1 , 1) = 1 and return height = max(0,1)+1 = 2.
- Reach on **Node 14 ,** Left height = 2 , Right height = 2 , so Diameter is (2 + 2) = 4. Hence , Maximum Diameter = Max(1 , 4) = 4 and return height = max(2,2)+1 = 3.
- Reach on **Node 3 ,** Left height = 3 , Right height = 0 as right == null , so Diameter is (3 + 0) = 3. Hence , Maximum Diameter = Max(4 , 3) = 4 and return height = max(3,0)+1 = 4.
- Hence , the maximum diameter is 4 .

```java
public class Solution {
    public int diameterOfBinaryTree(TreeNode root) {
        int[] diameter = new int[1];
        height(root, diameter);
        return diameter[0];
    }

    private int height(TreeNode node, int[] diameter) {
        if (node == null) {
            return 0;
        }
        int lh = height(node.left, diameter);
        int rh = height(node.right, diameter);
        diameter[0] = Math.max(diameter[0], lh + rh);
        return 1 + Math.max(lh, rh);
    }
}
```

15. Check if balanced BT or not

Problem Statement: Check whether the given Binary Tree is a Balanced Binary Tree or not. A binary tree is balanced if, for all nodes in the tree, the difference between left and right subtree height is not more than 1.

Examples: **Input Format**: Given the root of Binary Tree



Binary Tree

Result: False

Explanation: At Node 4, Left Height = 1 & Right Height = 3, Absolute Difference is 2 which is greater than 1, Hence, not a balanced tree.

Solution 1: Naive approach

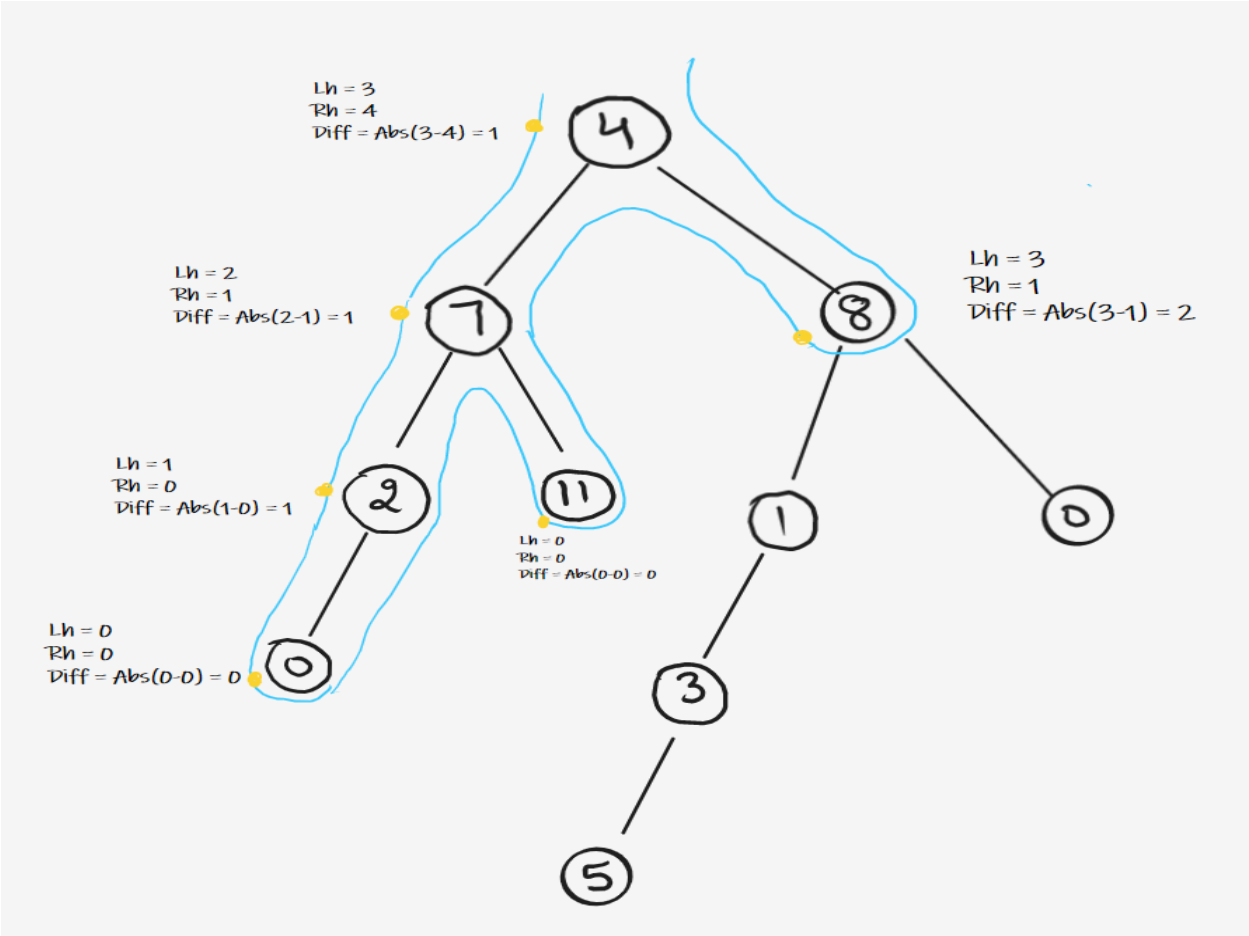
Intuition + Approach:

For a Balanced Binary Tree, Check left subtree height and right subtree height for every node present in the tree. Hence, traverse the tree recursively and calculate the height of left and right subtree from every node, and whenever the condition of Balanced tree violates, simply return false.

Condition for Balanced Binary Tree

For all Nodes , Absolute(Left Subtree Height – Right Subtree Height) <= 1

dry-run :



Start traversing the tree, the example given in above diagram:

- Reach on Node 4, call Height Function , Left height = 3 , Right height = 4 so Absolute Difference between two is Abs(3 – 4) = 1.
- Reach on Node 7, call Height Function , Left height = 2 , Right height = 1 so Absolute Difference between two is Abs(2 – 1) = 1.
- Reach on Node 2, call Height Function , Left height = 1 , Right height = 0 so Absolute Difference between two is Abs(1 – 0) = 1.
- Reach on Node 0, call Height Function , Left height = 0 , Right height = 0 so Absolute Difference between two is Abs(0 – 0) = 0.
- Now, on PostOrder of Node 0, the left subtree (null) gives true & right subtree (null) gives true , as both are true , return true.
- Now, on PostOrder of Node 2, the left subtree (0) gives true & right subtree (null) gives true , as both are true , return true.
- Reach on Node 11, call Height Function , Left height = 0 , Right height = 0 so Absolute Difference between two is Abs(0 – 0) = 0.
- Now , on PostOrder of Node 11, the left subtree (null) gives true & right subtree (null) gives true , as both are true , return true.
- Now ,on PostOrder of Node 7, the left subtree (2) gives true & right subtree (11) gives true , as both are true , return true.
- Reach on Node 8, call Height Function , Left height = 3 , Right height = 1 so Absolute Difference between two is Abs(3 – 1) = 2. Here Condition violates , simply return false, no need to call further
- Now , on PostOrder of Node 4, the left subtree (7) gives true & right subtree (8) gives false , so any one of subtree gives false , return false.

Time Complexity: O(N*N) (For every node, Height Function is called which takes O(N) Time. Hence for every node it becomes N*N)

Space Complexity: O(1) (Extra Space) + O(H) (Recursive Stack Space where “H” is the height of tree)

Solution 2: Post Order Traversal

Intuition: Can we optimize the above brute force solution? Which operation do you think can be skipped to optimize the time complexity?

Ain’t we traversing the subtrees again and again in the above example?

Yes, so can we skip the repeated traversals?

What if we can make use of post-order traversal?

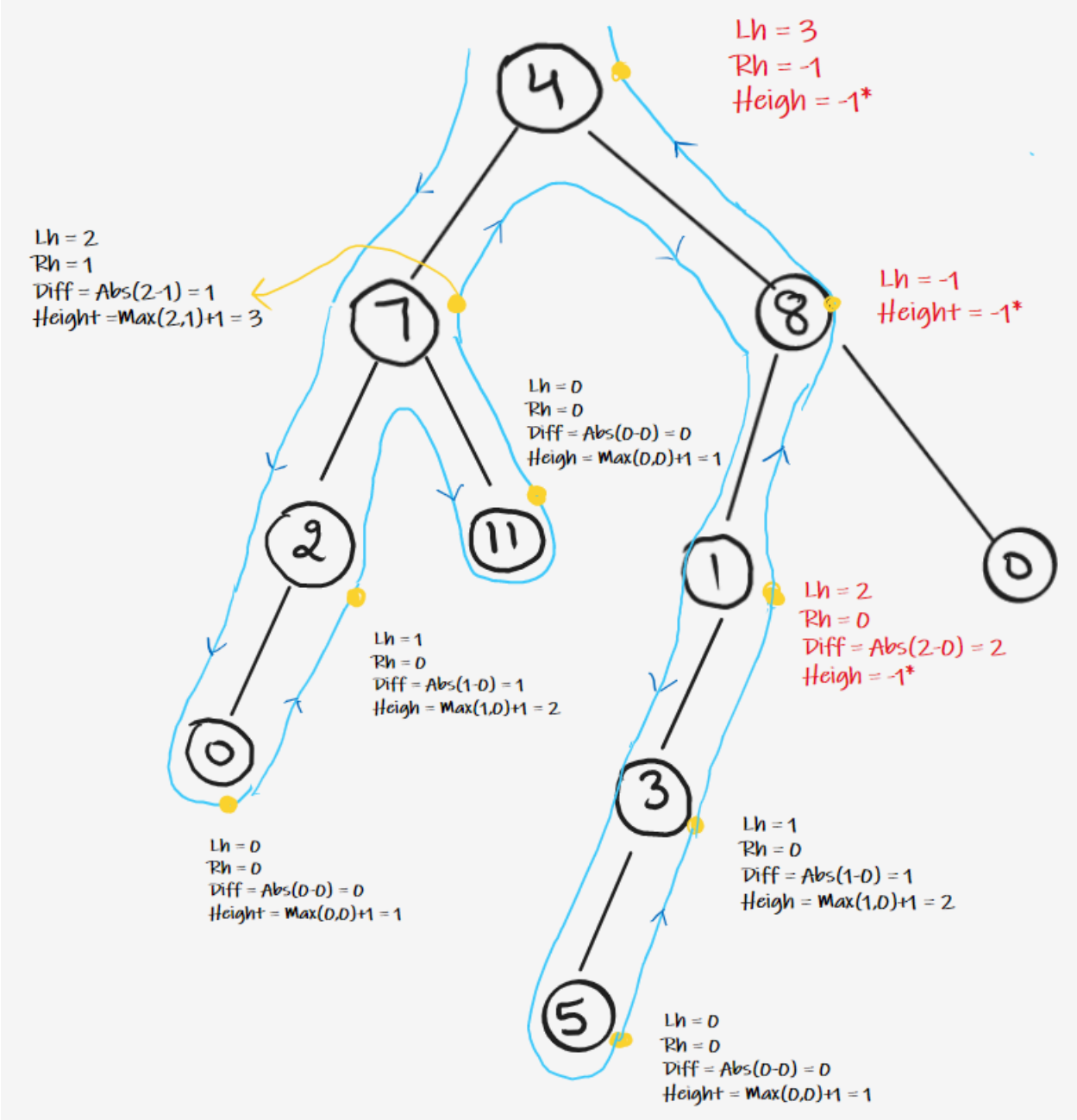
So, the idea is to use post-order traversal. Since, in postorder traversal, we first traverse the left and right subtrees and then visit the parent node, similarly instead of calculating the height of the left subtree and right subtree every time at the root node, use post-order traversal, and keep calculating the heights of the left and right subtrees and perform the validation.

Approach :

- Start traversing the tree recursively and do work in Post Order.
- For each call, caculate the height of the root node, and return it to previous calls.
- Simultaneously, in the Post Order of every node , Check for condition of balance as information of left and right subtree height is available.
- If it is balanced , simply return height of current node and if not then return -1.
- Whenever the subtree result is -1 , simply keep on returning -1.

Dry Run :

In Post Order, Start traversing the tree on the example given in below diagram



- Reach on Node 0, Left child = null so 0 Height , Right child = null so 0 Height , Difference is 0-0 = 0 , (0 <= 1) so return height , i.e. Max(0,0) + 1 = 1.

- Reach on Node 2 , Left subtree height = 1 , Right subtree height = 0, Difference is 1-0 = 1 , (1 <= 1) so return height , i.e. Max(1,0) + 1 = 2.
- Reach on Node 11 , Left child = null so 0 Height , Right child = null so 0 Height , Difference is 0-0 = 0 , (0 <= 1) so return height , i.e. Max(0,0) + 1 = 1.
- Reach on Node 7 , Left subtree height = 2 , Right subtree height = 1, Difference is 2-1 = 1 , (1 <= 1) so return height , i.e. Max(2,1) + 1 = 3.
- Reach on Node 5 , Left child = null so 0 Height , Right child = null so 0 Height , Difference is 0-0 = 0 , (0 <= 1) so return height , i.e. Max(0,0) + 1 = 1.
- Reach on Node 3 , Left subtree height = 1 , Right subtree height = 0, Difference is 1-0 = 1 , (1 <= 1) so return height , i.e. Max(1,0) + 1 = 2.
- Reach on Node 1 , Left subtree height = 2 , Right subtree height = 0, Difference is 2-0 = 2 , (2 > 1) i.e. Tree is not Balanced , so return -1.
- Reach on Node 8 , Left subtree height = -1 , indicates that tree is not balanced, simply return -1;
- Reach on Node 4 , Left subtree height = 3 , Right subtree height = -1, therefore indicates that tree is not balanced , simply return -1;
- In the Main function , If the final Height of tree is -1 return false as tree is not balanced , else return true.

Code:

```
class Solution {
    public boolean isBalanced(TreeNode root) {
        return dfsHeight (root) != -1;
    }
    int dfsHeight (TreeNode root) {
        if (root == null) return 0;

        int leftHeight = dfsHeight (root.left);
        if (leftHeight == -1) return -1;
        int rightHeight = dfsHeight (root.right);
        if (rightHeight == -1) return -1;

        if (Math.abs(leftHeight - rightHeight) > 1)  return -1;
        return Math.max(leftHeight, rightHeight) + 1;
    }
}
```

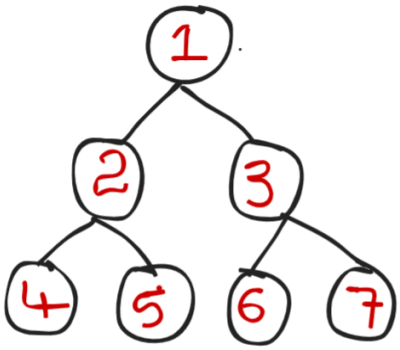
16. LCA of BT

Problem Statement: Given a binary tree, Find the Lowest Common Ancestor for two given Nodes (x,y).

Lowest Common Ancestor(LCA): The lowest common ancestor is defined between two nodes x and y as the lowest node in T that has both x and y as descendants (where we allow a node to be a descendant of itself.

Examples:

Consider the following Binary Tree



Example 1:

Input: x = 4 , y = 5 Output: 2

Explanation: All ancestors for 4,5 are 2,1. But we need Lowest Common ancestor, So we will consider lowest and also common ancestor that is 2

Solution: **Intuition:**

The very first thing we can observe from the question is that we can find the LCA of 2 given nodes from
i) Left subtree or in (ii)Right subtree, if not in both the subtrees then root will be the LCA.

Approach: If root is null or if root is x or if root is y then return root
Made a recursion call for both (i) Left subtree (ii)Right subtree

Because we would find LCA in the left or right subtree only.

1. If the left subtree recursive call gives a null value that means we haven't found LCA in the left subtree, which means we found LCA on the right subtree. So we will return right.
2. If the right subtree recursive call gives null value, that means we haven't found LCA on the right subtree, which means we found LCA on the left subtree. So we will return left .
3. If both left & right calls give values (not null) that means the root is the LCA.

Let's take an example and will try to understand the approach more clearly:

LCA of (x,y) = > (4,5) = ? (from above given example)

- Root is 1 which is not null and x,y is not equal to root, So the 1st statement in approach will not execute.
- i) Call left subtree, While calling recursively it will find 4 and this call will return 4 to its parent

Point to Note: At present, the root is 2 (Look at below recursion tree for better understanding)

i) Call the right subtree (i.e right of 2), While calling recursively it will find 5 and this call will return 5 to its parent.

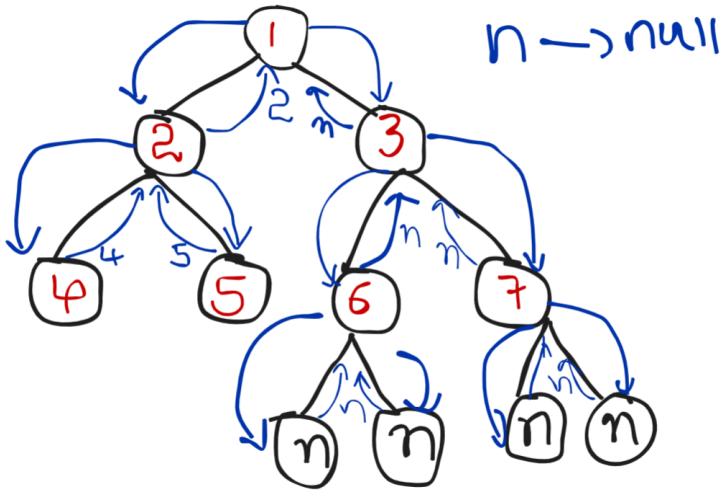
- Now the left recursive call returns value (not null) i.e 4 and also the right recursive call returns value (not null) i.e 5 to its root (at present root is 2) , and this 2 will return itself to its root i.e to 1 (main root).

Point to Note: At present, the root is 1 (Look at below recursion tree for better understanding)

- Now, the left subtree gives a value i.e 2.
- Right recursive call will give null value .because x,y are not present in the right subtree.

- As we know if the right recursive call gives null then we return the answer which we got from the left call, So we will return 2.
- Hence LCA of (4,5) is 2.

For a better understanding of the above example (LCA OF 4,5) :

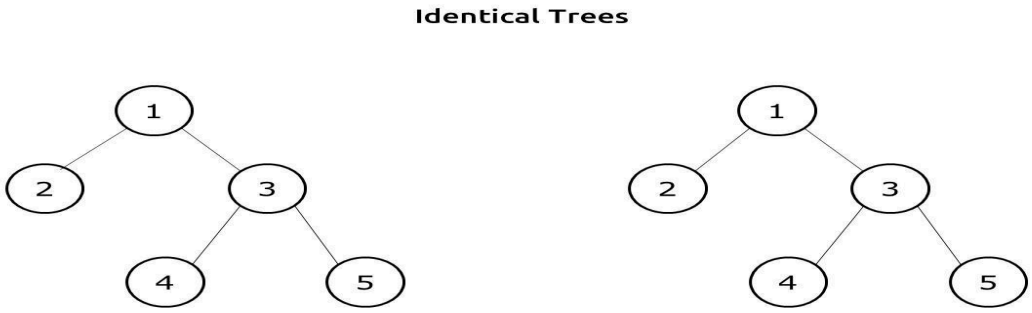


Code:

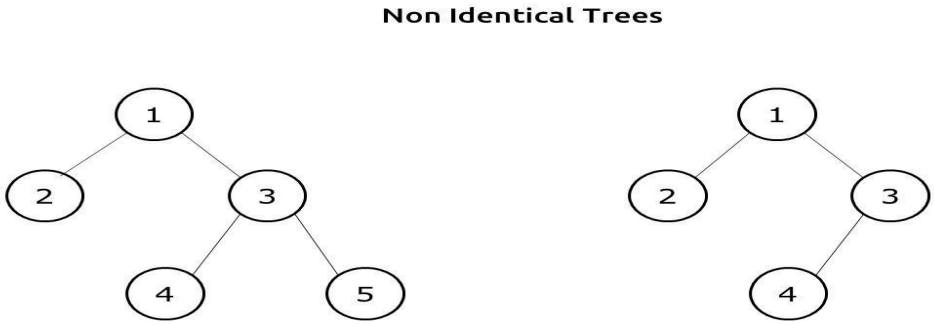
```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        //base case
        if (root == null || root == p || root == q) {
            return root;
        }
        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

        //result
        if(left == null) {
            return right;
        }
        else if(right == null) {
            return left;
        }
        else { //both left and right are not null, we found our result
            return root;
        }
    }
}
```

17. Check if 2 BT are identical

Problem Statement: Given two Binary Tree. Write a program to check if two trees are identical or not.
Example 1: Input:



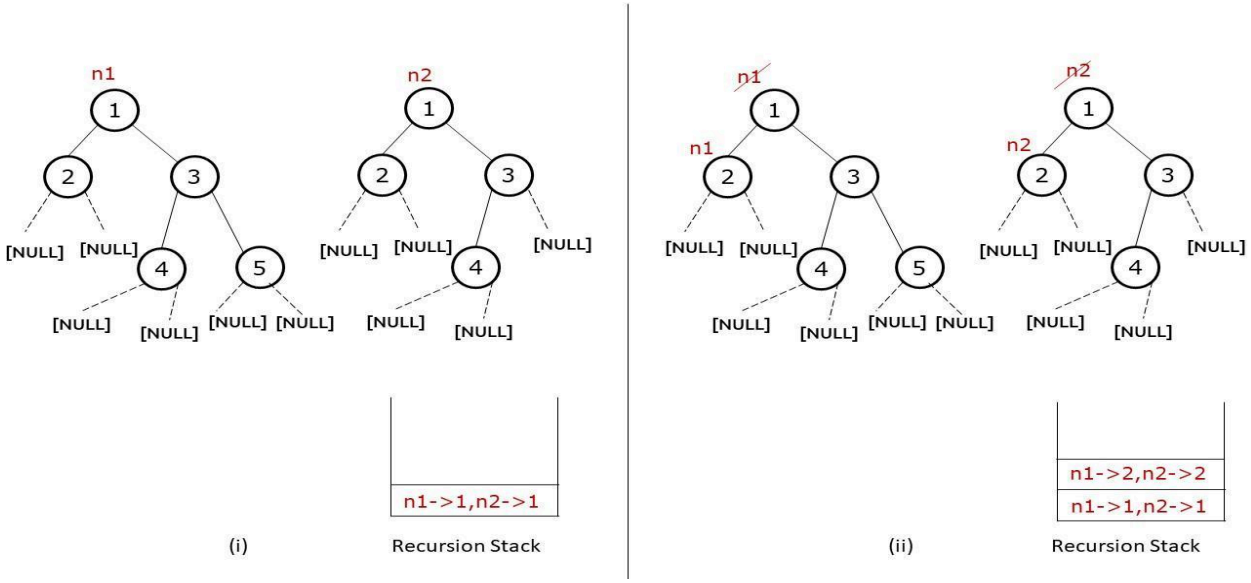
Output: Two Trees are identical
Explanation: Two trees T1 and T2 are said to be identical if these three conditions are met for every pair of nodes :
Value of a node in T1 is equal to the value of the corresponding node in T2.
Left subtree of this node is identical to the left subtree of the corresponding node.
Right subtree of this node is identical to the right subtree of the corresponding node.
Example 2: Input:



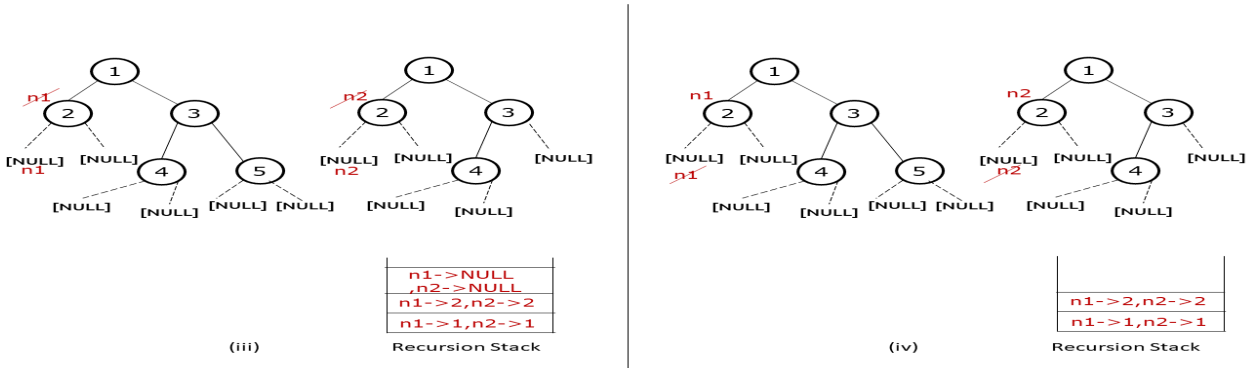
Output: Two Trees are not identical
Solution:
Approach: In order to check whether two trees are identical or not, we need to traverse the trees. While traversing we first check the value of the nodes, if they are unequal we can simply return false, as trees are non-identical. If they are the same, then we need to **recursively** check their left child as well as the

right child. When we get all the three values as true(node values, left child, right child) we can conclude that these are identical trees and can return true. Any other combination will return false.

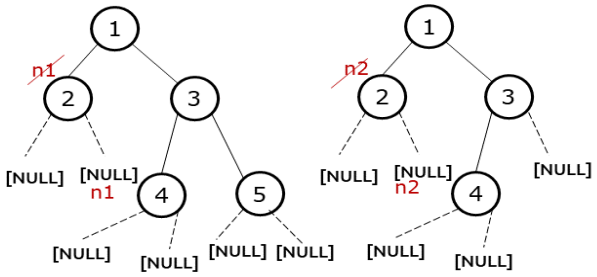
Dry Run:

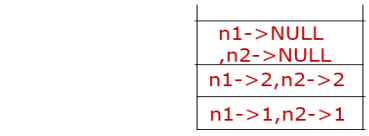


(i) Initially we have variables n1 and n2 pointing to the roots of both trees. This function is inside our recursion stack as well.
(ii) First we check the values at the nodes. As the values of nodes are equal, we proceed further and recursively call the left child of both nodes. This second function is pushed to our recursion stack.

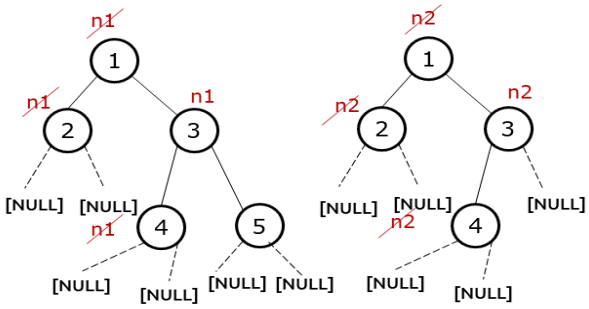


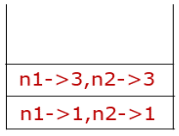
(iii) Again we check the nodes. As they are equal, we again check for the left subtrees and add the function to our recursion stack.
(iv) Both n1 and n2 point to NULL, so the base condition will be true and we will return true from this function.





(v) Recursion Stack

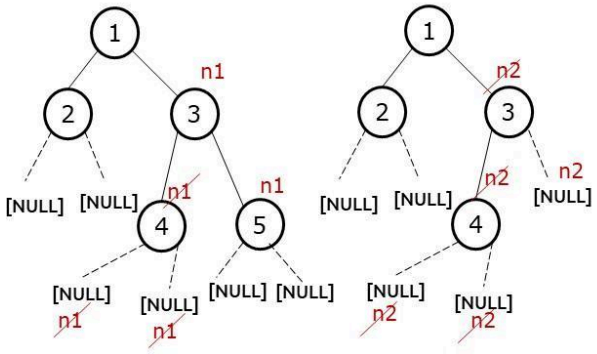


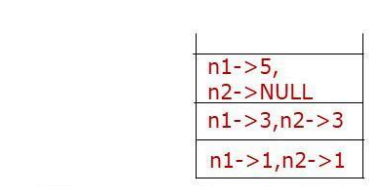


(vi) Recursion Stack

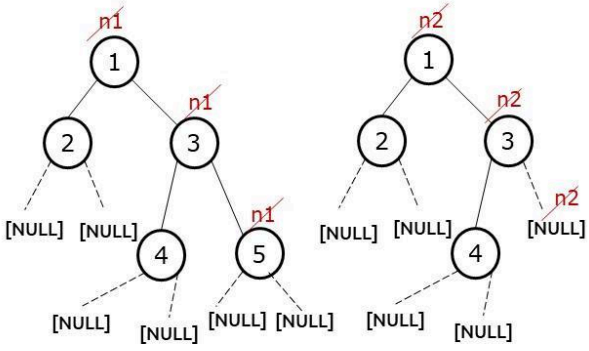
(v) At nodes pointing to 2, two out of three conditions return true, so we will proceed further and check the third condition. We will recursively call the right child of the nodes. Again that function will be added to our recursive stack.

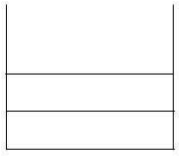
(vi) Similar to step 4, n1 and n2 point to NULL, therefore we will again return true and the function will be removed from our recursion stack. Now at nodes pointing to 2, all three conditions return true (node values, left child and right child), therefore we can return true from this function and remove it from the recursion stack. Then at node 1, two conditions are true, therefore we call the right child of n1 and n2 to check the third condition.





(vii) Recursion Stack





(viii) Recursion Stack

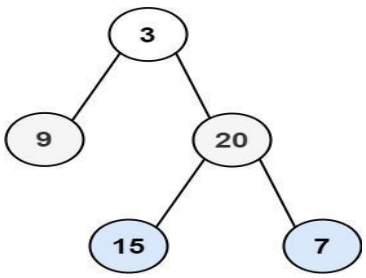
(vii) At nodes pointing to 3, we first check the data values which are equal, then we recursively call the function to check for their left child. It turns out to be identical and we get the return value true, then we recursively call the function to check for their right child.
(viii) Now n1 points to a node with value 5 whereas n2 points to NULL, this can't be the case with an identical tree, our second base condition hits and we return false from this function. In the parent function where n1 and n2 point to 3, **only two out of three conditions return true,** therefore we will return the value false. In its parent function where n1 and n2 points to 1, again **only two out of three conditions return true,** therefore we will return false. Hence we will return a false value.
As our first function returns a false to our main function, we can conclude that these two trees are **Non-identical.**

Code:

```
class Node {
    int data;
    Node left, right;
    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
public class TUF {
    static boolean isIdentical(Node node1, Node node2) {
        if (node1 == null && node2 == null)
            return true;
        else if (node1 == null || node2 == null)
            return false;

        return ((node1.data == node2.data) && isIdentical(node1.left, node2.left) &&
isIdentical(node1.right, node2.right));
    } }
```

18. Zigzag traversal of BT

Given the root of a binary tree, return the zigzag level order traversal of Binary Tree. (i.e., from left to right, then right to left for the next level and alternate between).
Example 1: Input: root = [3,9,20,null,null,15,7]



Output: [[3],[20,9],[15,7]]

Explanation: From the root, we follow this terminology, left to right -> right to left -> left to right and so on so forth.

Example 2:

Input: root = [[0]]

Output : [[0]]

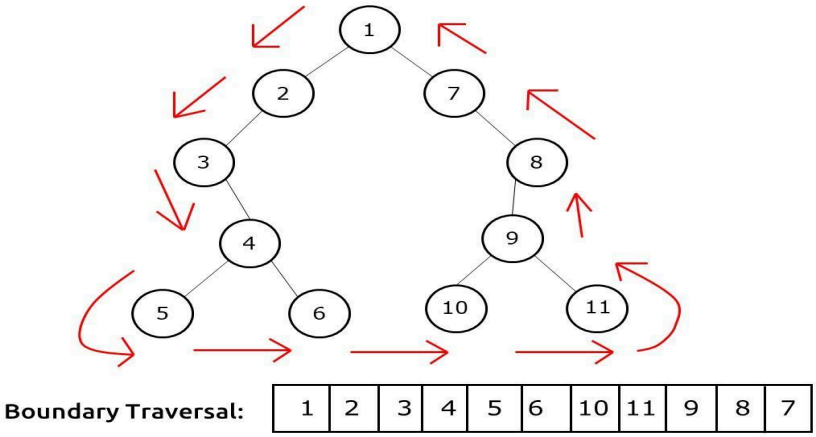
Explanation: We just have a single node which acts as the root, so going from left to right, we get just one node that is the root node itself.
Intuition: Considering the fact that we need to print the nodes, level by level, our first guess would definitely be that it must be related to level order traversal. If we closely examine, for even levels we need to go from left to right while for odd levels we need to go from right to left.
Approach: The above idea, could be implemented with a queue. We initially keep an empty queue and push the root node. We also need to keep the left to right bool variable that keeps check of the current level we are in. As we traverse nodes in the queue, we need to push them in a temporary array. If left to right is false we need to reverse the array and push it in our data structure or else, simply push it in our data structure. In the end, when we have finished traversing the current level, we need to toggle our left to the right variable.

Code:

```
class Node {
    int val;
    Node left, right;
    Node(int val) { this.val = val; left = null; right = null;}}

class TUF {
    public static ArrayList < ArrayList < Integer >> zigzagLevelOrder(Node root) {
        Queue < Node > queue = new LinkedList < Node > ();
        ArrayList < ArrayList < Integer >> wrapList = new ArrayList < > ();
        if (root == null) return wrapList;
        queue.offer(root);
        boolean flag = true;
        while (!queue.isEmpty()) {
            int levelNum = queue.size();
            ArrayList < Integer > subList = new ArrayList < Integer > (levelNum);
            for (int i = 0; i < levelNum; i++) {
                int index = i;
                if (queue.peek().left != null) queue.offer(queue.peek().left);
                if (queue.peek().right != null) queue.offer(queue.peek().right);
                if (flag == true) subList.add(queue.poll().val);
                else subList.add(0, queue.poll().val);
            }
            flag = !flag;
            wrapList.add(subList);
        }
        return wrapList;
    }
```

19. Boundary traversal of BT

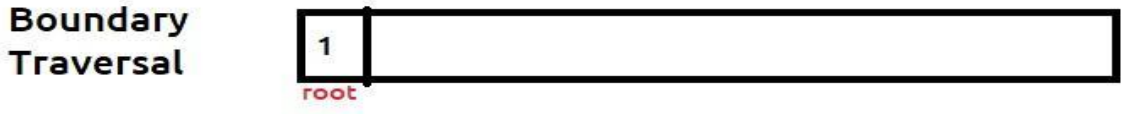
BoundaryTraversal of a binary tree. Write a program for the Anti-Clockwise Boundary traversal of a binary tree. **Example:**



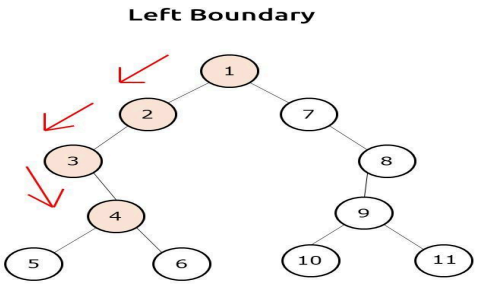
Solution N Approach: Boundary traversal in an anti-clockwise direction can be described as a traversal consisting of three parts:

1. **Part 1:** Left Boundary of the tree (excluding the leaf nodes).
2. **Part 2:** All the leaf nodes travelled in the left to right direction.
3. **Part 3:** Right Boundary of the tree (excluding the leaf nodes), traversed in the reverse direction.

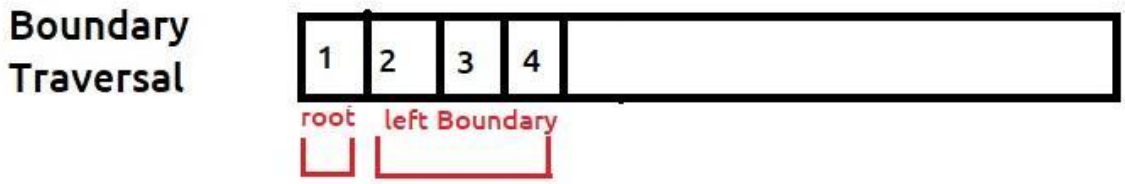
We take a simple data structure like a vector/Arraylist to store the Boundary Traversal. The root node is coming from both the boundaries (left and right). Therefore, to avoid any confusion, we push it on our list at the very start.



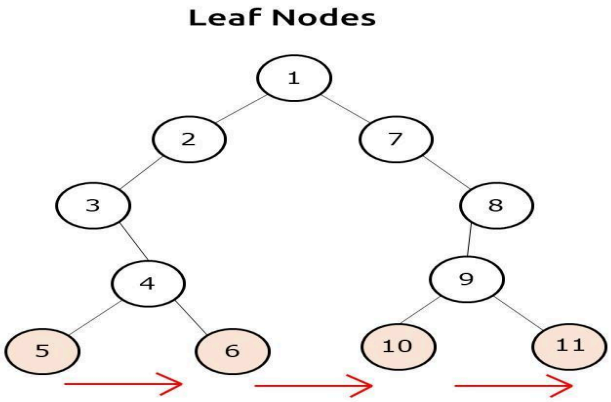
We will now see the approach to finding these three parts. **Part 1:** Left Boundary



To traverse the left boundary, we can set a simple iteration. Initially, we make the cur pointer point to the root's left. In every iteration, if the cur node is not a leaf node, we print it. Then we always try to move left of the cur pointer. If there is no left child, then we move to the right of cur and in the next iteration, again try to move to the left first. We stop our execution when cur is pointing to NULL.

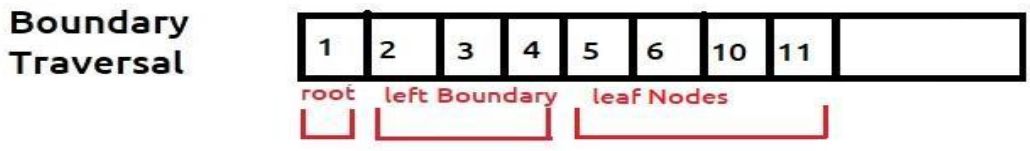


Part 2: Leaf nodes

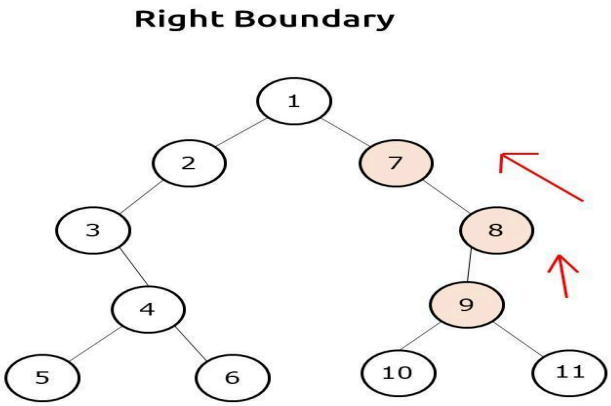


To print the leaf nodes, we can do a simple preorder traversal, and check if the current node is a leaf node or not. If it is a leaf node just print it.

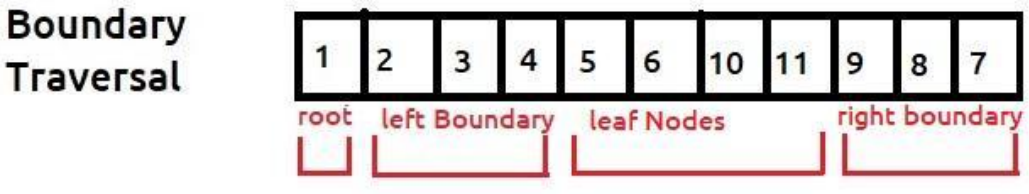
Please note that we want the leaves to come in a specific order which is bottom-left to top-right, therefore a level order traversal will not work because it will print the upper-level leaves first. Therefore, we use a preorder traversal.



Part 3: Right Boundary



We need to print the right boundary in the **Reverse** direction. It is very similar to the left boundary traversal. We initialize our cur pointer to the right child of the root node and set an iterative loop. To achieve the reverse direction, we take an auxiliary list. In every iteration, we check if the current node is not a leaf node then we push it to the auxiliary list. Then we first try to move right of cur, if there is no right child only then we move left. We stop our execution once cur points to NULL.
Now the auxiliary list contains the nodes of the right boundary. We iterate from the end to start off this list and in every iteration, push the value to our main boundary traversal list. This way we get the nodes in the reverse direction.



Dry Run: In case you want to watch the dry run for this approach, please watch the video attached below.

Code:

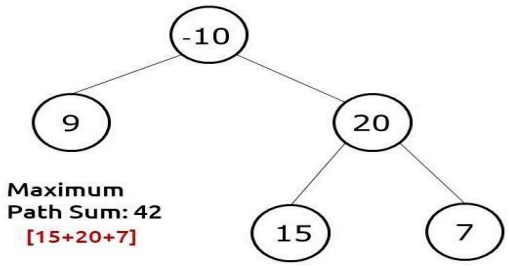
```
import java.util.*;
class Node {
    int data;
    Node left, right;
    Node(int data) { this.data = data; left = null; right = null; } }
public class Main {
    static Boolean isLeaf(Node root) {
        return (root.left == null) && (root.right == null);
    }
```

```
    static void addLeftBoundary(Node root, ArrayList < Integer > res) {
        Node cur = root.left;
        while (cur != null) {
            if (isLeaf(cur) == false) res.add(cur.data);
            if (cur.left != null) cur = cur.left;
            else cur = cur.right;
        }
    }
    static void addRightBoundary(Node root, ArrayList < Integer > res) {
        Node cur = root.right;
        ArrayList < Integer > tmp = new ArrayList < Integer > ();
        while (cur != null) {
            if (isLeaf(cur) == false) tmp.add(cur.data);
            if (cur.right != null) cur = cur.right;
            else cur = cur.left;
        }
        for (int i = tmp.size() - 1; i >= 0; --i) {
            res.add(tmp.get(i));
        }
    }
    static void addLeaves(Node root, ArrayList < Integer > res) {
        if (isLeaf(root)) {
            res.add(root.data);
            return;
        }
        if (root.left != null) addLeaves(root.left, res);
        if (root.right != null) addLeaves(root.right, res);
    }
    static ArrayList < Integer > printBoundary(Node node) {
        ArrayList < Integer > ans = new ArrayList < Integer > ();
        if (isLeaf(node) == false) ans.add(node.data);
        addLeftBoundary(node, ans);
        addLeaves(node, ans);
        addRightBoundary(node, ans);
        return ans;
    }
    public static void main(String args[]) {
        Node root = new Node(1);
        root.left = new Node(2);
        root.left.left = new Node(3);
        root.left.left.right = new Node(4);
        root.left.left.right.left = new Node(5);
        root.left.left.right.right = new Node(6);
        root.right = new Node(7);
        root.right.right = new Node(8);
        root.right.right.left = new Node(9);
        root.right.right.left.left = new Node(10);
        root.right.right.left.right = new Node(11);

        ArrayList < Integer > boundaryTraversal;
        boundaryTraversal = printBoundary(root);

        System.out.println("The Boundary Traversal is : ");
        for (int i = 0; i < boundaryTraversal.size(); i++) {
            System.out.print(boundaryTraversal.get(i) + " "); }}}
```

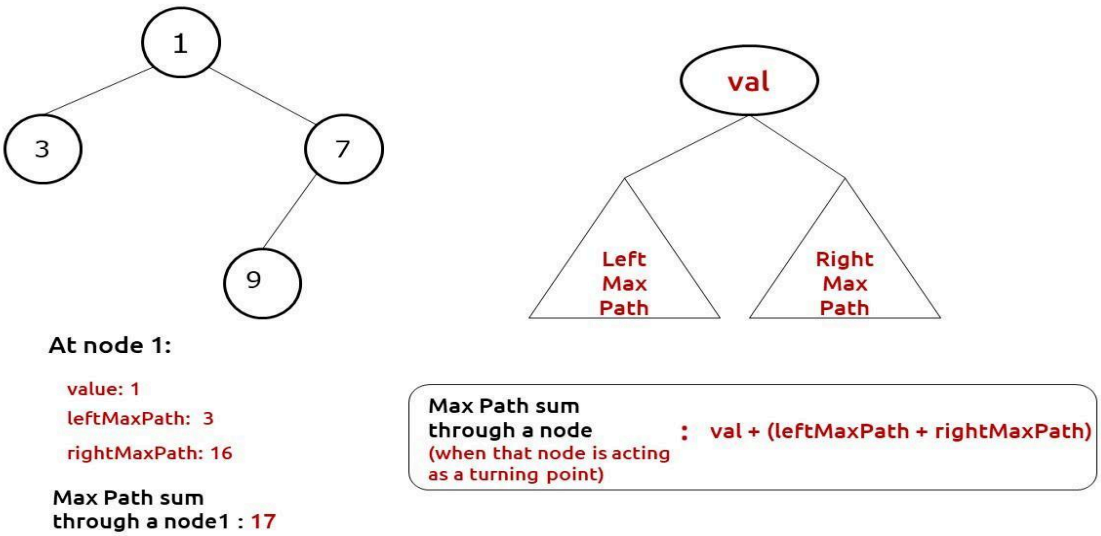
20. Maximum path sum of BT

Problem Statement: Write a program to find the maximum sum path in a binary tree. A path in a binary tree is a sequence of nodes where every adjacent pair of nodes are connected by an edge. A node can only appear in the sequence at most once. A path need not pass from the root. We need to find the path with the maximum sum in the binary tree.

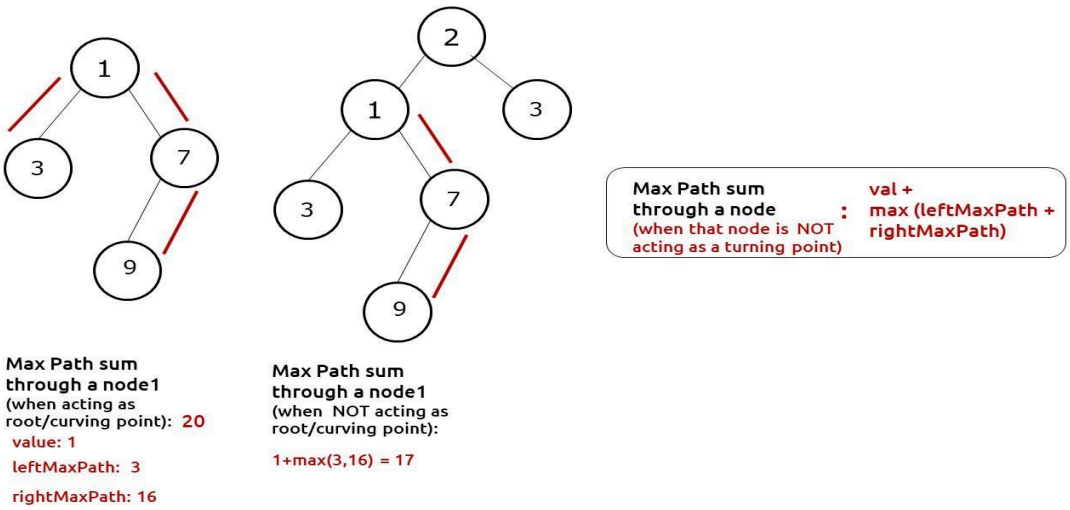
Example: Input:



Output: The Max Path Sum for the Tree is 42
Solution N Approach: A brute force approach would be to generate all paths and compare them. Generating all paths will be a time-costly activity therefore we need to look for something else.
We first need to define what is the maximum path sum through a given node (when that node is acting as the root node/curving point). At a given node with a value, if we find the max leftSumPath in the left-subtree and the max rightSumPath in the right subtree, then the maxPathSum through that node is value+(leftSumPath+rightSumPath).



Now we can apply this formula at every node by doing a simple tree traversal and storing the maximum value (our answer) in a reference variable.
For our recursion to work, it is very important to understand what value we return from our function. In our recursive function, we find and compare the maxPathSum from a given node when it is the root/turning point of the path. But what we return is the maxPathSum of that same node when it is **NOT** the root/turning point of the path. To find the latter maxPath, we no longer have the liberty to consider both leftMaxPath and rightMaxPath, we will simply take the maximum of the two and it to the value of the node.



To summarize:

- Initialize a maxi variable to store our final answer.
- Do a simple tree traversal. At each node, find recursively its leftMaxPath and its rightMaxPath.
- Calculate the maxPath through the node as val + (leftMaxPath + rightMaxPath) and update maxi accordingly.
- Return the maxPath when node is not the curving point as val + max(leftMaxPath, rightMaxPath).

Code:

```
class TUF {
    public static int maxPathSum(Node root) {
        int maxValue[] = new int[1];
        maxValue[0] = Integer.MIN_VALUE;
        maxPathDown(root, maxValue);
        return maxValue[0];
    }
    public static int maxPathDown(Node node, int maxValue[]) {
        if (node == null) return 0;
        int left = Math.max(0, maxPathDown(node.left, maxValue));
        int right = Math.max(0, maxPathDown(node.right, maxValue));
        maxValue[0] = Math.max(maxValue[0], left + right + node.val);
        return Math.max(left, right) + node.val;
    }}
```

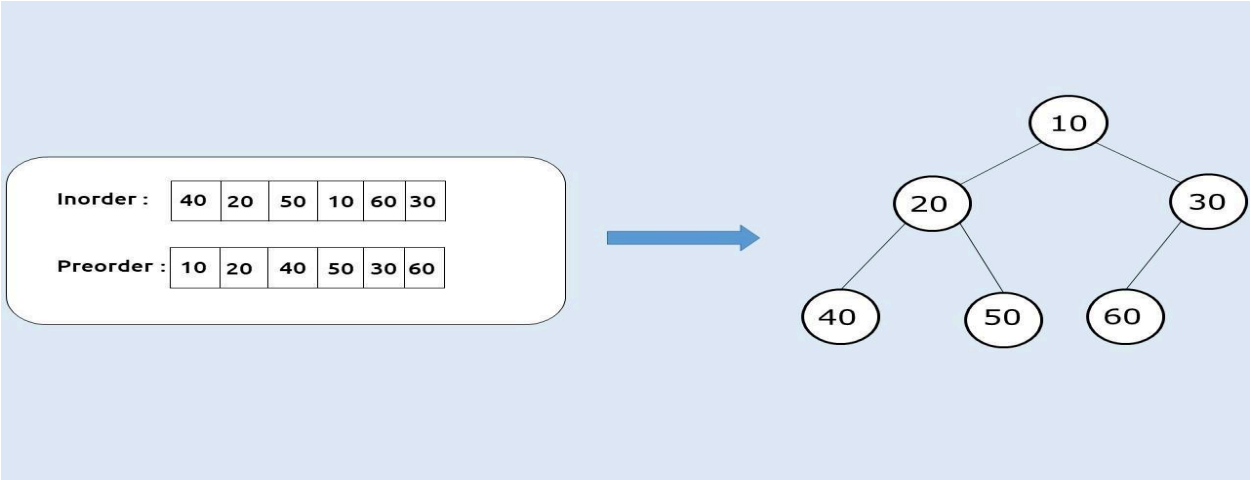
21. Construct the BT from in order to preorder

In this article we will solve the most asked coding interview problem: Construct A Binary Tree from Inorder and Preorder Traversal.

Construct A Binary Tree from Inorder and Preorder Traversal

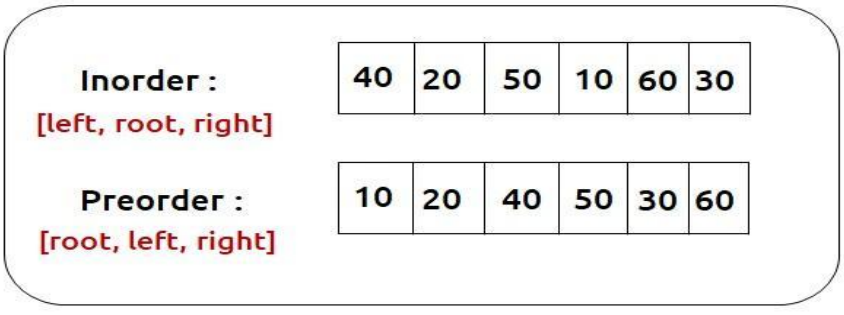
Given the Inorder and Preorder Traversal of a binary tree, we need to construct the unique binary tree represented by them.

Example:



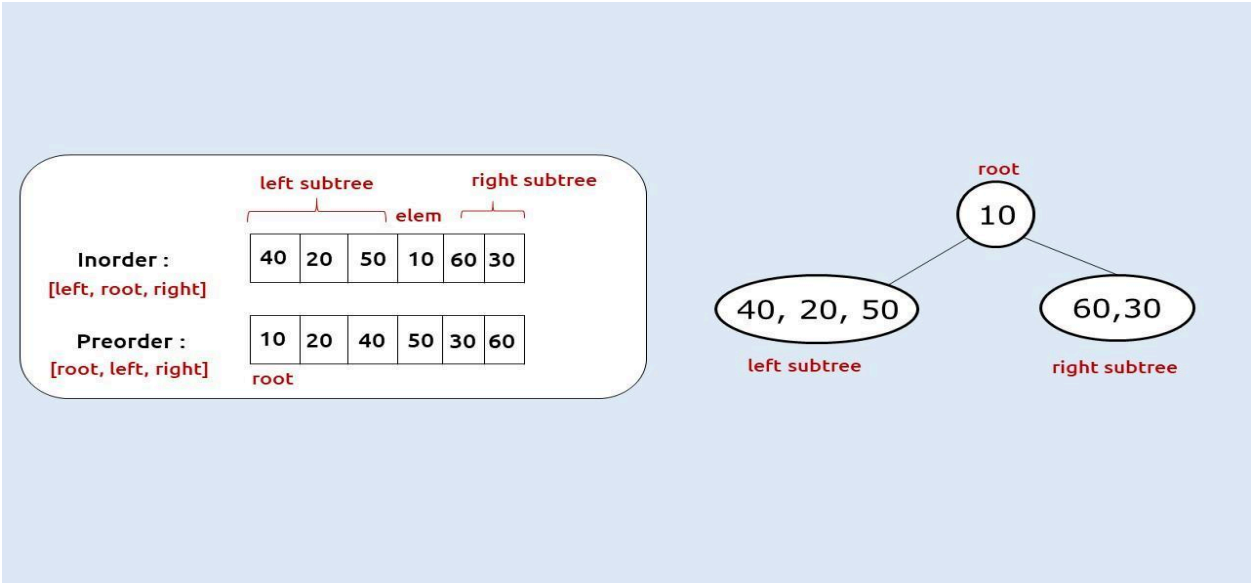
Solution: In this article, we learned that a unique binary tree can be constructed using a preorder and an inorder traversal. Here we will discuss the solution. **Intuition:** Inorder traversal is a special traversal that helps us to identify a node and its left and right subtree. Preorder traversal always gives us the root node as the first element. Using these properties we can construct the unique binary tree.

Given this example:

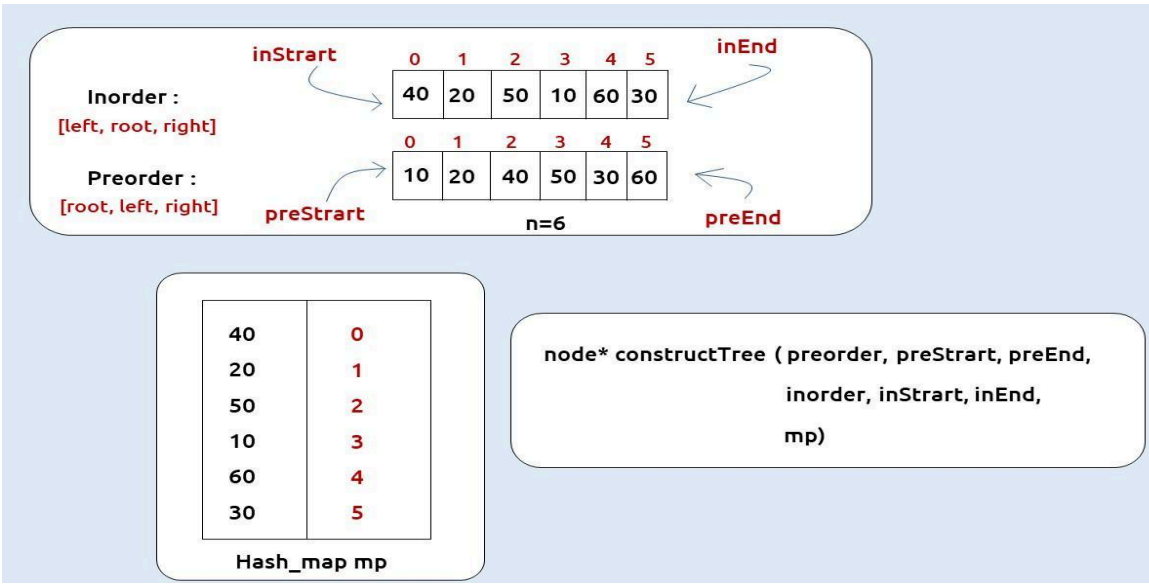


Here 10 (first element of preorder) is the root element. So we can find its index in the inorder traversal(say elem). The left subtree of the root will be present to the left side of in order whereas the right subtree of root will be present on the right side of elem in the inorder traversal:

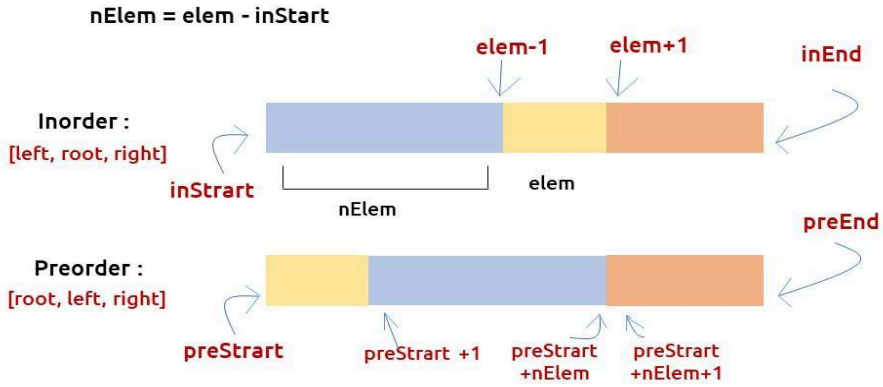
We can define a recursive function that creates one node at a time. First, we create the root node, and then we can take the help of recursion to create its left and right subtrees. In order to make recursion work, we need to provide the correct inorder and preorder traversal of the subtree for every recursive call.



To make more efficient function calls we can use variables (inStart, inEnd) and (preStart and preEnd) in order to point to the start and end of the inorder and preorder traversal respectively, and avoid copying of arrays. Next, we need to figure out how we are going to search the root index in the inorder traversal. For this, we have two options: Linear Search and Hashmaps. We will choose the second one because it will return us the index in constant time. Before making the first recursive call, we will simply add all the (value, index) pairs to a map and pass it to our recursive function. If n is the size of the Inorder traversal/Preorder traversal. Then our first function call will be :



Now the main task left is to pass the correct preStart, preEnd, inStart, inEnd to the respective recursive calls for the left and right subtree. We can calculate the number of elements in the left subtree from the root index, say nElems (elem – InStart, where elem is the index of root in inorder traversal). As inorder is **[left, root, right] and preorder is [root, left, right]** the number of elements (nElems) will easily tell us the preorder and inorder traversal of the subtrees according to the following table:



From Parent Function : (prestart, preEnd, inStart, inEnd)

	preStart	preEnd	inStart	inEnd
left subtree	prestart+1	prestart+ nElem	inStart	elem-1
right subtree	prestart+ nElem+1	preEnd	elem+1	inEnd

The base case will be when inStart> inEnd or preStart > preEnd, in that case, we can simply return NULL.

Approach:

The algorithm approach can be stated as:

- Create a map to store the inorder indexes.
- Call the function constructTree with all 7 parameters as shown above.
- In the recursive function, first check the base case, if (preStart,>preEnd || inStart> inEnd) then return NULL.
- Construct a node (say root) with the root value(first element of preorder).
- Find the index of the root, say elem from the hashmap.
- Find the number of elements (say nElem) in the left subtree = elem – inStart
- Call recursively for the left subtree with correct values (shown in the above table) and store the answer received in root->left.
- Call recursively for the right subtree with correct values (shown in the above table) and store the answer received in root->right.
- Return root

Dry Run: To understand a detailed dry run of this approach, please watch the video attached below.

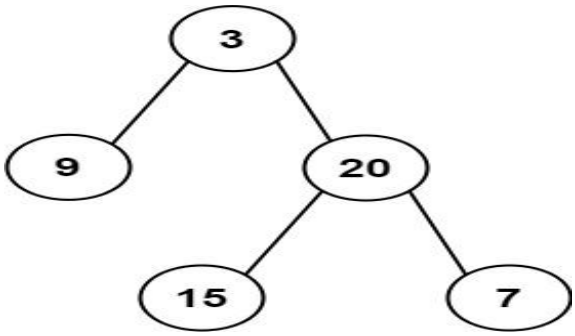
Code:

```
class TreeNode {
  int val;
  TreeNode left;
  TreeNode right;
  TreeNode(int val) {  this.val = val; }}

//utility function to insert node in the list
class TUF {
  static TreeNode buildTree(int[] preorder, int[] inorder) {
    Map < Integer, Integer > inMap = new HashMap < Integer, Integer > ();
    for (int i = 0; i < inorder.length; i++) {
      inMap.put(inorder[i], i);
    }

    TreeNode root = buildTree(preorder, 0, preorder.length - 1, inorder, 0,
    inorder.length - 1, inMap);
    return root;
  }

  static TreeNode buildTree(int[] preorder, int preStart, int preEnd, int[]
  inorder, int inStart, int inEnd, Map < Integer, Integer > inMap) {
    if (preStart > preEnd || inStart > inEnd) return null;

    TreeNode root = new TreeNode(preorder[preStart]);
    int inRoot = inMap.get(root.val);
    int numsLeft = inRoot - inStart;

    root.left = buildTree(preorder, preStart + 1, preStart + numsLeft, inorder,
    inStart, inRoot - 1, inMap);
    root.right = buildTree(preorder, preStart + numsLeft + 1, preEnd, inorder,
    inRoot + 1, inEnd, inMap);

    return root; }

  public static void main(String args[]) {

    int preorder[] = {10,20,40,50,30,60};
    int inorder[] = {40,20,50,10,60,30};
    TreeNode root = buildTree(preorder, inorder);
  }
}
```

22. Construct Binary Tree from Inorder and Postorder

Given two integer arrays `inorder` and `postorder` where `inorder` is the inorder traversal of a binary tree and `postorder` is the postorder traversal of the same tree, construct and return *the binary tree*.

Example 1:




```
Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]

Output: [3,9,20,null,null,15,7]
```

Example 2: `Input: inorder = [-1], postorder = [-1] Output: [-1]`

```
Method - 1 recursion

class Solution {
    HashMap<Integer, Integer> hm = new HashMap();
    int ind;
    public TreeNode buildTree(int[] inorder, int[] postorder) {
        ind = postorder.length - 1;
        for (int i = 0; i < inorder.length; i++) hm.put(inorder[i],i);
        return build(postorder, 0, postorder.length - 1);
    }
    private TreeNode build(int[] P, int st, int end){
        if (st > end || ind < 0) return null;
        TreeNode node = new TreeNode(P[ind--]);

        node.right = build(P, hm.get(node.val) + 1, end);
        node.left = build(P, st, hm.get(node.val) - 1);
        return node;
    }}

Method - 2 iterative
```

This is my iterative solution, think about "Constructing Binary Tree from inorder and preorder array", the idea is quite similar. Instead of scanning the preorder array from beginning to end and using inorder array

as a kind of mark, in this question, the key point is to scanning the postorder array from end to beginning and also use inorder array from end to beginning as a mark because the logic is more clear in this way.
The core idea is: Starting from the last element of the postorder and inorder array, we put elements from postorder array to a stack and each one is the right child of the last one until an element in postorder array is equal to the element on the inorder array. Then, we pop as many as elements we can from the stack and decrease the mark in inorder array until the peek() element is not equal to the mark value or the stack is empty. Then, the new element that we are gonna scan from postorder array is the left child of the last element we have popped out from the stack.

```
public TreeNode buildTree(int[] inorder, int[] postorder) {
    if (inorder.length == 0 || postorder.length == 0) return null;
    int ip = inorder.length - 1;
    int pp = postorder.length - 1;

    Stack<TreeNode> stack = new Stack<TreeNode>();
    TreeNode prev = null;
    TreeNode root = new TreeNode(postorder[pp]);
    stack.push(root);
    pp--;

    while (pp >= 0) {
        while (!stack.isEmpty() && stack.peek().val == inorder[ip]) {
            prev = stack.pop();
            ip--;
        }
        TreeNode newNode = new TreeNode(postorder[pp]);
        if (prev != null) {
            prev.left = newNode;
        } else if (!stack.isEmpty()) {
            TreeNode currTop = stack.peek();
            currTop.right = newNode;
        }
        stack.push(newNode);
        prev = null;
        pp--;
    }

    return root;
}
```

23. Symmetric Binary Tree

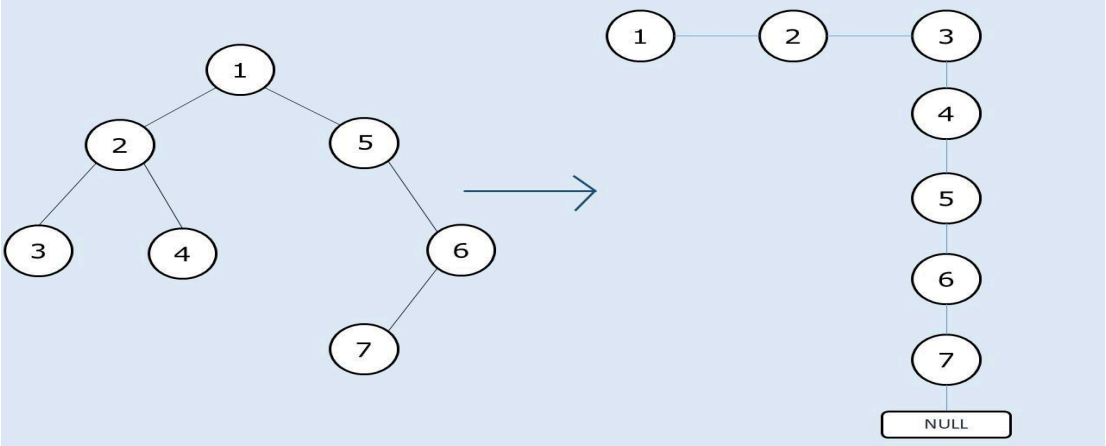
24. Flatten Binary Tree to LinkedList

Problem Statement: Flatten Binary Tree To Linked List. Write a program that flattens a given binary tree to a linked list. **Pre-req: Traversal Techniques**

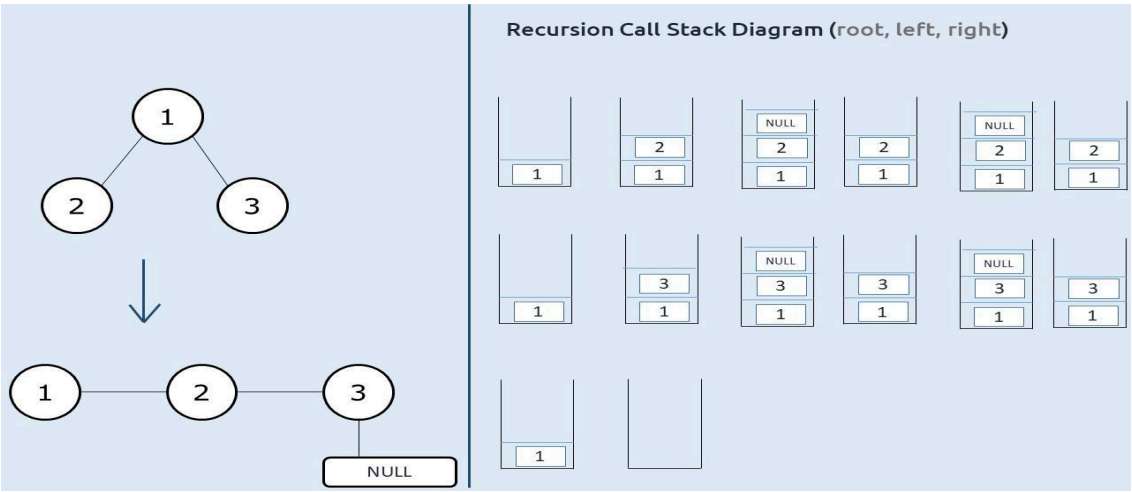
Note: The sequence of nodes in the linked list should be the same as that of the **preorder traversal** of the binary tree.

- The linked list nodes are the same binary tree nodes. You are not allowed to create extra nodes.
- The right child of a node points to the next node of the linked list whereas the left child points to NULL.

Example:



Solution 1: Using Recursion Intuition: The sequence of nodes in the linked list should be that of the preorder traversal. We will take a smaller example.

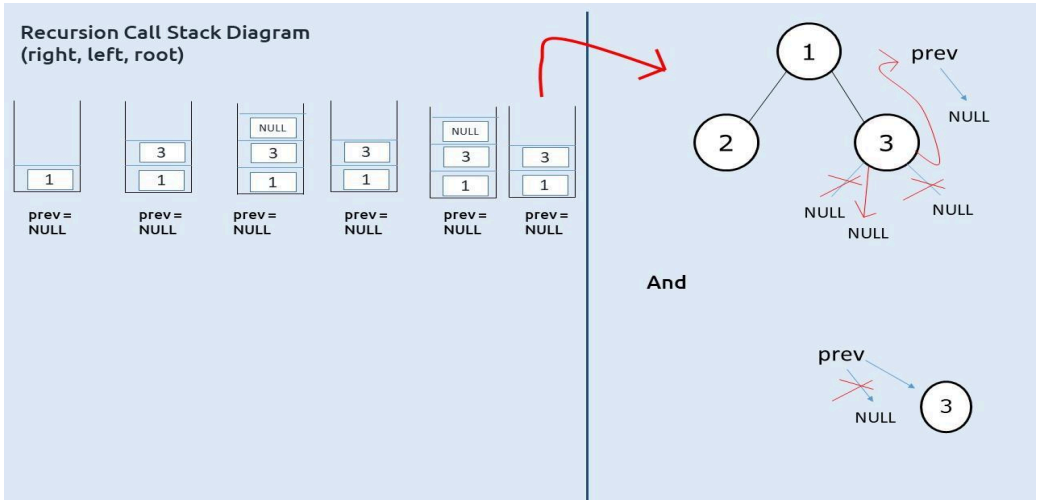


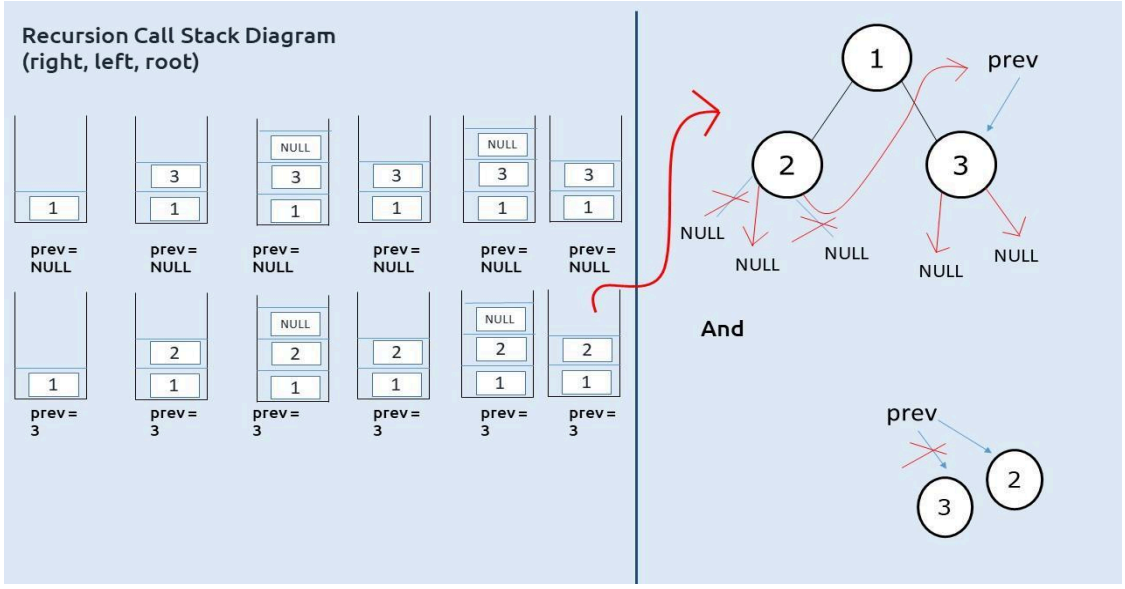
At starting node 1, what we finally want is that node 1's right child should point to its current left child (node 2). Now If we set it like this in our preorder traversal there will be no way to reach node 3. We need to modify our traversal technique. If we somehow start at node 3 (last node of the linked list), we need not traverse its right child as it is NULL, therefore we can straightaway set its right child to its left child(which is again NULL) and set its left child to NULL. Now we need to get to the second last node of the linked list (node 2) and set the right child to node 3. After that, we need to move to the third last node i.e node 1.

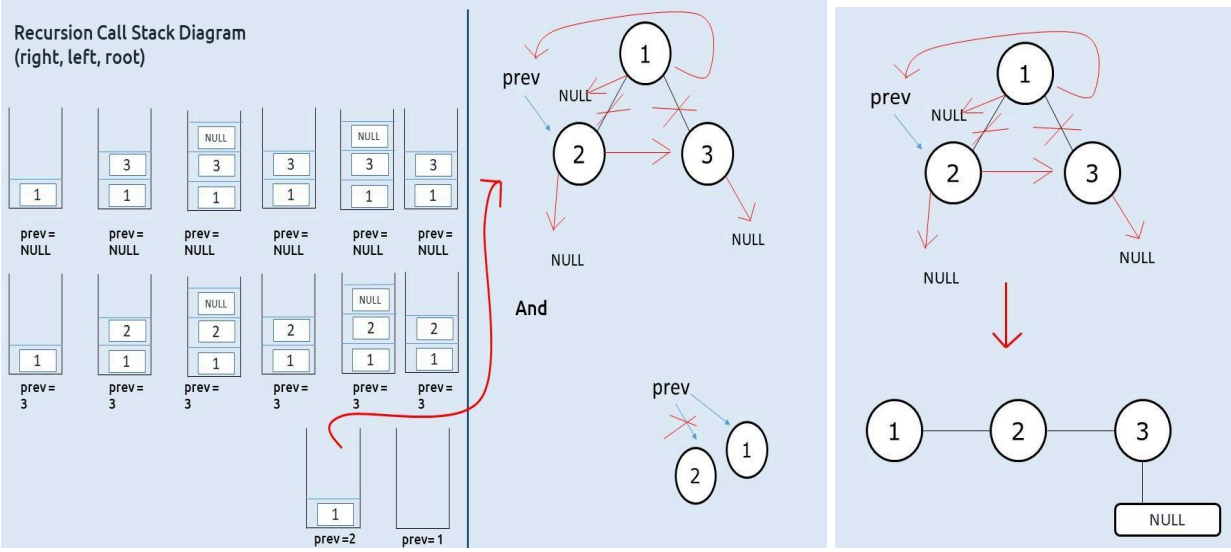
Approach: The algorithm steps can be stated as:

- If we observe, we are moving in a **reverse postorder** way : i.e **right, left, root.**
- We take a reference variable (say prev) to store the previous node(initialized to NULL).
- Whenever we visit a node, we set the right child to the prev and left child to NULL.
- Next we assign this current node to prev.
- We perform the above two operations on all the nodes in the traversal.

Dry run: The following illustrations will give a clear idea.





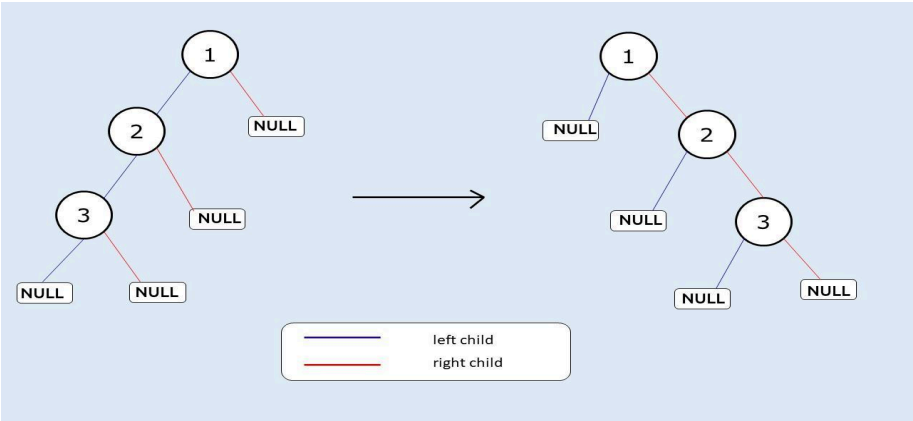


Code:

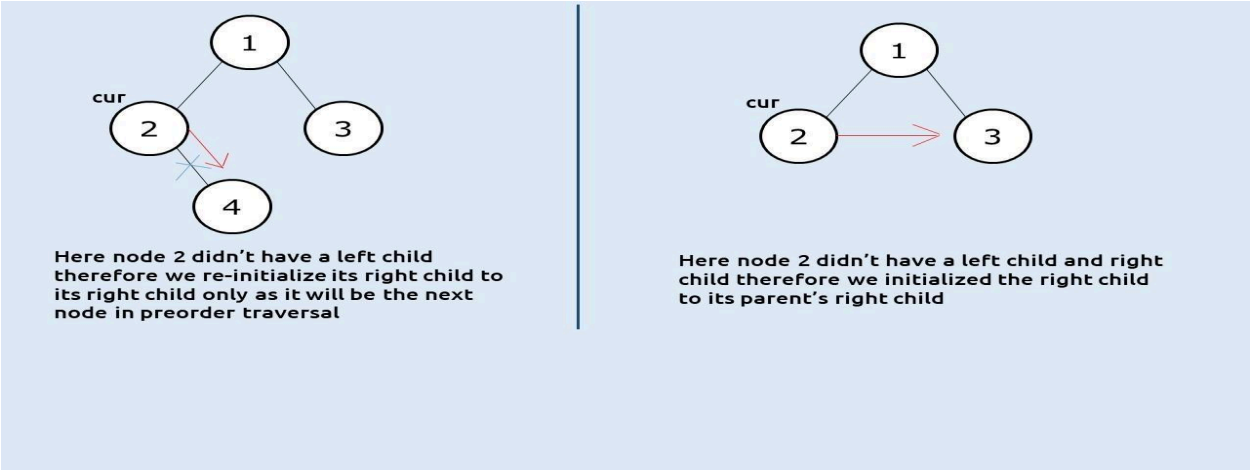
```
public class Solution {
  static Node prev = null;
    static void flatten(Node  root) {
      if (root == null) return;

      flatten(root . right);
      flatten(root . left);
      root.right = prev;
      root.left = null;
      prev = root;
    }}
```

Solution 2: Iterative Approach – Using Stack

Intuition: In a binary tree, generally, we need to set the right child of the node to the left and the left child to NULL. If the given tree is a left-skewed tree, this action alone will flatten the binary tree.



Now the main question arises is what if the current node doesn't have a left child? In that case, we don't want to assign its right child to NULL(its left child), rather we want it to assign to itself so that our preorder sequence is maintained. In case the right child is also not present(a leaf node) we would want to assign the right child to some parent node's right child.

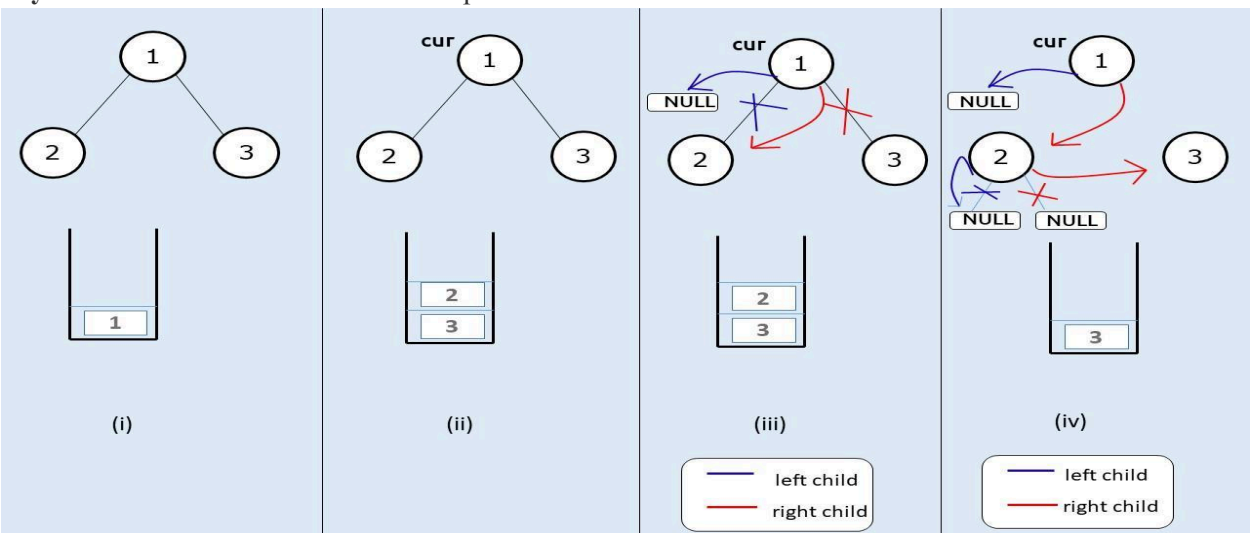


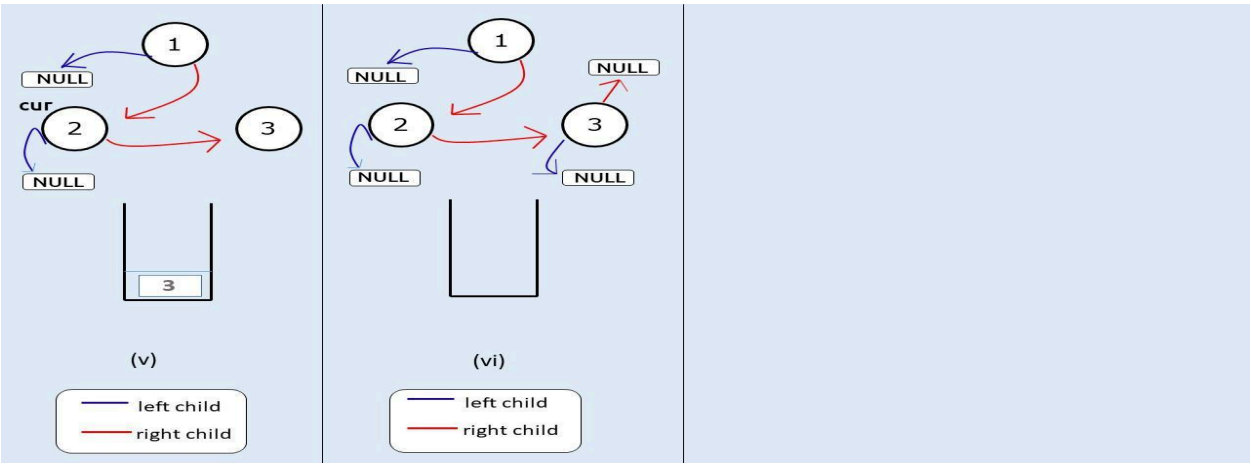
To get to this parent's right node we will use a stack. Whenever we are at a node we want to prioritize its left child if it is present. If it is not present we want to look at the right child. A stack is a LIFO data structure, we first push the right child and then the left child. Then we set the right child of the node to the stack's top and left child as NULL. This way the stack always provides the correct next node.

Approach: The algorithm approach can be stated as:

- Take a stack and push the root node to it.
- Set a while loop till the stack is non-empty.
- In every iteration, take the node at the top of the stack(say cur) and pop the stack.
- If cur has a right child, push it to the stack.
- If cur has a left child, push it to the stack.
- Set the right child of cur to node at stack's top.
- Set the left child of cur as NULL.

Dry Run: We will take a smaller example.





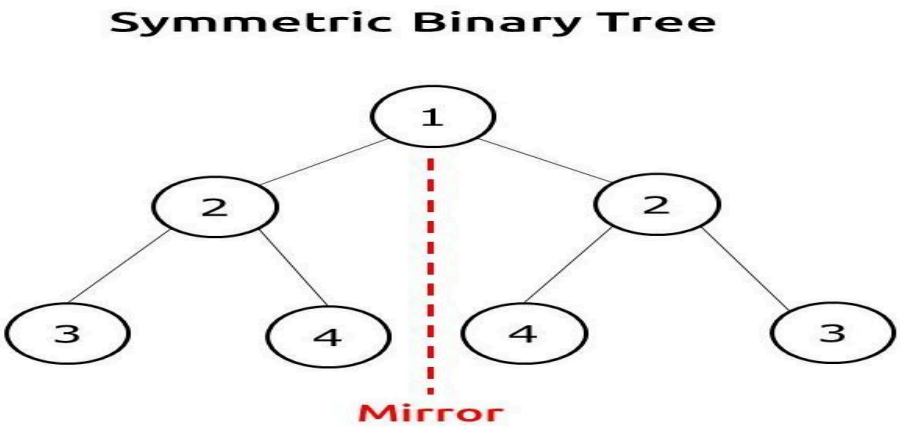

```
public class Solution {
  static Node prev = null;
    static void flatten(Node  root) {
  if (root == null) return;
  Stack < Node  > st=new Stack<>();
  st.push(root);
  while (!st.isEmpty()) {
    Node  cur = st.peek();
    st.pop();

    if (cur.right != null) {
      st.push(cur.right);
    }
    if (cur.left != null) {
      st.push(cur.left);
    }
    if (!st.isEmpty()) {
      cur.right = st.peek();
    }
    cur.left = null;
  }}
```

25. Check if Binary Tree is the mirror of itself or not

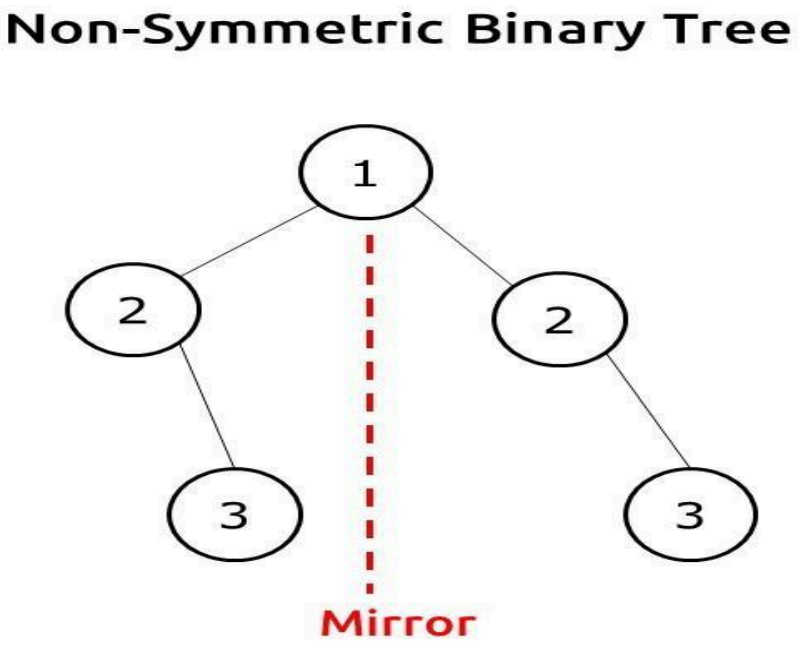
Check for Symmetrical Binary Trees Write a program to check whether a binary tree is symmetrical or not.

Problem Description: A symmetrical binary tree is a tree that forms a mirror of itself around the center. In other words, every node in the left subtree will have a mirror image in the right subtree.

Example 1:



Example 2:

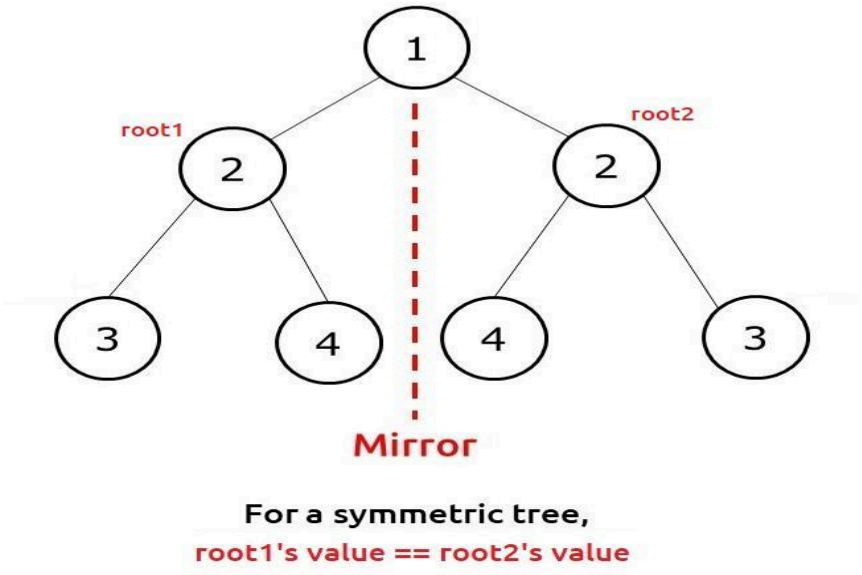


Disclaimer: *Don't jump directly to the solution, try it out yourself first.*

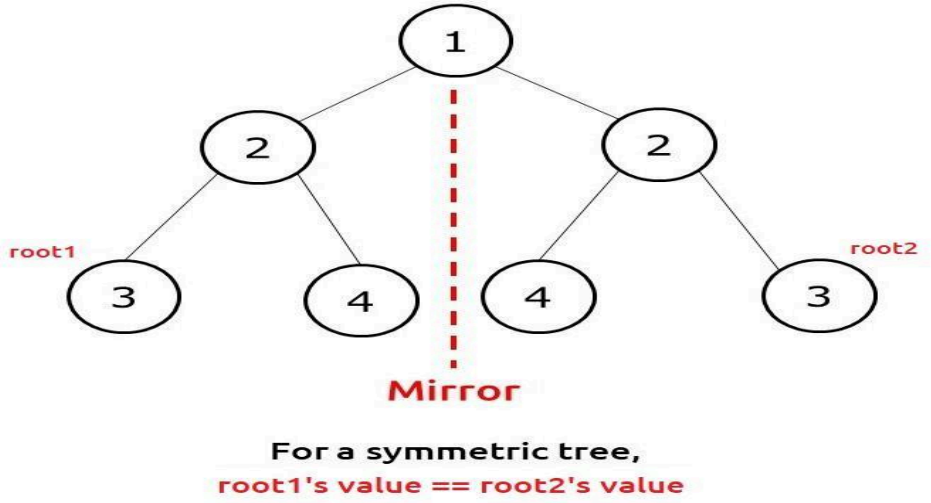
***Pre-req**: Tree Traversal*

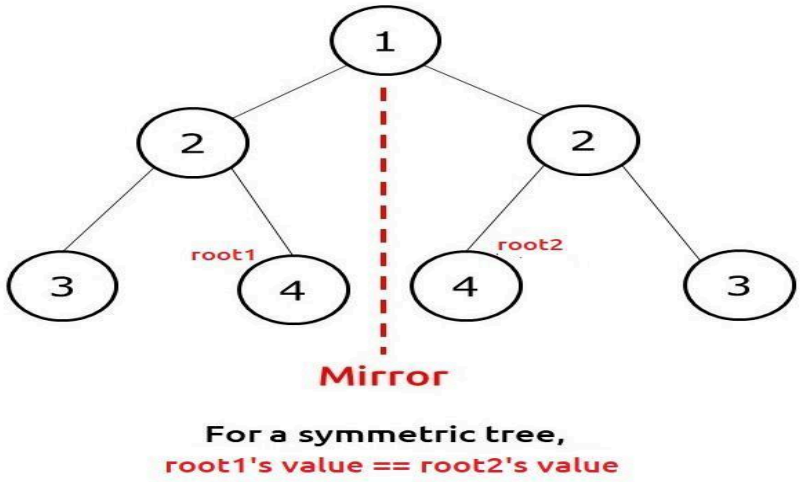
Solution N Intuition: We need to understand the property of the mirror. We can ignore the root node as it is lying on the mirror line. In the next level, for a symmetric tree, the node at the root's left should be equal to the node at the root's right.

If we take two variables root1 and root2 to represent the left child of root and right child of the root, then



Further, we need to understand that when root1's value is equal to root2's value, we need to further check for its children. As we are concerned about node positions through a mirror, root1's left child should be checked with root2's right child and root1's right child should be checked with root2's left child.

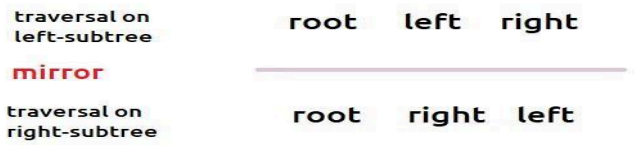




Approach: The algorithm steps can be summarized as follows::

- We take two variables root1 and root2 initially both pointing to the root.
- Then we use any tree traversal to traverse the nodes. We will simultaneously change root1 and root2 in this traversal function.
- For the base case, if both are pointing to NULL, we return true, whereas if only one points to NULL and other to a node, we return false.
- If both points to a node, we first compare their value, if it is the same we check for the lower levels of the tree.
- We recursively call the function to check the root1's left child with root2's right child; then we again recursively check the root1's right child with root2's left child.

- When all three conditions (node values of left and right and two recursive calls) return true, we return true from our function else we return false.



Dry Run: In case you want to watch the dry run for this approach, please watch the video attached below.

Code:

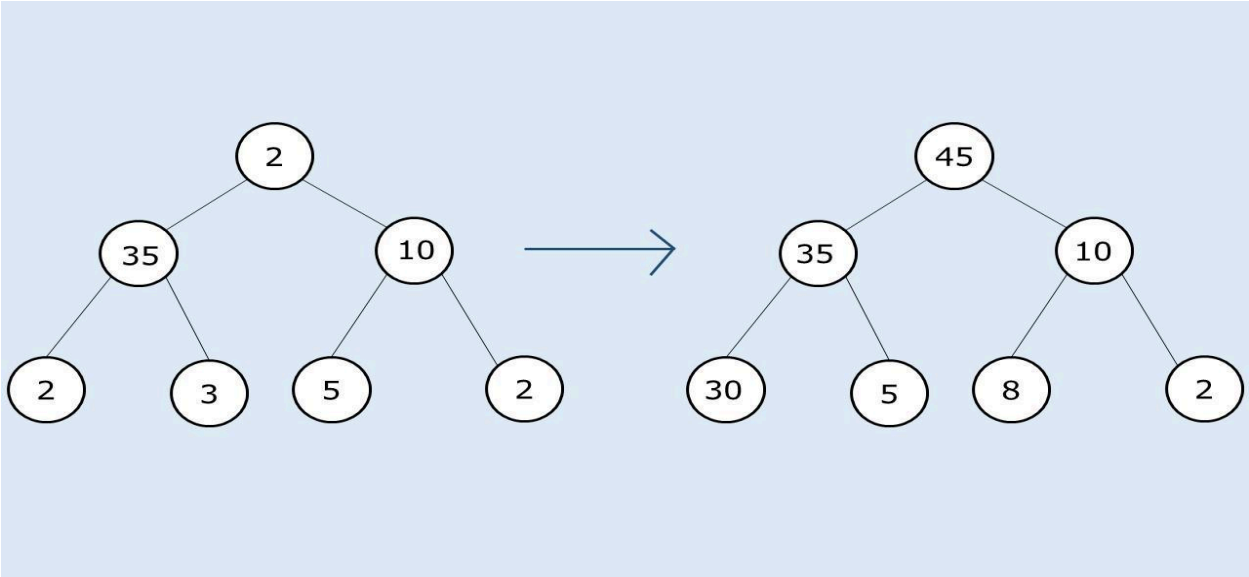
```
static boolean isSymmetricUtil(Node  root1, Node  root2) {
  if (root1 == null || root2 == null)
    return root1 == root2;
  return (root1 . data == root2 . data) && isSymmetricUtil(root1 . left, root2 .
  right) && isSymmetricUtil(root1 . right, root2 . left);
}
static boolean isSymmetric(Node  root) {
  if (root==null) return true;
  return isSymmetricUtil(root . left, root . right);
}
```

26. Check for Children Sum Property

Problem Statement: Children Sum Property in a Binary Tree. Write a program that converts any binary tree to one that follows the children sum property.
The children sum property is defined as, For every node of the tree, the value of a node is equal to the sum of values of its children(left child and right child).
Note:

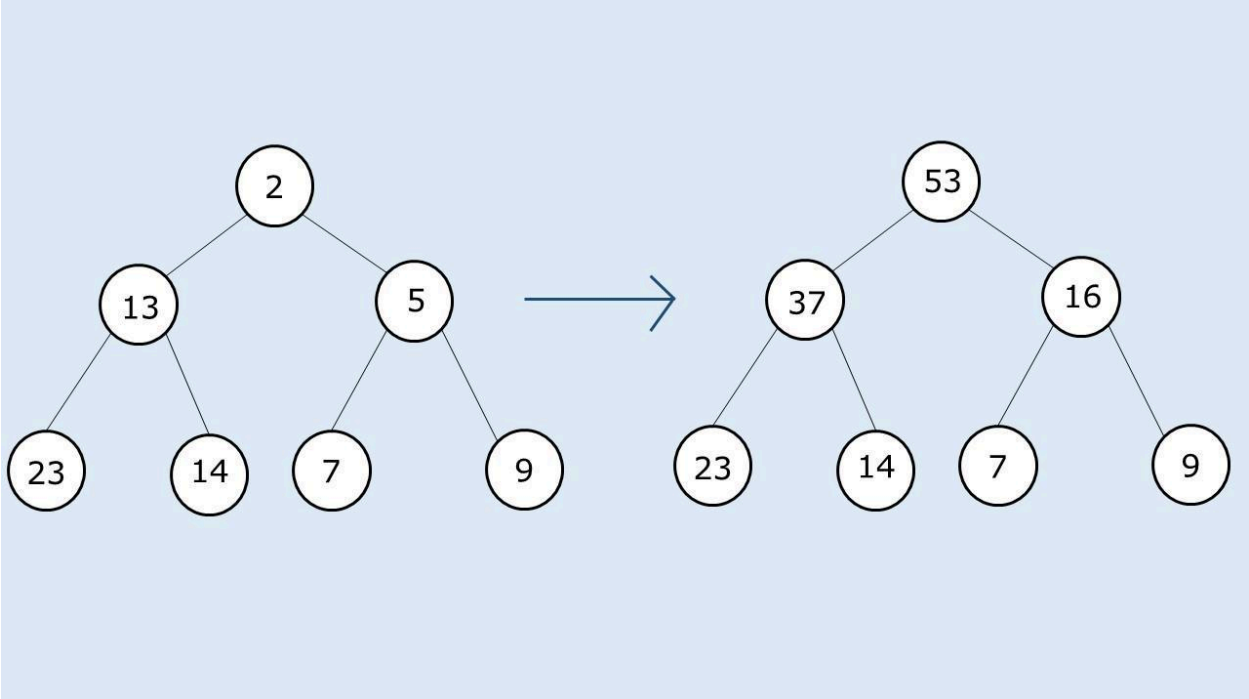
- The node values can be increased by 1 any number of times but decrement of any node value is not allowed.
- A value for a NULL node can be assumed as 0.
- You are not allowed to change the structure of the given binary tree.

Example:

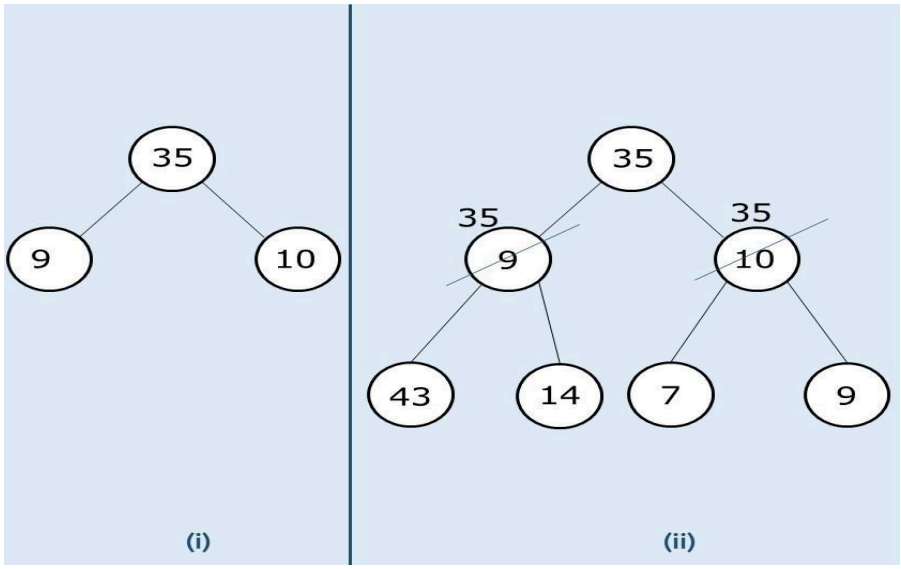


Pre-req: Tree Traversals

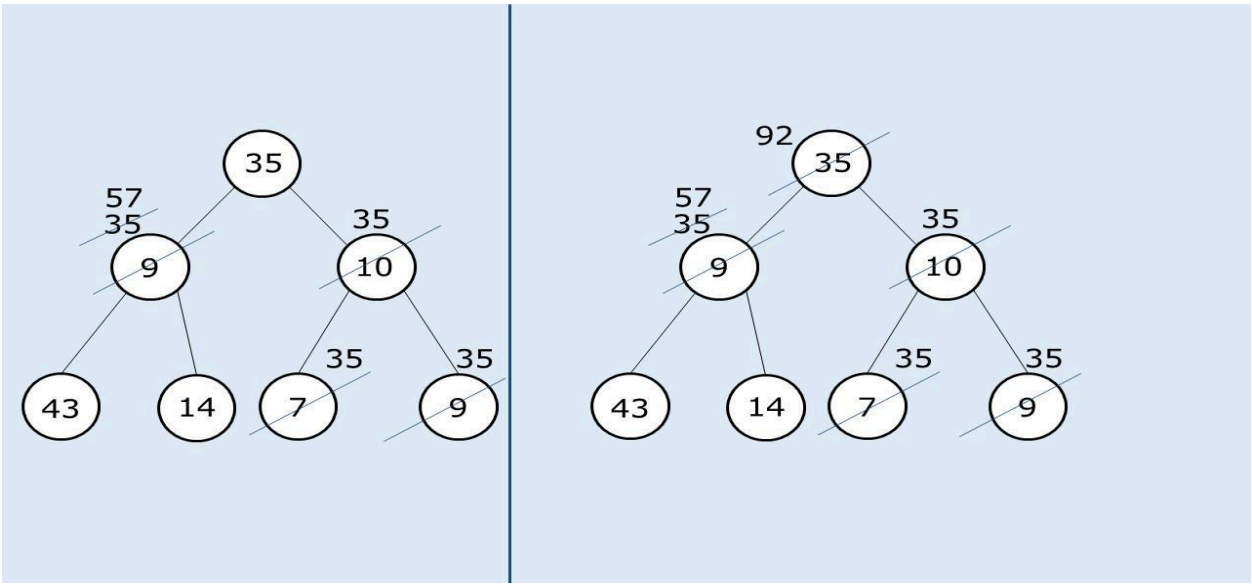
Solution N Intuition: The first intuition that comes to our mind is to do a tree traversal and at every node make the node value equal to the sum of value(s) of its children. This approach will work in a tree-like this(given below), if we do a postorder traversal and update the node values after visiting the children.



As in the given tree, every node value is less than the sum of the node values of its children; this approach works. Now we will see where this approach will fail and why. If we consider the following case:



- In case (i), we see that the root node's value is greater than the sum of both the leaf nodes. In such a case it is not possible to assign a node as the sum of its children(as we are not allowed to decrement a value). In such a case we simply assign the node value of the children to its parent.
- In case (ii) as we have a tree further down, we first implement the strategy discussed in case(i), now we see that at the second level we are having both the cases where, 35 is less than (43+14) and 35 is greater than (7+9). For the latter case we again implement the strategy and make both its children 35. For the former case, we simply traverse the tree and set it to the sum of its children.



Approach:

We perform a tree traversal and check whether the current node value is greater than the sum of node values of its children. If this is the case, we simply assign its children to the same value of the current node and then recurse for the children. We do so because we are not allowed to decrement a node value. So we set the children to a large value in order to increment the parent.

It can happen in subsequent recursions that this child value is further changed, therefore it is necessary that when we return to a node after returning from its children, we set it to the sum of node values of its children explicitly.

The algorithm approach can be stated as follows:

- We perform a simple dfs traversal on the tree.
- For the base case, if the node is pointing to NULL, we simply return.
- At every node, first we find the sum of values of the children(For a NULL child, value is assumed to be 0).
- If node's value > sum of children node value, we assign both the children's value to their parent's node value.
- Then we visit the children using recursion.
- After we return to the node after visiting its children, we explicitly set its value to be equal to the sum of its values of its children.

Code

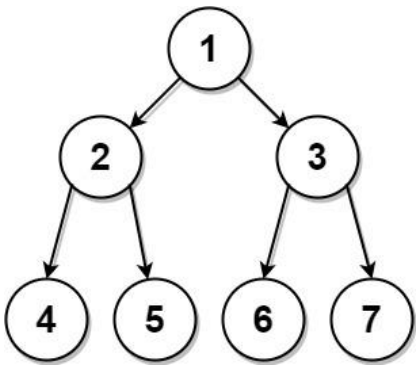
```
static void reorder(Node  root) {
  if (root == null) return;
  int child = 0;
  if (root . left!=null) {
    child += root . left . data;
  }
  if (root . right!=null) {
    child += root . right . data;
  }

  if (child < root . data) {
    if (root . left!=null) root . left . data = root . data;
    else if (root . right!=null) root . right . data = root . data;
  }

  reorder(root . left);
  reorder(root . right);

  int tot = 0;
  if (root . left!=null) tot += root . left . data;
  if (root . right!=null) tot += root . right . data;
  if (root . left!=null || root . right!=null) root . data = tot;
}
static void changeTree(Node  root) {
  reorder(root);
}
```

27. Construct Binary Tree from Preorder and Postorder Traversal

Given two integer arrays, `preorder` and `postorder` where `preorder` is the preorder traversal of a binary tree of distinct values and `postorder` is the postorder traversal of the same tree, reconstruct and return *the binary tree*.

If there exist multiple answers, you can return any of them.

Example 1:



Input: preorder = [1,2,4,5,3,6,7], postorder = [4,5,2,6,7,3,1] **Output:** [1,2,3,4,5,6,7]

Solution Explanation - We know that the first element in the pre array is the root. We also know that the second element in the pre array is the root of the left subtree. Now if we can find this second element in the post array then all elements before it and including it would belong to the left subtree and all elements after it (excluding the last element which is the root) will be the elements in the right subtree. We can apply this logic recursively while processing each element in the pre array and build the tree. We use a hashmap for fast lookups in the post array.

```
class Solution {
    private int preInd, len;
    private int[] preorder, postorder;
    private HashMap<Integer, Integer> postIndMap;

    public TreeNode constructFromPrePost(int[] preorder, int[] postorder) {
        int n = postorder.length;
        this.len = n;
        this.postIndMap = new HashMap();
        this.preorder = preorder;
        this.postorder = postorder;

        for (int i = 0; i < n; i++){
            postIndMap.put(postorder[i], i);
        }
        return buildTree(0, n - 1);
```

```
    }
    private TreeNode buildTree(int start, int end){
        if (preInd == len || start > end){  return null; }
        TreeNode node = new TreeNode(preorder[preInd++]);
        if (start == end) return node;

        int getPostInd = postIndMap.get(preorder[preInd]);
        node.left = buildTree(start, getPostInd);
        node.right = buildTree(getPostInd + 1, end - 1);

        return node;
    }
}
```

Other solution with O(N) on each iteration hence it becomes O(n2)**

```
class Solution {
    public TreeNode constructFromPrePost(int[] preorder, int[] postorder) {
        return find(preorder, 0, preorder.length - 1, postorder, 0, postorder.length
- 1);
    }

    public TreeNode find(int[] preorder, int prsi, int prei, int[] postorder, int
posi, int poei) {
        if(prsi > prei || posi > poei)
            return null;
      if(prsi == prei || posi == poei){
            TreeNode res = new TreeNode(preorder[prei]);
             return res;
        }
      int rootval = preorder[prsi], leftval = preorder[prsi + 1];
      int leftidx = posi;
      while(postorder[leftidx] != leftval)
            leftidx++;
      TreeNode root = new TreeNode(rootval);
      int len = leftidx - posi + 1;
      root.left = find(preorder, prsi + 1, prsi + len, postorder, posi, leftidx);
      root.right = find(preorder, prsi + len + 1, prei, postorder, leftidx + 1, poei
- 1);
      return root;
    }
}
```

Iterative Solution

We will pre order generate TreeNodes, push them to `stack` and postorder pop them out.

1. Iterate on a pre array and construct nodes one by one.
2. `stack` save the current path of the tree.
3. `node = new TreeNode(pre[i])`, if not left child, add node to the left. otherwise add it to the right.

4. If we meet the same value in the pre and post, it means we complete the construction for the current subtree. We pop it from the stack.

```
public TreeNode constructFromPrePost(int[] pre, int[] post) {
        Deque<TreeNode> s = new ArrayDeque<>();
        s.offer(new TreeNode(pre[0]));
        for (int i = 1, j = 0; i < pre.length; ++i) {
            TreeNode node = new TreeNode(pre[i]);
            while (s.getLast().val == post[j]) {
                s.pollLast(); j++;
            }
            if (s.getLast().left == null) s.getLast().left = node;
            else s.getLast().right = node;
            s.offer(node);
        }
        return s.getFirst();
    }
```

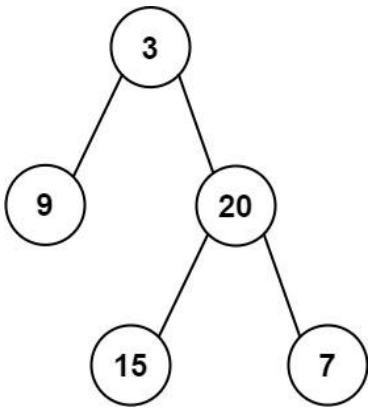
Recursive Solution

Create a node `TreeNode(pre[preIndex])` as the root. Because root node will be lastly iterated in post order, `if root.val == post[posIndex]`, it means we have constructed the whole tree, If we haven't completed constructed the whole tree, So we recursively `constructFromPrePost` for the left subtree and right subtree.

And finally, we'll reach the `posIndex` that `root.val == post[posIndex]`. We increment `posIndex` and return our `root` node.

```
int preIndex = 0, posIndex = 0;
    public TreeNode constructFromPrePost(int[]pre, int[]post) {
        TreeNode root = new TreeNode(pre[preIndex++]);
        if (root.val != post[posIndex])
            root.left = constructFromPrePost(pre, post);
        if (root.val != post[posIndex])
            root.right = constructFromPrePost(pre, post);
        posIndex++;
        return root;
    }
```

VARIATION: Construct Binary Tree from Preorder and Inorder Traversal Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return *the binary tree*.



Example 1:

```
Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]

Output: [3,9,20,null,null,15,7]
```

Example 2:

```
Input: preorder = [-1], inorder = [-1]
```

```
Output: [-1]
```

NORMAL APPROACH

```
class Solution {
    int[] P, I;
    HashMap<Integer, Integer> hm;
    int Pind, len;
    public TreeNode buildTree(int[] P, int[] I) {
        this.P = P; this.I = I; this.len = P.length;
        this.hm = new HashMap();
        for (int i = 0; i < I.length; i++)    hm.put(I[i], i);
        return build(0, len - 1);
    }
    private TreeNode build(int st, int end){
        if (st > end) return null;
        int CurVal = P[Pind++];
        TreeNode node = new TreeNode(CurVal);
        if (st == end) return node;
        int split = hm.get(CurVal);
        node.left = build(st, split - 1);
        node.right = build(split + 1, end);
        return node;
    }}
```

ITERATIVE APPROACH and it's extension below:

```
public TreeNode buildTreeIterative(int[] pre, int[] in) {
      if (pre.length == 0) return null;
      Stack<TreeNode> stack = new Stack<>();
      TreeNode root = new TreeNode(pre[0]);
      stack.push(root);
      var preOrderIndex = 1, inOrderIndex = 0;
      while (!stack.isEmpty()) {
            TreeNode top = stack.peek();
            TreeNode node;
            if (top.val != in[inOrderIndex]) {
// Top node in the stack has not yet encountered its counterpart in inOrder,
// so next element in pre must be left child of this node
                  node = new TreeNode(pre[preOrderIndex++]);
                  top.left = node;
            } else {
// if all the elements in inOrder have been visited, we are done
                  if (++inOrderIndex == in.length)
                        break;
                  stack.pop();
// Check if still unvisited nodes in the left subtree of the top node in the stack
                  if (!stack.isEmpty() && (stack.peek().val == in[inOrderIndex]))
                        continue;
// As top node in stack still has not encountered its counterpart
// in inOrder, so next element in pre must be right child of the removed node
                  node = new TreeNode(pre[preOrderIndex++]);
                  top.right = node;
            }
            stack.push(node);
```

```
        }
        return root; }
```

Other recursion way

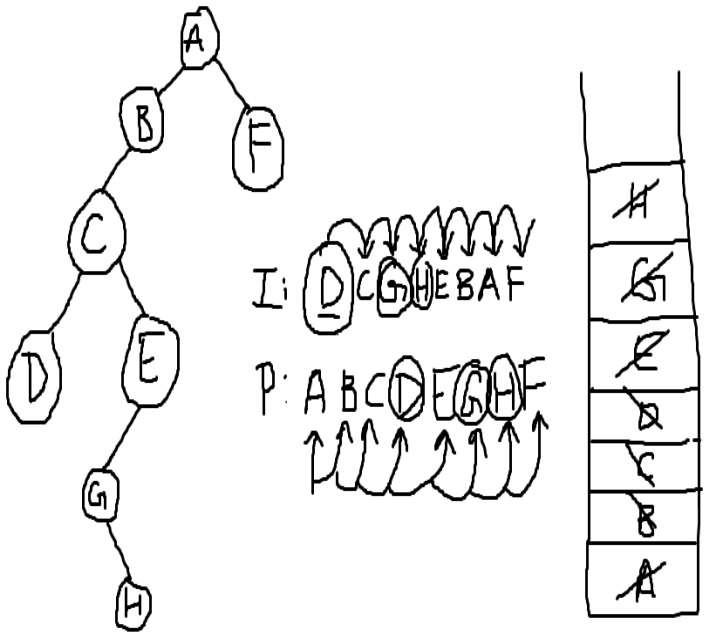
```
class Solution {
    public TreeNode buildTree(int[] P, int[] I) {
        Map<Integer, Integer> M = new HashMap<>();
        for (int i = 0; i < I.length; i++)
            M.put(I[i], i);
        return splitTree(P, M, 0, 0, I.length-1);
    }
    private TreeNode splitTree(int[] P, Map<Integer, Integer> M, int pix, int ileft,
int iright) {
        int rval = P[pix], imid = M.get(rval);
        TreeNode root = new TreeNode(rval);
        if (imid > ileft)
            root.left = splitTree(P, M, pix+1, ileft, imid-1);
        if (imid < iright)
            root.right = splitTree(P, M, pix+imid-ileft+1, imid+1, iright);
        return root;
    }}
```

Simplest solution.

```
class Solution {
int[] preorder, inorder;
int index;
public TreeNode buildTree(int[] preorder, int[] inorder) {
    this.preorder = preorder;
    this.inorder = inorder;
    this.index = 0;
    return fun(0, inorder.length - 1);
}
public TreeNode fun(int l, int r) {
    if(l > r) {
        return null;
    }
    int mid = find(preorder[index++], l, r);
    return new TreeNode(inorder[mid], fun(l, mid - 1), fun(mid + 1, r));
}
public int find(int num, int l, int r) {
    for(int i = l; i <= r; i++) {
        if(num == inorder[i]) {
            return i;
        }
    }
    return -1;
}}
```

I see quite a few recursive solutions with good explanations. Posts with an iterative version though were scanty, and I couldn't find many resources online either. So here's how I did it: I took an example, this one in fact, wrote the inorder traversal, and then the preorder traversal, and observed the pattern and that's how I got it. Cuz I see a lot of people wondering how one arrived at such a solution, well it took me hours to figure out the pattern but here's what I figured out finally.

So you start off with a stack, push the root into it and your inorderIndex is set to 0. Then you iterate from preorderIndex 1 to length of preorder traversal(because root is already in the stack, so preorderIndex 0 is taken care of).So you start off with a stack, push the root into it and your inorderIndex is set to 0.
Then you iterate from preorderIndex 1 to length of preorder traversal(because root is already in the stack, so preorderIndex 0 is taken care of).



You check whether the stack's top and inorder[inorderIndex] are the same, and while that's true, you keep popping(and storing the popped TreeNode in some variable, say prev, because we'll need it) while incrementing inorderIndex.
The point is that since they were equal, this means that you're out of nodes to insert towards the left, and now you're going to have to insert at the right. But right of what? The last popped node, which is stored in prev.
So after you're done popping from the stack, you check if you actually popped anything out(by checking if prev==null). If you didn't, then you're still able to insert trees at the left. So you insert a new node(preorder[preorderIndex]) to the left of stack's top's node. Otherwise, you insert to the right of the prev node that popped. And then you're done. Here's the code to clarify further:

```
class Solution {
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        ArrayDeque<TreeNode> stack = new ArrayDeque<>();
        TreeNode root = new TreeNode(preorder[0]);
        int inorderIndex = 0;
        stack.push(root);

        for(int i = 1; i < preorder.length; i++){
            TreeNode prev = null;
            while(!stack.isEmpty() && stack.peek().val == inorder[inorderIndex]){
                prev = stack.pop();
                inorderIndex++;
```

```
            }
            TreeNode toInsert = new TreeNode(preorder[i]);
            if(prev == null){
                prev = stack.peek();
                prev.left = toInsert;
            }
            else{
                prev.right = toInsert;
            }
            stack.push(toInsert);
        }
        return root;
    }
}
```

VARIATION Construct Binary Tree from Inorder and Postorder Traversal

```
class Solution {
    int PostInd, len;
    HashMap<Integer, Integer> map;
    int[] in, post;
    public TreeNode buildTree(int[] inorder, int[] postorder) {
        this.len = inorder.length;
        this.PostInd = len - 1;
        this.map = new HashMap();
        this.in = inorder;
        this.post = postorder;
        for (int i = 0; i < len; i++){
            map.put(inorder[i], i);
        }
        return build(0, len - 1);
    }

    private TreeNode build(int s, int e){
        if (s > e){  return null; }

        TreeNode node = new TreeNode(post[PostInd--]);
        if (s == e) return node;
        int getInInd = map.get(node.val);

        node.right = build(getInInd + 1, e);
        node.left = build(s, getInInd - 1);
        return node;
    }}
```

Q. Why do we need to build the right first rather than left?

A. Since it's post-order, it's left right root. So we have to build right first.

B. Post order traversal order is Left, Right, Root.

Postorder: 9, 15, 7, 20, 3. **ie**, L ->R-> Root

Inorder: 3, 20, 7, 15, 9 **i.e.** Root->R->L

Instead of reversing, we can start from the end of the postorder array.

- So we get the root of the Tree first from postorder, which is 3.

- Then the postorder Index is incremented and the next one is 20 which is right of the root.

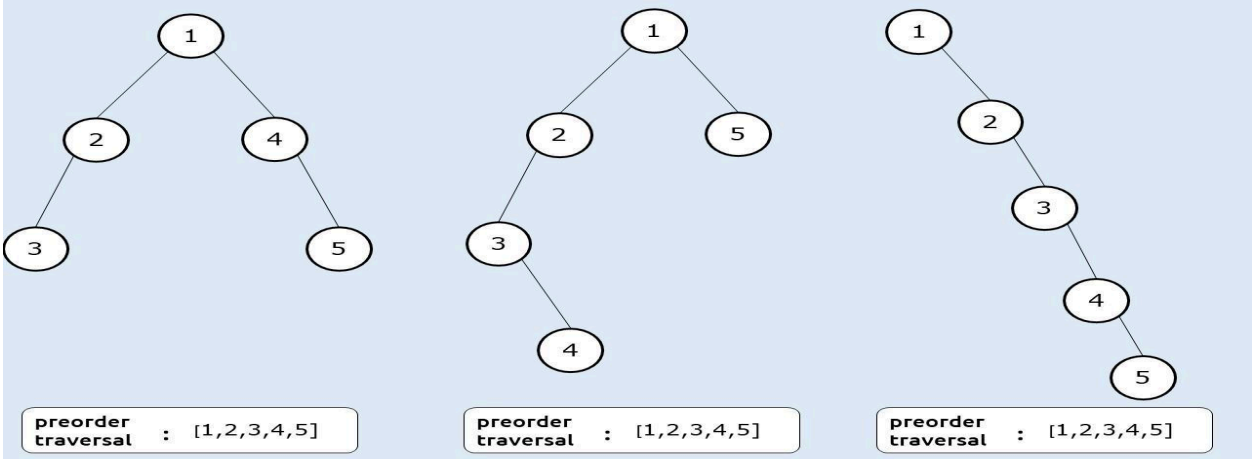
Therefore, we find right first as per postorder array from the end.

28. How to create different BT using different traversals

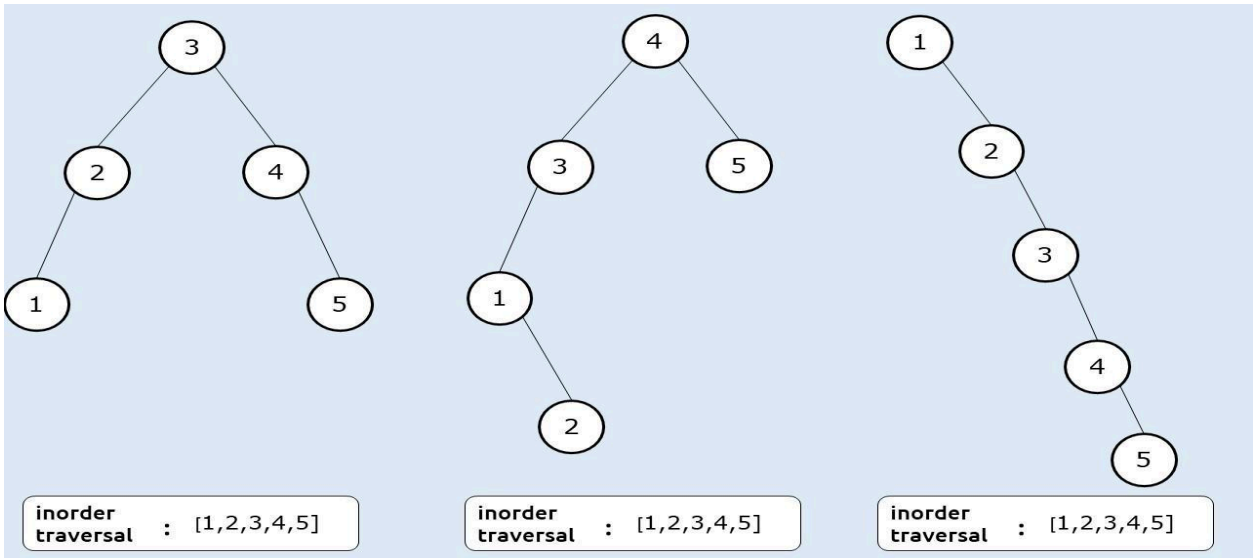
Problem Statement: Requirements are needed to construct a unique binary tree. In this article, we will understand the traversals required to construct a **unique** binary tree.

Constructing A Unique Binary tree using a single traversal.

If we are given any traversal, be it postorder, preorder, or inorder; it is not possible to construct a unique binary tree.
For example: For a preorder traversal: [1 2 3 4 5], all the following trees can be constructed.



For an inorder traversal : [1 2 3 4 5], all the following trees can be constructed.

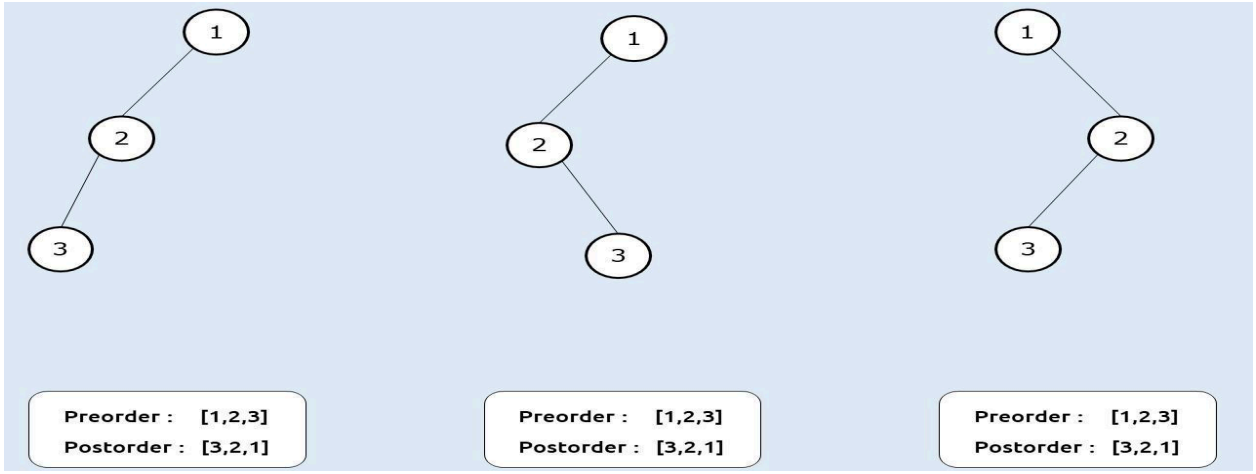


Conclusion: We **cannot** construct a unique binary tree with just one traversal.
Constructing A Unique Binary tree using two traversals.
Case 1: Using Preorder and Postorder Traversal Given:

Preorder : [1,2,3]

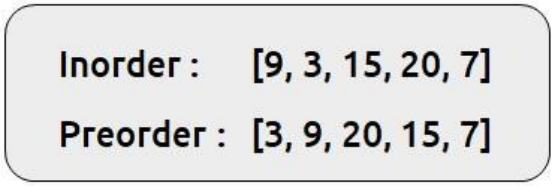
Postorder : [3,2,1]

We see that the following different trees can be constructed using the above two traversals:



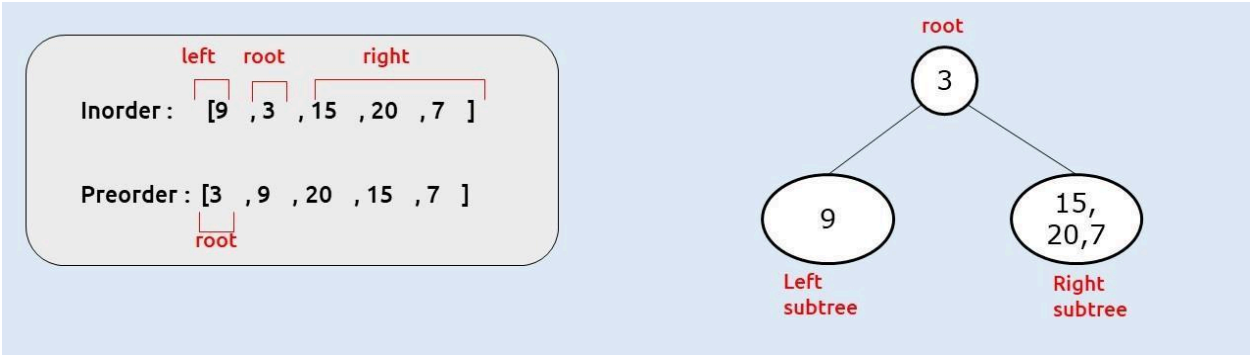
Conclusion: We **cannot** construct a unique binary tree using preorder and postorder traversal.

Case 2: Using preorder and inorder traversal Given:

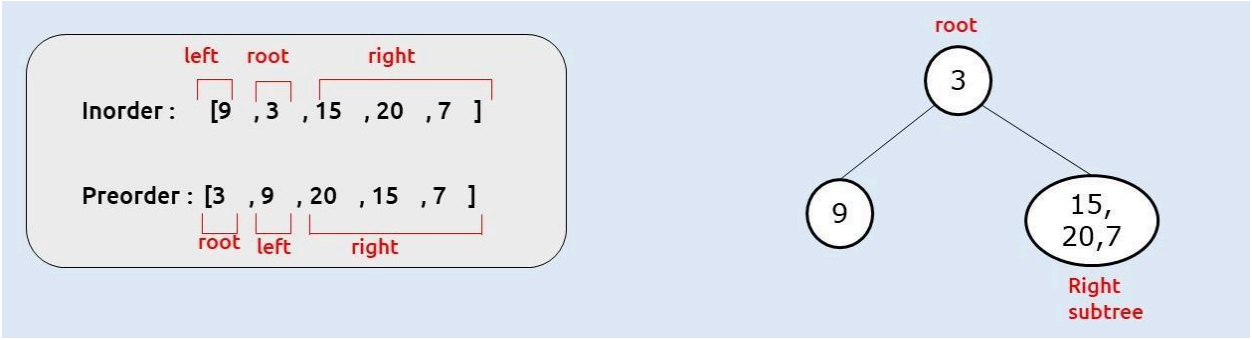


We will try to construct a binary tree using the above two traversals.

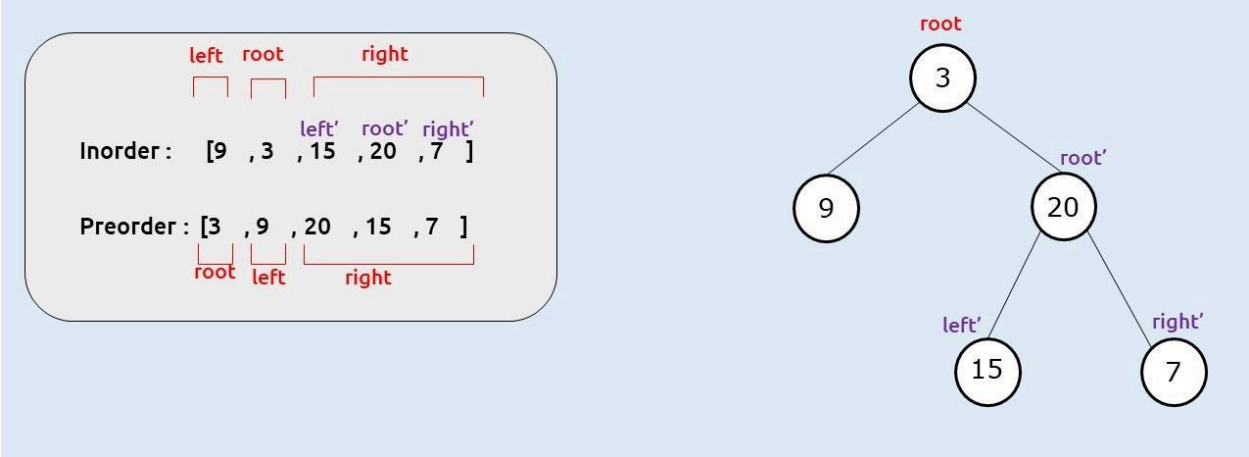
Step 1: We know that the first element of a preorder traversal (root, left, right) is the root of the tree therefore, we can say that 3 is the root of the tree. We can then find 3 in the inorder traversal and find the left subtree and right subtree of node 3 from it as inorder traversal is (left, root, right).



Step 2: As 9 is the only element to the left of 3 in inorder traversal, we can conclude that the left subtree of node 3 contains a single node 9. Preorder traversal also contains node 9 as the second element. Therefore the remaining nodes in the preorder traversal will be the right subtree of node 3.



Step 3: Now, we will work on the right subtree of node 3. We know that is given by the last three elements of preorder traversal. As discussed in step 1, the first node will give us the root of the right subtree, then we can find that element (20) in the inorder traversal. Elements left to element 20 will give us the left subtree and elements right to it will give the right subtree of node 20.



Conclusion: We **can** construct a unique binary tree using inorder and preorder traversal.

Similarly, it can be shown that a unique binary tree can be constructed using inorder and postorder traversal as in postorder traversal the root node is given as the last element. The special feature of the inorder traversal is that it helps us to identify a node and its left and right subtree. This is not provided by any other traversal therefore whenever we are provided in order traversal with one other traversal, we can easily construct a unique binary tree.

Key takeaways

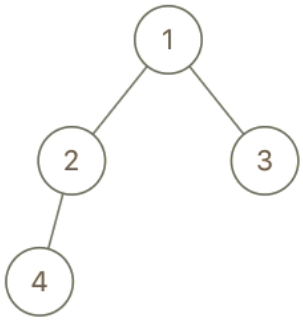
- We cannot construct a unique binary tree using only one traversal
- We cannot construct a unique binary tree using preorder and postorder traversal.
- We can construct a unique binary tree using inorder traversal and preorder traversal or using inorder and postorder traversal.

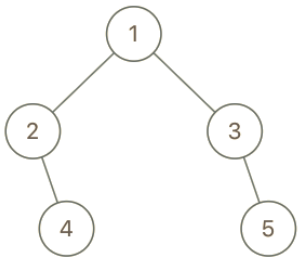
29. Cousins in Binary Tree

Given the `root` of a binary tree with unique values and the values of two different nodes of the tree `x` and `y`, return `true` *if the nodes corresponding to the values* `x` *and* `y` *in the tree are cousins, or* `false` *otherwise.*

Two nodes of a binary tree are cousins if they have the same depth with different parents.

Note that in a binary tree, the root node is at the depth `0`, and children of each depth `k` node are at the depth `k + 1`.





Example 1: `Input: root = [1,2,3,4], x = 4, y = 3 Output: false`

Example 2: `Input: root = [1,2,3,null,4,null,5], x = 5, y = 4 Output: true`

DFS Approach / Preorder traversal :

```
class Solution {
    TreeNode xParent, yParent;
    int xHt, yHt;
    public boolean isCousins(TreeNode root, int x, int y) {
        dfs(root, x, y, 0, null);
        return (xHt == yHt && xParent != yParent);
    }
    private void dfs(TreeNode root, int x, int y, int ht, TreeNode parent){
       if (root == null){  return;  }
       if (root.val == x){
            xHt = ht;
            xParent = parent;
            return ;
        } if (root.val == y){
            yHt = ht;
            yParent = parent;
            return ;
        }
        dfs(root.right, x, y, ht + 1, root);
        dfs(root.left, x, y, ht + 1, root);
    }}
```

BFS Approach

```
public boolean isCousins(TreeNode root, int A, int B) {
    if (root == null) return false;
```

```
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);
        while (!queue.isEmpty()) {
                int size = queue.size();
                boolean isAexist = false;
                boolean isBexist = false;
                for (int i = 0; i < size; i++) {
                        TreeNode cur = queue.poll();
                        if (cur.val == A) isAexist = true;
                        if (cur.val == B) isBexist = true;
                        if (cur.left != null && cur.right != null)
                        {
                        if (cur.left.val == A && cur.right.val == B){ return false;}
                        if (cur.left.val == B && cur.right.val == A) { return false; }
                        }
                        if (cur.left != null) {
                                queue.offer(cur.left);
                        }
                        if (cur.right != null) {
                                queue.offer(cur.right);
                        }
                }
                if (isAexist && isBexist)  return true;
        }
        return false;
}
```

Bad approach

```
class Node {
    TreeNode parentNode, node;
    int layer;
    public Node(TreeNode inputParentNode, TreeNode inputNode, int inputLayer) {
        parentNode = inputParentNode;
        node = inputNode;
        layer = inputLayer;
    } }
class Solution {
    public boolean isCousins(TreeNode root, int x, int y) {
        Deque<Node> q = new ArrayDeque<Node>();
        HashMap<Integer, Integer> nodeParentNode= new HashMap<Integer, Integer>();
        HashMap<Integer, Integer> nodeLayer= new HashMap<Integer, Integer>();
        q.add(new Node(null, root, 0));
        while (q.peek() != null) {
            Node cur = q.poll();
            int parentVal = 0;
            if (cur.parentNode != null) { parentVal = cur.parentNode.val;}
            nodeParentNode.put(cur.node.val, parentVal);
            nodeLayer.put(cur.node.val, cur.layer);
            if (cur.node.left != null) {
                 q.add(new Node(cur.node, cur.node.left, cur.layer+1));
            }if (cur.node.right != null) {
                q.add(new Node(cur.node, cur.node.right, cur.layer+1)); }
        }
        if (!nodeParentNode.containsKey(x)) { return false; }
```

```
        if (!nodeParentNode.containsKey(y)) { return false; }
        if (nodeParentNode.get(x) == nodeParentNode.get(y)) {return false;}
        if (nodeLayer.get(x) != nodeLayer.get(y)) { return false; }
        return true;
    }
}
```

Other approach

```
class Solution {
    public boolean isCousins(TreeNode root, int x, int y) {
        return (findParent(root,x,-1) != findParent(root,y,-1)) &&
                            (findLevel(root, x, 1) == findLevel(root,y,1));
    }
    public int findLevel(TreeNode root, int data, int level) {
        if (root == null) return 0
        if (root.val == data) return level;
        int left = findLevel(root.left, data, level + 1);
        if (left != 0) return left;
        return findLevel(root.right, data, level + 1);
    }
    public int findParent(TreeNode root, int node, int parent){
        if (root == null)  return 0;
        if (root.val == node) return parent;

        int left = findParent(root.left, node, root.val);
        if (left != 0) return left;
        return findParent(root.right, node, root.val);
    }}
```

30.

Binary Search Tree:

1. Populate Next Right pointers of Tree

You are given a perfect binary tree where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {
  int val;
  Node *left, *right, *next; }
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL. Initially, all next pointers are set to NULL.

Example 1:

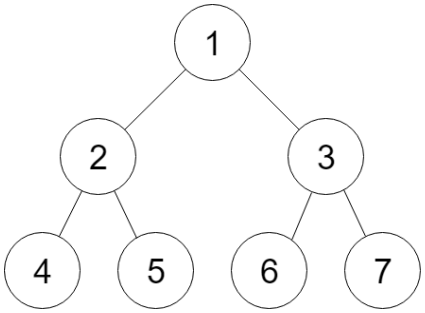


Figure A

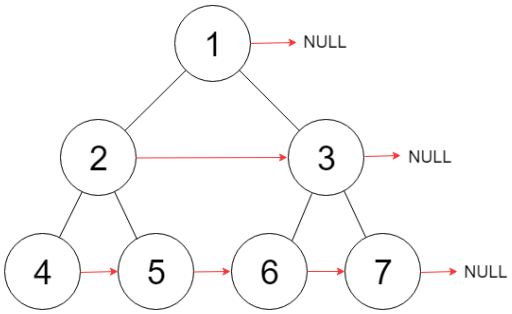


Figure B

Input: root = [1,2,3,4,5,6,7] Output: [1,#,2,3,#,4,5,6,7,#]
Explanation: Given the above perfect binary tree (Figure A), your function should populate each next pointer to point to its next right node, just like in Figure B. The serialized output is in level order as connected by the next pointers, with '#' signifying the end of each level.

Method - 1 recursive

```
class Solution {
    public Node connect(Node root) {
        if (root == null) return root;
        if (root.left != null) root.left.next = root.right;
        if (root.next != null && root.right != null) root.right.next =
root.next.left;
        connect(root.right); connect(root.left);
        return root; }}
```

Method - 2 iterative

```
public class Solution {
    public void connect(TreeLinkNode root) {
        if (root == null) { return; }
        for (TreeLinkNode head=root; head.left!=null; head=head.left) {
            for (TreeLinkNode parent=head; parent!=null; parent=parent.next) {
                parent.left.next = parent.right;
                if (parent.next != null) { parent.right.next = parent.next.left; }
            }
        }}}
```

```
// dfs iteratively
public void connect1(TreeLinkNode root) {
    Stack<TreeLinkNode> stack = new Stack<>();
    stack.push(root);
    while (!stack.isEmpty()) {
        TreeLinkNode n = stack.pop();
        if (n != null) {
            if (n.right != null) {
                n.left.next = n.right;
                if (n.next != null) {
                    n.right.next = n.next.left;
                }
```

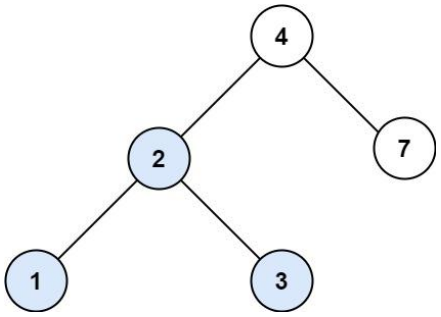
```
            }
            stack.push(n.right);
            stack.push(n.left);
        }
    }
}


// bfs iteratively
public void connect2(TreeLinkNode root) {
    Queue<TreeLinkNode> queue = new LinkedList<>();
    queue.add(root);
    while(!queue.isEmpty()) {
        TreeLinkNode n = queue.poll();
        if (n != null) {
            if (n.right != null) {
                n.left.next = n.right;
                if (n.next != null) {
                    n.right.next = n.next.left;
                }
            }
            queue.add(n.left);
            queue.add(n.right);
        }
    }
}

// dfs recursively
public void connect(TreeLinkNode root) {
    if (root != null) {
        if (root.right != null) {
            root.left.next = root.right;
            if (root.next != null) {
                root.right.next = root.next.left;
            }
        }
        connect(root.left);
        connect(root.right);
    }
}
```

2. Search given Key in BST

You are given the `root` of a binary search tree (BST) and an integer `val`.

Find the node in the BST that the node's value equals `val` and return the subtree rooted with that node. If such a node does not exist, return `null`.

Example 1:

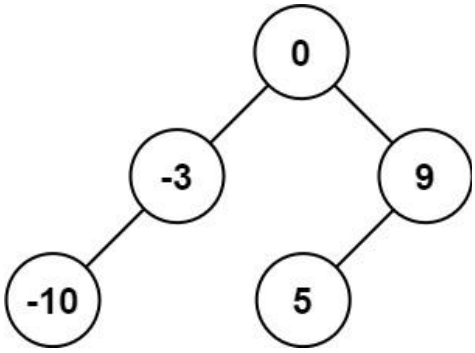



```
Input: root = [4,2,7,1,3], val = 2

Output: [2,1,3]

Method - 1 recursive soln.

class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        if (root == null || root.val == val) return root;
        return (root.val < val) ? searchBST(root.right , val) : searchBST(root.left ,
val);
    }
}

Method - 2 iterative soln.
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        TreeNode head = root;

        while (head != null) {
            if (head.val > val) {
                head = head.left;
            } else if (head.val < val) {
                head = head.right;
            } else {
                return head;
            }
        }

        return null; }}
```

3. Construct BST from given keys

Given an integer array `nums` where the elements are sorted in ascending order, convert *it to a height-balanced binary search tree*.A height-balanced binary tree is a binary tree in which the depth of the two subtrees of every node never differs by more than one.

Example 1:




```
Input: nums = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: [0,-10,5,null,-3,null,9] is also accepted:
```

```
Method - 1 using recursion
```

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return helper(nums, 0, nums.length - 1);}
    private TreeNode helper(int[] nums, int st, int end){
        if (st > end) return null;
        int md = (st + end )/2;
        TreeNode root = new TreeNode(nums[md]);
        root.left = helper(nums, st, md - 1);
        root.right = helper(nums, md + 1, end);
        return root; }}
```

```
Method - 2 using divide and conquer
```

```
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        if (nums.length == 0) return null;
        TreeNode node = new TreeNode();

        int mid = nums.length / 2;
        node.val = nums[mid];
        node.left = sortedArrayToBST(Arrays.copyOfRange(nums, 0, mid));
       node.right = sortedArrayToBST(Arrays.copyOfRange(nums, mid + 1, nums.length));
        return node;
    }}
```

```
Method - 3 using iteration and stack
public class Solution {
    class Node{ // need another class to store multi information
        int low, up; // means the TreeNode covers [low, up], low and up are all index
        TreeNode t;
        Node(int l, int p, TreeNode node){
            low = l;
            up = p;
            t = node;
        }
```

```
    }
    public TreeNode sortedArrayToBST(int[] num) {
        if(num == null || num.length == 0) return null;
        Stack<Node> stack = new Stack<Node>();
        // initialize
        TreeNode root = new TreeNode(num[(num.length-1)/2]);
        Node rootNode = new Node(0,num.length-1,root);
        stack.push(rootNode);
        // iteration
        while(!stack.isEmpty()){
            Node node = stack.pop();
            int middle = (node.low+node.up)/2; // cut half for [low, up]

            // [low, middle-1]
            if(middle-1 >= node.low){
                TreeNode leftnode = new TreeNode(num[(middle-1+node.low)/2]);
                node.t.left = leftnode;
                Node left = new Node(node.low, middle-1, leftnode);
                stack.push(left);
            }
            // [middle+1, up]
            if(middle+1 <= node.up){
                TreeNode rightnode = new TreeNode(num[(middle+1+node.up)/2]);
                node.t.right = rightnode;
                Node right = new Node(middle+1, node.up, rightnode);
                stack.push(right);
            }
        }
        return root; }}
```

4. Construct BST from preorder traversal

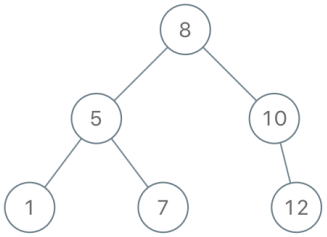
Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return *its root*.

It is guaranteed that there is always possible to find a binary search tree with the given requirements for the given test cases.

A binary search tree is a binary tree where for every node, any descendant of `Node.left` has a value strictly less than `Node.val`, and any descendant of `Node.right` has a value strictly greater than `Node.val`.

A preorder traversal of a binary tree displays the value of the node first, then traverses `Node.left`, then traverses `Node.right`.

Example 1:




```
Input: preorder = [8,5,1,7,10,12]
Output: [8,5,10,1,7,null,12]
```

```
Method - 1 recursive
```

```
class Solution {
    public TreeNode bstFromPreorder(int[] preorder) {
        if (preorder.length == 0) return null;
        TreeNode node = new TreeNode(preorder[0]);
        for (int i = 1; i < preorder.length; i++) buildTree(node, preorder[i]);
        return node;
    }
    private void buildTree(TreeNode node, int val){
        if (node.val < val){
            if (node.right == null) node.right = new TreeNode(val);
            else buildTree(node.right, val);
        }else{
            if (node.left == null) node.left = new TreeNode(val);
            else buildTree(node.left, val);
        } }}
```

```
Method - 2 iterative
```

```
class Solution {
    public TreeNode bstFromPreorder(int[] preorder) {
        TreeNode root = new TreeNode(preorder[0]);
        for (int i = 1; i < preorder.length; i++) {
            TreeNode curr = root, prev = null;
            while (curr != null) {
                prev = curr;
                if (preorder[i] > curr.val)
                    curr = curr.right;
                else curr = curr.left;
            }
            if (prev.val < preorder[i])
                prev.right = new TreeNode(preorder[i]);
            else prev.left = new TreeNode(preorder[i]);
        }
        return root; }}
```

THIS IS A CONTINUATION OF MY PREVIOUS POST!

WHICH HAD O(N^2) APPROACH!

ok let's do this!!

so we are given an array which is the preorder traversal of the tree!
We are used to traversing a tree but are not privy to reconstructing the tree from the array!!
anyways!!!
so we are given an array whose first element is the root of out tree!!(because of preorder traversal)!
BUT I strongly suggest you go through this solution below so that you can get the gist of the logic and then move on to the more complex linear solutions I posted!
LET'S DO THIS:
so we follow steps:
1>we create the node
2>we traverse the array for values which are less than the current node!-- these values will become our left subtree.we stop whenever we get a value larger than the current root of the subtree!
3>we take the rest of the array(values which are greater than the value of the current root)-these are the values which will make out the right subtree!
so we make a root!
make the left subtree(recursively)
then make right subtree(recursively)
code here!!
do a couple of dry runs!
u will get it!

AS A LOT OF PEOPLE ARE ASKING FOR LINEAR SOLUTIONS HERE IT IS!
APPROACH 1-O(N)
JAVA-LOWER AND UPPER BOUND RECURSIVE

```
class Solution {
    int nodeIndex;
    public TreeNode bstFromPreorder(int[] preorder) {
        if(preorder == null) {
            return null;
        }
        nodeIndex = 0;
        return bstHelper(preorder, Integer.MIN_VALUE , Integer.MAX_VALUE);
    }
    private TreeNode bstHelper(int[] preorder, int start, int end) {
        if(nodeIndex == preorder.length || preorder[nodeIndex]<start ||
preorder[nodeIndex]>end) {
            return null;
        }
        int val = preorder[nodeIndex++];
        TreeNode node = new TreeNode(val);
        node.left = bstHelper(preorder, start, val);
        node.right = bstHelper(preorder, val, end);
        return node;
```

```
    }
}
```

APPROACH 2- O(N)!

JAVA ONLY UPPER BOUND- RECURSIVE

EXPLANATION-

Every node has an upper bound.

Left node is bounded by the parent node's value.

Right node is bounded by the ancestor's bound.

Using the example in the question:

The nodes [5, 1, 7] are all bounded by 8.

The node 1 is bounded by 5.

8 is the root node, but if you think deeper about it, it is bounded by Integer.MAX_VALUE. i.e. imagine there is a root parent node Integer.MAX_VALUE with left node being 8.

This also means that both 10 and 12 nodes, which are also right nodes, are also bounded by Integer.MAX_VALUE.

We use a recursive function together with an outer index variable i to traverse and construct the tree. When we create a tree node, we increment i to process the next element in the preorder array.

We don't need to care about lower bound. When we construct the tree, we try to create left node first. If the condition fails (i.e. current number is greater than the parent node value), then we try to create the right node which automatically satisfies the condition, hence no lower bound is needed

```
class Solution {
int i = 0;
    public TreeNode bstFromPreorder(int[] arr) {
        return helper(arr, Integer.MAX_VALUE);
    }

    public TreeNode helper(int[] arr, int bound) {
        if (i == arr.length || arr[i] > bound) return null;
        TreeNode root = new TreeNode(arr[i++]);
        root.left = helper(arr, root.val);
        root.right = helper(arr, bound);
        return root;
    }
       }
       ```

EXPLANATON-
"explanation- It is possible to do this because when we construct the " left child "
the upper bound will be the node value itself and no lower bound will be needed!
```
```

```
      -no lower bound is required for "right child" because we have arrived at this
point of creating the right child only because these elements failed to satisfy the
left subtree conditions!"
```

APPROACH 3-IF YOU ARE NOT COMFORTABLE WITH RECURSION!

JAVA ITERATIVE APPROACH -

```
class Solution {
    public TreeNode bstFromPreorder(int[] preorder) {

        if(preorder == null){
            return null;
        }
        int size = preorder.length;
        if(size==0){
            return null;
        }
        TreeNode root = new TreeNode(preorder[0]);

        for(int i=1;i<size;i++){
             generateBST(preorder[i],root);
        }
        return root;
    }

    public void generateBST(int target, TreeNode tree){
        TreeNode root = tree;
        TreeNode node = new TreeNode(target);
            while(root!=null){
                if(target<root.val){
                    if(root.left==null){
                        root.left = node;
                        break;
                    }
                    else{
                         root=root.left;
                    }
                }else{
                    if(root.right==null){
                        root.right=node;
                        break;
                    }else{
                        root=root.right;
                    }
                } }}}
```

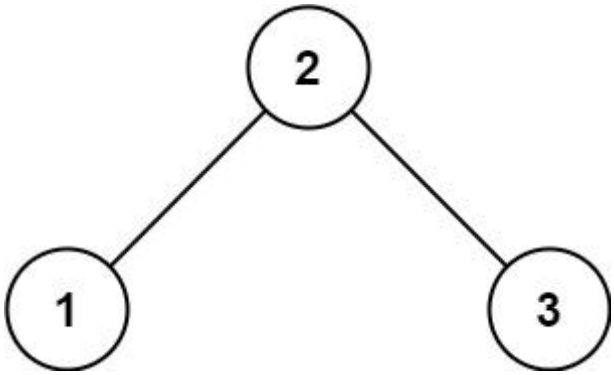
5. **Check is a BT is BST or not**

Given the root of a binary tree, *determine if it is a valid binary search tree (BST)*.

A valid BST is defined as follows:

- The left subtree of a node contains only nodes with keys less than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:




```
Input: root = [2,1,3]
Output: true
```

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return isValid(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }
    private boolean isValid(TreeNode root, long l, long m){
        if (root == null) return true;
        if (root.val <= l || root.val >= m) return false;
        boolean rt = isValid(root.right, root.val, m);
        boolean lt = isValid(root.left, l, root.val);
        return rt && lt;
    }}
```

```
private boolean isValidBSTviaDFS(TreeNodeRange root) {
      Stack<TreeNodeRange> nodes = new Stack<>();
            nodes.push(root);
            while (!nodes.isEmpty()) {
                  TreeNodeRange currentNode = nodes.pop();
                  if (!currentNode.isWithinRange()) {
                        return false;
                  }
                  if (currentNode.hasLeftChild()) {
                        nodes.push(currentNode.leftChild());
                  }
                  if (currentNode.hasRightChild()) {
                        nodes.push(currentNode.rightChild());
                  }
```

```
            }

            return true;
      }

      private boolean isValidBSTviaBFS(TreeNodeRange root) {
            Queue<TreeNodeRange> nodes = new LinkedList<>();
            nodes.offer(root);

            while (!nodes.isEmpty()) {
                  TreeNodeRange currentNode = nodes.poll();
                  if (!currentNode.isWithinRange()) {
                        return false;
                  }
                  if (currentNode.hasLeftChild()) {
                        nodes.offer(currentNode.leftChild());
                  }
                  if (currentNode.hasRightChild()) {
                        nodes.offer(currentNode.rightChild());
                  }
            }
            return true;
      }
```

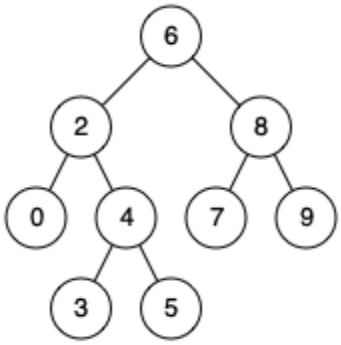
```
public boolean isValidBST(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        Stack<TreeNode> stack = new Stack<>();
        TreeNode curr = root;
        if(root == null) {
            return false;
        }
        while (curr != null || !stack.isEmpty()) {
            while (curr != null) {
                stack.push(curr);
                curr = curr.left;
            }
            curr = stack.pop();
            res.add(curr.val);
            curr = curr.right;
        }

        for(int i = 1; i < res.size(); i++) {
            if(res.get(i) <= res.get(i-1) )
                return false;
        }
        return true;
    }
```

6. Find LCA of two nodes in BST

Given a binary search tree (BST), find the lowest common ancestor (LCA) of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow a node to be a descendant of itself).”

Example 1:




```
Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
Output: 6
Explanation: The LCA of nodes 2 and 8 is 6.
```

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) return root;

        if (root.val > p.val && root.val > q.val)
                    return lowestCommonAncestor(root.left, p, q);
        if (root.val < p.val && root.val < q.val)
                    return lowestCommonAncestor(root.right, p, q);
        else return root;
    }
}
```

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode cur = root;
        while (cur != null) {
            if (p.val < cur.val && q.val < cur.val) cur = cur.left;
            else if (p.val > cur.val && q.val > cur.val) cur = cur.right;
            else break;
            }
        return cur; }}
```

7. Find the inorder predecessor/successor of a given Key in BST.

There is BST given with root node with key part as integer only. You need to find the inorder successor and predecessor of a given key. In case, if the either of predecessor or successor is not found print -1.

Input:

The first line of input contains an integer T denoting the number of test cases. Then T test cases follow. Each test case contains n denoting the number of edges of the BST. The next line contains the edges of the BST. The last line contains the key.

Output: Print the predecessor followed by successor for the given key. If the predecessor or successor is not found print -1.

Constraints:

1<=T<=100

1<=n<=100

1<=data of node<=100

1<=key<=100

Example:

Input:

6

50 30 L 30 20 L 30 40 R 50 70 R 70 60 L 70 80 R

65

Output:

60 70

```
public static void findPreSuc(Node root, Res p, Res s, int key)
    {
        if(root==null)return;

        if(key>root.data){
            p.pre = root;
            findPreSuc(root.right,p,s,key);
        }
        else if(key<root.data){
            s.succ = root;
            findPreSuc(root.left,p,s,key);
        }
        else{
            findPreSuc(root.left,p,s,key);
            findPreSuc(root.right,p,s,key);
        }}
```

8. Delete Node in a BST

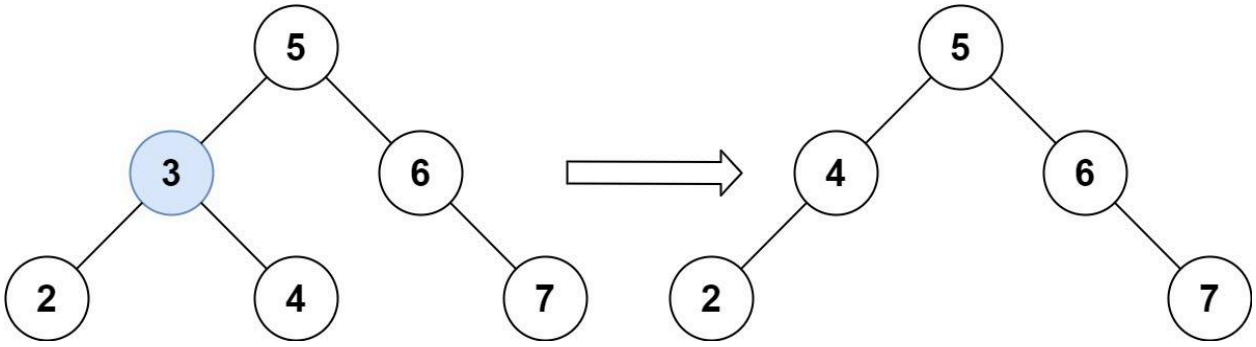
Given a root node reference of a BST and a key, delete the node with the given key in the BST.

Return the root node reference (possibly updated) of the BST.

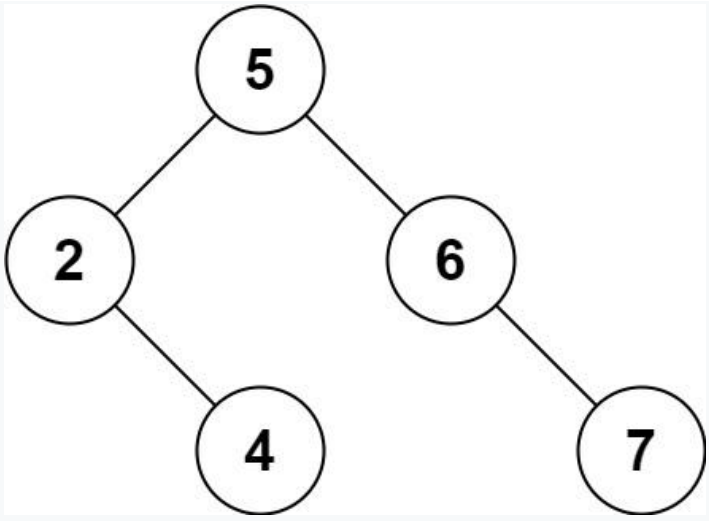
Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

Example 1:




```
Input: root = [5,3,6,2,4,null,7], key = 3
Output: [5,4,6,2,null,null,7]
Explanation: Given key to delete is 3. So we find the node with value 3 and delete
it.
One valid answer is [5,4,6,2,null,null,7], shown in the above BST.
Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also
accepted.
```



Example 2: `Input: root = [5,3,6,2,4,null,7], key = 0 Output: [5,3,6,2,4,null,7] Explanation: The tree does not contain a node with value = 0.`

```
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null){
            return root;
        }
        if (root.val > key){
            root.left = deleteNode(root.left, key);
        }else if (root.val < key){
            root.right = deleteNode(root.right, key);
        }else{
            if (root.left == null){
                return root.right;
            }else if (root.right == null){
                return root.left;
            }else{
                TreeNode node = root.right;
                while (node.left != null){
                    node = node.left;
                }
                root.val = node.val;
                root.right = deleteNode(root.right, node.val);
            }
        }
        return root; }}
```

9. **Floor in a BST**
10. **Ceil in a BST**
11. **Find K-th smallest element in BST**
12. **Find K-th largest element in BST**
13. **Find a pair with a given sum in BST**
14. **BST iterator**
15. **Size of the largest BST in a Binary Tree**
16. **Serialize and deserialize Binary Tree**
17. **BIT MANIPULATION:**

1. Gray Code

An n-bit gray code sequence is a sequence of 2^n integers where:

- Every integer is in the inclusive range $[0, 2^n - 1]$,
- The first integer is 0,
- An integer appears no more than once in the sequence,
- The binary representation of every pair of adjacent integers differs by exactly one bit, and
- The binary representation of the first and last integers differs by exactly one bit.

Given an integer n, return *any valid n-bit gray code sequence*.

Example 1:

```
Input: n = 2
Output: [0,1,3,2]
Explanation:
The binary representation of [0,1,3,2] is [00,01,11,10].
- 00 and 01 differ by one bit
- 01 and 11 differ by one bit
- 11 and 10 differ by one bit
- 10 and 00 differ by one bit
Output for n = 3 [0,1,3,2,6,7,5,4] and n = 4 [0,1,3,2,6,7,5,4,12,13,15,14,10,11,9,8]
```

Method - 1 using common sense about the comparing with n = 3 and n = 2

```
class Solution {
    public List<Integer> grayCode(int n) {
        List<Integer> ans = new ArrayList();
        ans.add(0);
        for (int i = 0; i < n; i++){
            int msb = 1 << i;
            for (int j = msb - 1; j >= 0; j--){
                ans.add(ans.get(j) | msb);
            }
        }
```

```
        return ans; }}
```

Method - 2 Using recursion

```
class Solution {
    public List<Integer> grayCode(int n) {
        if (n == 0) return Arrays.asList(0);
        List<Integer> prev = grayCode(n - 1);
        List<Integer> ans = new ArrayList<>(prev);
        int pow = 1 << (n - 1);
        for (int i = pow - 1; i >= 0; i--){
            int num = (prev.get(i) | pow);
            ans.add(num);
        }
        return ans; }}
```

I. DESIGN Q'S:

1. LRU Cache

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the `LRUCache` class:

- `LRUCache(int capacity)` Initialize the LRU cache with positive size `capacity`.
- `int get(int key)` Return the value of the `key` if the key exists, otherwise return `-1`.
- `void put(int key, int value)` Update the value of the `key` if the `key` exists. Otherwise, add the `key-value` pair to the cache. If the number of keys exceeds the `capacity` from this operation, evict the least recently used key.

The functions `get` and `put` must each run in `O(1)` average time complexity.

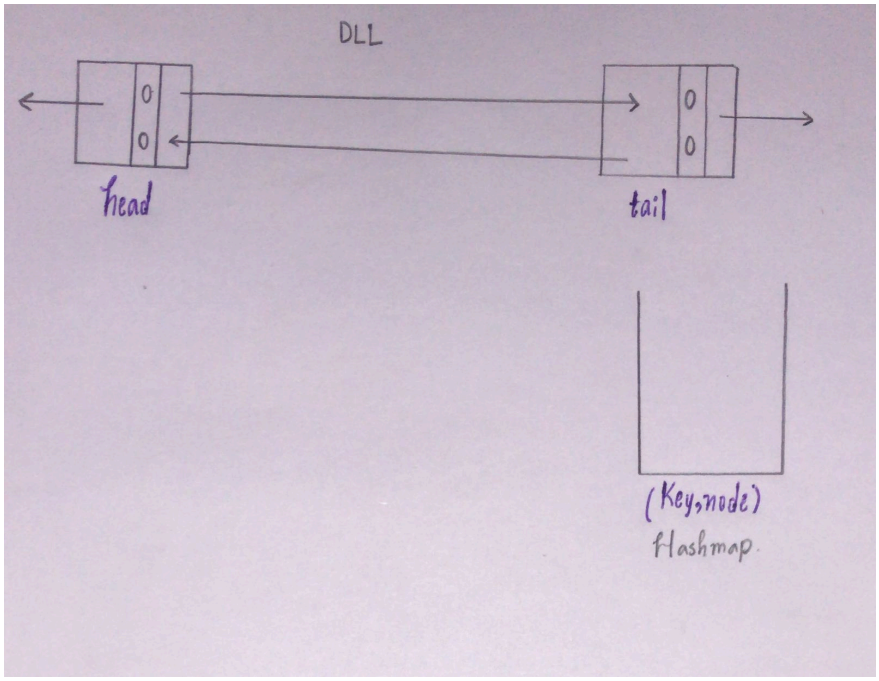
Example 1:

```
Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // cache is {1=1}
lRUCache.put(2, 2); // cache is {1=1, 2=2}
lRUCache.get(1);    // return 1
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lRUCache.get(2);    // returns -1 (not found)
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
lRUCache.get(1);    // return -1 (not found)
lRUCache.get(3);    // return 3

lRUCache.get(4);    // return 4
```

Intuition:

While inserting the {key,val} pair into the **DDL** make sure that we are inserting it from the back tail to head.
The cache will tell us when the {key, value} pair is used/inserted.

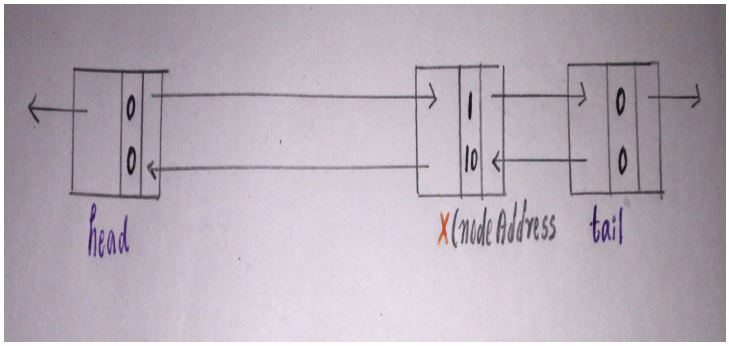
Approach:
Size = 3
put(1,10) put(3,15) put(2,12) get(3) put(4,25)
Create a DLL and hashmap

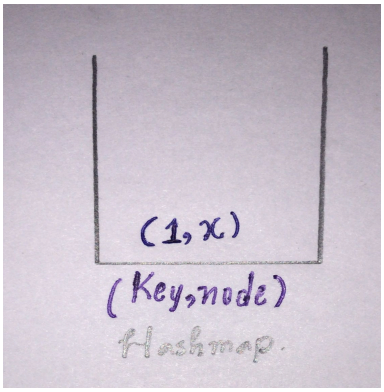


While inserting the {key,val}pair consider the following

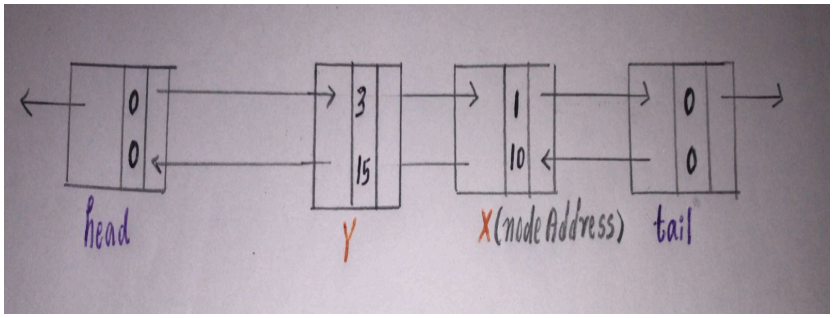
- Check if the {key,val} pair is already present in Cache.
- Check for the capacity. if the Cache size == capacity then while inserting the new pair remove the **LRU** and insert the **new pair right after the head**.while removing the node make sure to remove the {value, node} pair from the cache.
- If the key is not present in the Cache then return -1;

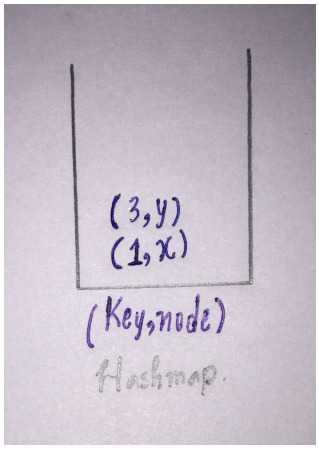
Query 1: put(1,10) Initialling Cache is empty so (n = 0) and (1,10) are not present in the Cache. It is a new element so we should insert it into the Cache.
Since n < capacity(0 < 3), so we can insert the pair.
Insert the pair({1,10}) right after the head.And store the **{key,address of the node x}** in the Cache.



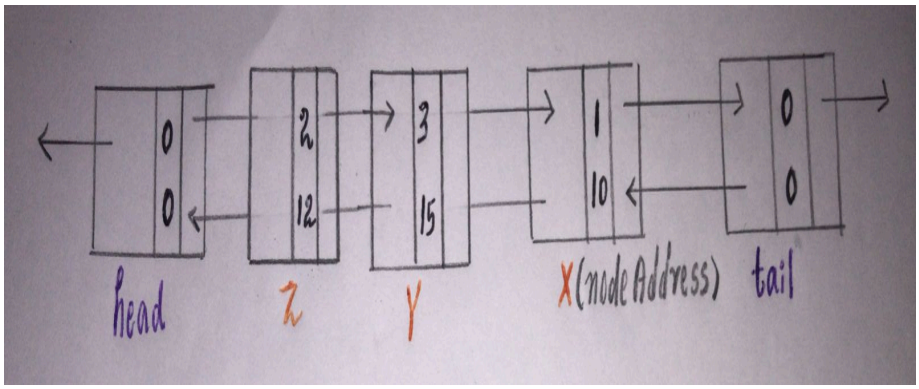


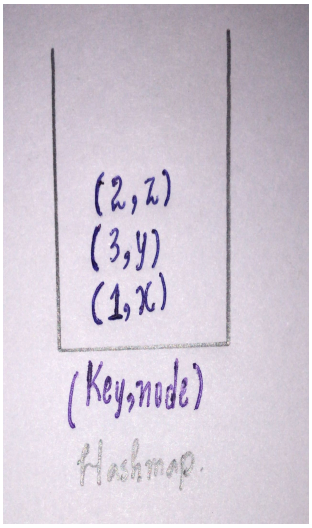
Query 2: put(3,15)
Repeat the same process.
(3,15) is not present in the Cache and the size of the Cache (n = 1) is less than capacity 3.
Insert the pair(3,10) right after the head and store the address of that node in the Cache with its key value.





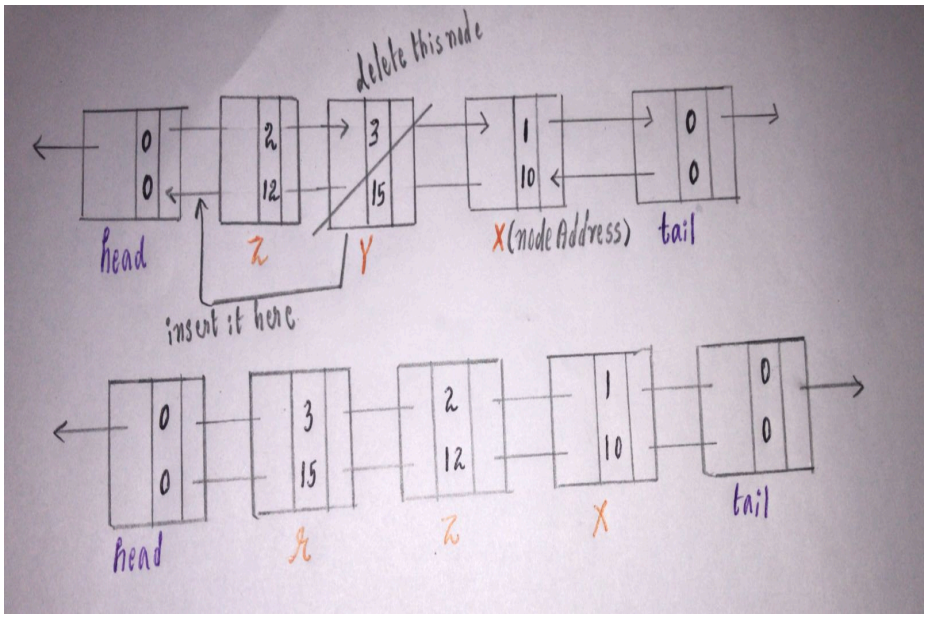
Query 3: put(2,12)
Repeat again. check if (2,12) Is present in the Cache.
No, it's not present in the cache so now check for the size Cache size is 2 < capacity(3).so insert the pair right after the head.

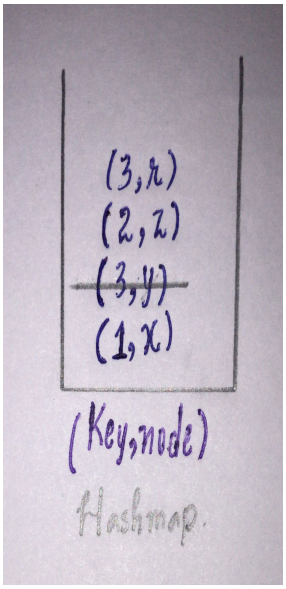




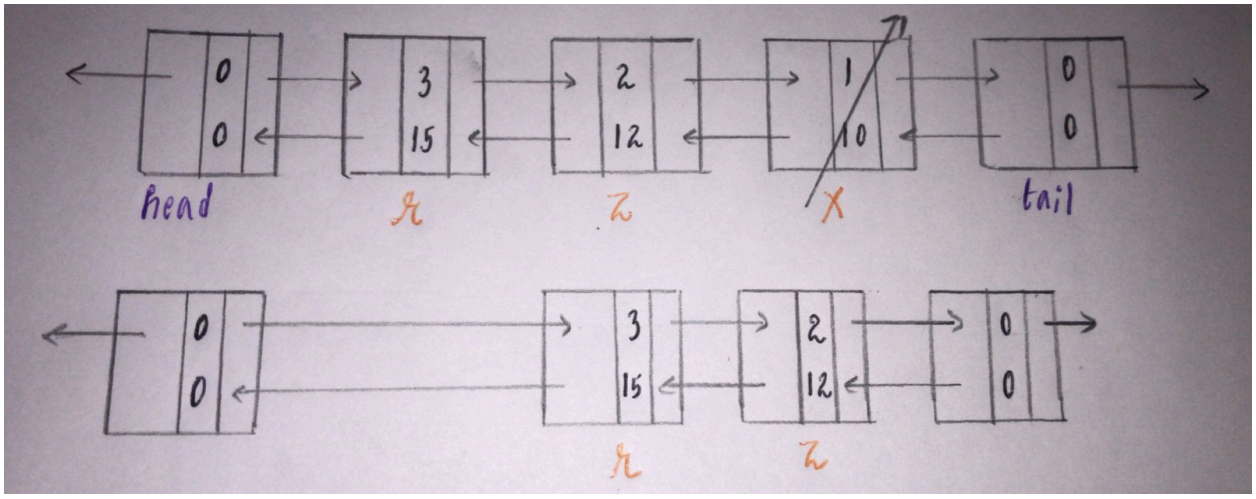
Query 4: get(3) Check if the value is present in the cache. Yes, 3 is present in the cache so take the address of the node(Y) and output the value which is present in that node.

Now the most recently used should be changed because the flow is from head(most recent) to tail(least recent).
Delete the node and add that node right after the head. Since the node is moved to a new address update the address of the key in the cache.

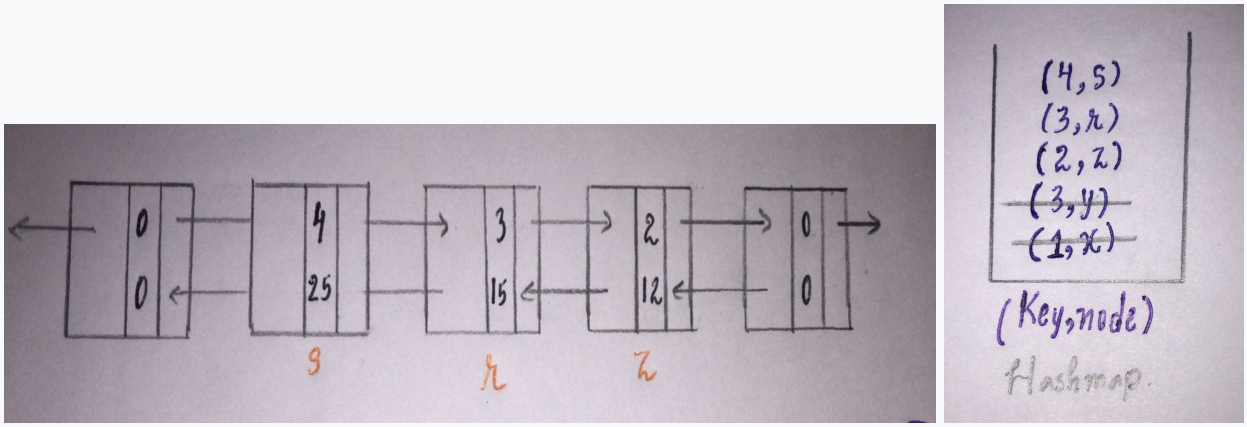




Query 5: put(4,25)
Check for the size of the cache n = 3 which is equal to capacity so we have to remove the LRU which is present right before the tail.
Before removing the node delete the pair{key, node} of that node present in the cache.



Now the size of the cache will be 2 now we have space to insert the pair(4,25).
Take (4,25) right after the head and take the address of the node(s) and insert it into the cache.



METHOD - 1 Using doubly linked list

```
class LRUCache {
	Node head = new Node();
	Node tail = new Node();

	int capacity;
	Map<Integer, Node> nodeMap = new HashMap<>();

	public LRUCache(int capacity) {
	    this.capacity = capacity;
	    head.next = tail;
	    tail.prev = head;
	}

	public int get(int key) {
```

```
        if(nodeMap.containsKey(key)) {
            removeNode(nodeMap.get(key));
            Node newNode = new Node(key, nodeMap.get(key).value);
            moveNodeToHead(newNode);
            nodeMap.put(key, newNode);
            return nodeMap.get(key).value;
        } else {
            return -1;
        }
    }

    public void put(int key, int value) {
        if(!nodeMap.containsKey(key)) {
            if(nodeMap.size() == capacity) {
                nodeMap.remove(tail.prev.key);
                removeNode(tail.prev);
            }
        } else {
            removeNode(nodeMap.get(key));
        }
        Node newNode = new Node(key, value);
        moveNodeToHead(newNode);
        nodeMap.put(key, newNode);
    }

    public void moveNodeToHead(Node newNode) {
        Node temp = head.next;
        head.next = newNode;
        newNode.prev = head;
        newNode.next = temp;
        temp.prev = newNode;
    }

    public void removeNode(Node temp) {
        temp.prev.next = temp.next;
        temp.next.prev = temp.prev;
    }
}

class Node {
    int key, value;
    Node prev, next;
    Node(int key, int value) {
        this.key = key;
        this.value = value;
    }
    Node() {
        this(0,0);
```

```
        }
}
```

Method - 2 using linkedHashMap

```
public class LRUCache {

    private LinkedHashMap<Integer, Integer> cache = new LinkedHashMap<>();
    private int cap;
    public LRUCache(int capacity) {
        this.cap = capacity;
    }
    public int get(int key) {
        if (cache.containsKey(key)){
            moveToRecent(key);
            return cache.get(key);
        }
        return -1;
    }

    public void put(int key, int val) {
        if (cache.containsKey(key)) {
            cache.put(key, val);
            moveToRecent(key);
            return ;
        }
        if (cache.size() >= cap) {
            RemoveTop();
        }
        cache.put(key, val);
    }

    private void moveToRecent(int key){
        int val = cache.get(key);
        cache.remove(key);
        cache.put(key, val);
    }

    private void RemoveTop(){
        for (int k: cache.keySet()){
            cache.remove(k);
            return ; }}}
```

2. Design a Browser History

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- BrowserHistory(string homepage) Initializes the object with the homepage of the browser.
- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

```
Input:

["BrowserHistory","visit","visit","visit","back","back","forward","visit","forwar
d","back","back"]

[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["l
inkedin.com"],[2],[2],[7]]

Output:

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.co
m","google.com","leetcode.com"]



Explanation:

BrowserHistory browserHistory = new BrowserHistory("leetcode.com");

browserHistory.visit("google.com");       // You are in "leetcode.com". Visit
"google.com"

browserHistory.visit("facebook.com");     // You are in "google.com". Visit
"facebook.com"

browserHistory.visit("youtube.com");      // You are in "facebook.com". Visit
"youtube.com"

browserHistory.back(1);                   // You are in "youtube.com", move back
to "facebook.com" return "facebook.com"

browserHistory.back(1);                   // You are in "facebook.com", move back
to "google.com" return "google.com"

browserHistory.forward(1);                // You are in "google.com", move
forward to "facebook.com" return "facebook.com"

browserHistory.visit("linkedin.com");     // You are in "facebook.com". Visit
"linkedin.com"

browserHistory.forward(2);                // You are in "linkedin.com", you
cannot move forward any steps.
```

```
browserHistory.back(2);                   // You are in "linkedin.com", move back
two steps to "facebook.com" then to "google.com". return "google.com"

browserHistory.back(7);                   // You are in "google.com", you can
move back only one step to "leetcode.com". return "leetcode.com"
```

Solution implementing the doubly linkedlist

```
class DLL{
    String val;
    DLL next;
    DLL prev;
    DLL(String val){
        this.val = val;
    }
}
class BrowserHistory {
    DLL DList;
    public BrowserHistory(String homepage) {
        this.DList = new DLL(homepage);
    }

    public void visit(String url) {
        DLL newPage = new DLL(url);
        DList.next = newPage;
        newPage.prev = DList;
        DList = DList.next;
    }

    public String back(int steps) {
        int i = 0;
        while (i++ < steps && DList.prev != null){
            DList = DList.prev;
        }
        return DList.val;
    }

    public String forward(int steps) {
        // print();
        int i = 0;
        while (i++ < steps && DList.next != null){
            DList = DList.next;
        }
        return DList.val;
    }
    // private void print(){
    //     List<String> ans = new ArrayList();
    //     DLL newNode = head;
    //     while (newNode != null){
```

```
    //          ans.add(newNode.val);
    //          newNode = newNode.next;
    //      }
    //      System.out.println(ans);
    // }
}
```

3. Design Twitter

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the `10` most recent tweets in the user's news feed.

Implement the `Twitter` class:

- `Twitter()` Initializes your twitter object.
- `void postTweet(int userId, int tweetId)` Composes a new tweet with ID `tweetId` by the user `userId`. Each call to this function will be made with a unique `tweetId`.
- `List<Integer> getNewsFeed(int userId)` Retrieves the `10` most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themself. Tweets must be ordered from most recent to least recent.
- `void follow(int followerId, int followeeId)` The user with ID `followerId` started following the user with ID `followeeId`.

- `void unfollow(int followerId, int followeeId)` The user with ID `followerId` started unfollowing the user with ID `followeeId`.

```
Input
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet", "getNewsFeed",
"unfollow", "getNewsFeed"]
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
Output
[null, null, [5], null, null, [6, 5], null, [5]]

Explanation
Twitter twitter = new Twitter();
twitter.postTweet(1, 5); // User 1 posts a new tweet (id = 5).
twitter.getNewsFeed(1);  // User 1's news feed should return a list with 1 tweet id
-> [5]. return [5]
twitter.follow(1, 2);    // User 1 follows user 2.
twitter.postTweet(2, 6); // User 2 posts a new tweet (id = 6).
twitter.getNewsFeed(1);  // User 1's news feed should return a list with 2 tweet ids
-> [6, 5]. Tweet id 6 should precede tweet id 5 because it is posted after tweet id
5.
twitter.unfollow(1, 2);  // User 1 unfollows user 2.
twitter.getNewsFeed(1);  // User 1's news feed should return a list with 1 tweet id
-> [5], since user 1 is no longer following user 2.
```

```java
public class Twitter {
	private static int timeStamp = 0;
	// easy to find if user exist
	private Map<Integer, User> userMap;

    // Tweet link to next Tweet so that we can save a lot of time
	// when we execute getNewsFeed(userId)
	private class Tweet{
		public int id;
		public int time;
		public Tweet next;
		public Tweet(int id){
			this.id = id;
			time = timeStamp++;
			next = null;
		}
	}
	// OO design so User can follow, unfollow and post itself
	public class User{
		public int id;
		public Set<Integer> followed;
```

```
            public Tweet tweet_head;

            public User(int id){
                  this.id=id;
                  followed = new HashSet<>();
                  follow(id); // first follow itself
                  tweet_head = null;
            }

            public void follow(int id){
                  followed.add(id);
            }

            public void unfollow(int id){
                  followed.remove(id);
            }

            // everytime user post a new tweet, add it to the head of tweet list.
            public void post(int id){
                  Tweet t = new Tweet(id);
                  t.next = tweet_head;
                  tweet_head = t;
            }
      }
      /** Initialize your data structure here. */
      public Twitter() {
            userMap = new HashMap<Integer, User>();
      }
      /** Compose a new tweet. */
      public void postTweet(int userId, int tweetId) {
            if(!userMap.containsKey(userId)){
                  User u = new User(userId);
                  userMap.put(userId, u);
            }
            userMap.get(userId).post(tweetId);
      }

      // Best part of this.
      // first get all tweets lists from one user including itself and all people it
followed.
      // Second add all heads into a max heap. Every time we poll a tweet with
      // largest time stamp from the heap, then we add its next tweet into the heap.
      // So after adding all heads we only need to add 9 tweets at most into this
      // heap before we get the 10 most recent tweet.
      public List<Integer> getNewsFeed(int userId) {
            List<Integer> res = new LinkedList<>();
            if (!userMap.containsKey(userId)) return res;

            Set<Integer> users = userMap.get(userId).followed;
            PriorityQueue<Tweet> q = new PriorityQueue<Tweet>(users.size(), (a,b) ->
(b.time - a.time));
            for (int user: users){
                  Tweet t = userMap.get(user).tweet_head;
                  // very imporant! If we add null to the head we are screwed.
                  if(t != null){ q.add(t);}
```

```
            }
            int n = 0;
            while (!q.isEmpty() && n < 10){
             Tweet t = q.poll();
             res.add(t.id);
             n++;
             if(t.next != null)
               q.add(t.next);
            }
            return res;
      }

      /** Follower follows a followee. If the operation is invalid, it should be a
no-op. */
      public void follow(int followerId, int followeeId) {
            if (!userMap.containsKey(followerId)){
                  User u = new User(followerId);
                  userMap.put(followerId, u);
            }
            if (!userMap.containsKey(followeeId)){
                  User u = new User(followeeId);
                  userMap.put(followeeId, u);
            }
            userMap.get(followerId).follow(followeeId);
      }

      /** Follower unfollows a followee. If the operation is invalid, it should be a
no-op. */
      public void unfollow(int followerId, int followeeId) {
            if (!userMap.containsKey(followerId) || followerId==followeeId)
                  return;
            userMap.get(followerId).unfollow(followeeId);
      }
}
```